

Object Oriented Programming (3-0-2-4)

Lecture 1

Types of Programming Methodologies

- Unstructured programming
- Procedural/Process oriented programming
 - Modular programming
- Object-oriented programming

Unstructured Programming

- Direct code (historically earliest programming paradigm)
 - Ex: Assembly Language

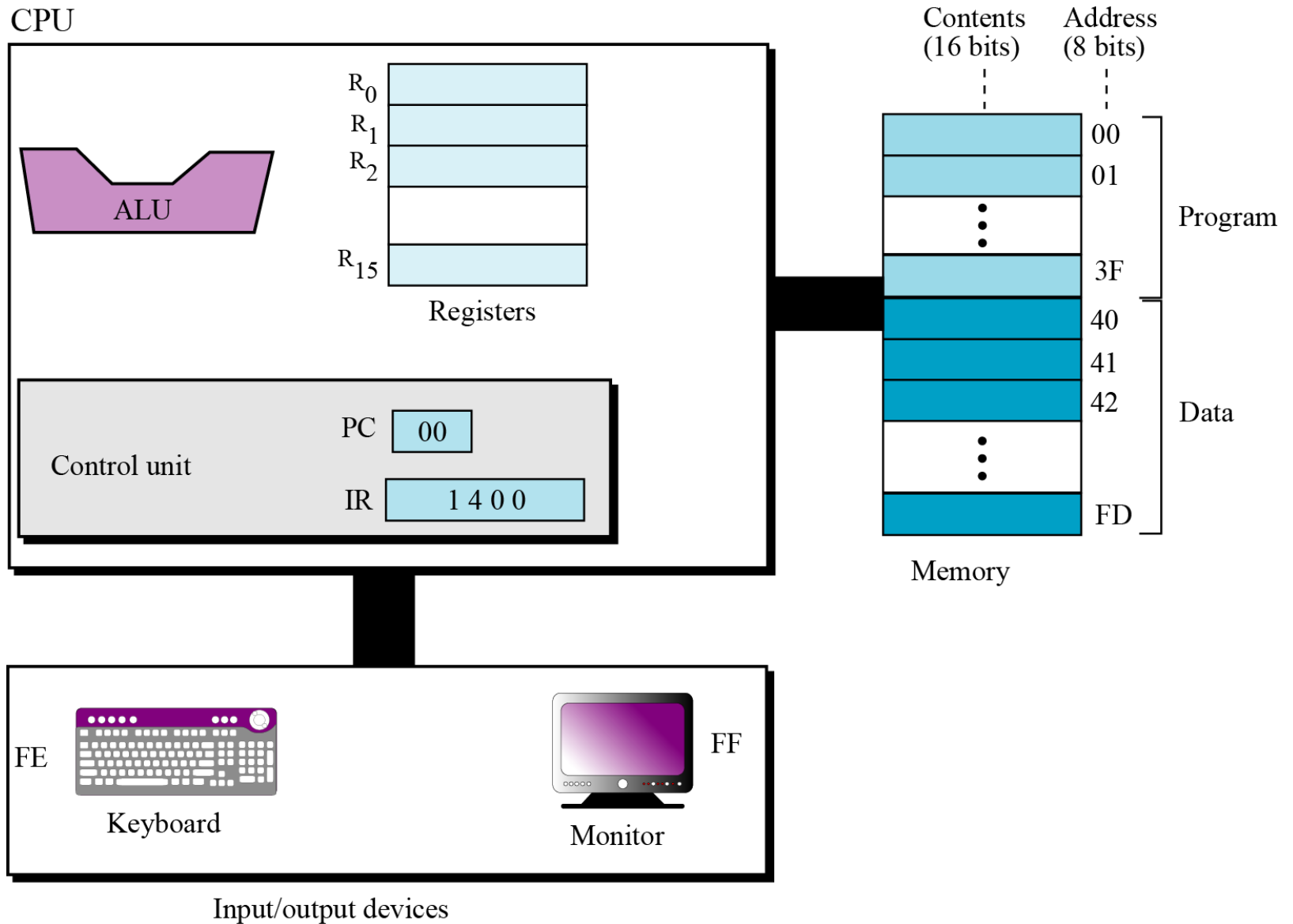


Figure: The components of a simple computer

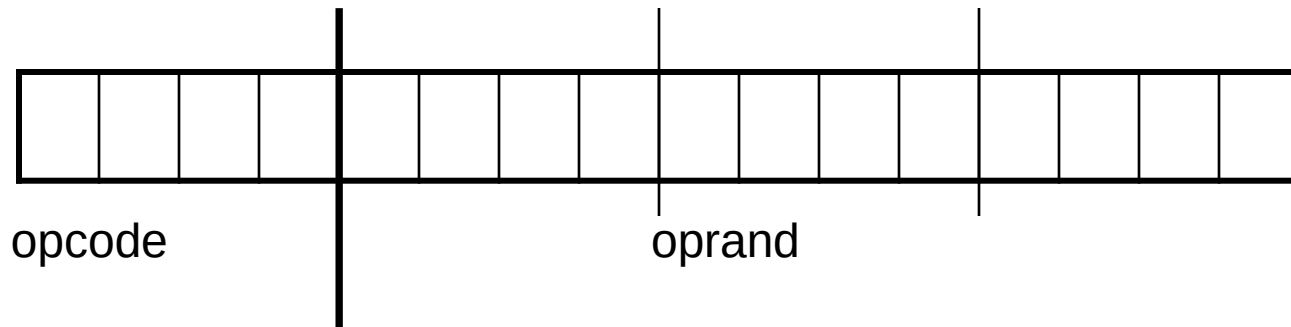
Instruction set

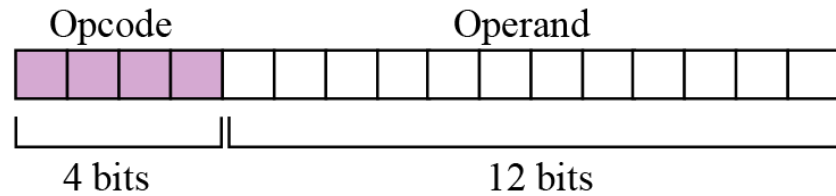
Each computer instruction consists of two parts: the **operation code (opcode)** and the **operand (s)**.

The opcode specifies the type of operation to be performed on the operand (s).

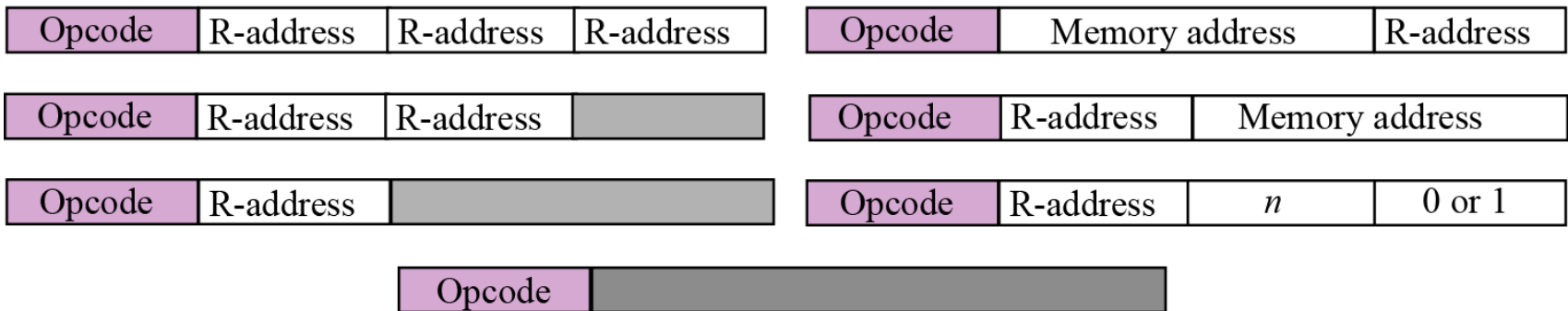
Each instruction consists of sixteen bits divided into four 4-bit fields.

The leftmost field contains the opcode and the other three fields contains the operand or address of operand (s), as shown in next Figure.





a. Instruction format



b. Instruction types

Figure: Format and different instruction types

R-address: address of a register

Memory address: Main memory address

Table List of instructions for the simple computer

Instruction	Code	Operands			Action
	d ₁	d ₂	d ₃	d ₄	
HALT	0				Stops the execution of the program
LOAD	1	R _D	M _S		R _D ← M _S
STORE	2	M _D		R _S	M _D ← R _S
ADDI	3	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} + R _{S2}
ADDF	4	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} + R _{S2}
MOVE	5	R _D	R _S		R _D ← R _S
NOT	6	R _D	R _S		R _D ← $\overline{R_S}$
AND	7	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} AND R _{S2}
OR	8	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} OR R _{S2}
XOR	9	R _D	R _{S1}	R _{S2}	R _D ← R _{S1} XOR R _{S2}
INC	A	R			R ← R + 1
DEC	B	R			R ← R – 1
ROTATE	C	R	n	0 or 1	Rot _n R
JUMP	D	R	n		IF R ₀ ≠ R then PC = n, otherwise continue

Key: R_S, R_{S1}, R_{S2}: Hexadecimal address of source registers
R_D: Hexadecimal address of destination register
M_S: Hexadecimal address of source memory location
M_D: Hexadecimal address of destination memory location
n: hexadecimal number
d₁, d₂, d₃, d₄: First, second, third, and fourth hexadecimal digits

An example

Add two integers A and B and create the result as C.

$$C = A + B$$

We assume that the first two integers are stored in memory locations $(40)_{16}$ and $(41)_{16}$ and the result should be stored in memory location $(42)_{16}$.

To do the simple addition needs five assembly language instructions, as shown next:

1. Load the contents of M_{40} into register R_0 ($R_0 \leftarrow M_{40}$).
2. Load the contents of M_{41} into register R_1 ($R_1 \leftarrow M_{41}$).
3. Add the contents of R_0 and R_1 and place the result in R_2 ($R_2 \leftarrow R_0 + R_1$).
4. Store the contents R_2 in M_{42} ($M_{42} \leftarrow R_2$).
5. Halt.

In the language of our simple computer, these five instructions are encoded as:

<i>Code</i>	<i>Interpretation</i>			
$(1040)_{16}$	1: LOAD	0: R_0	40: M_{40}	
$(1141)_{16}$	1: LOAD	1: R_1	41: M_{41}	
$(3201)_{16}$	3: ADDI	2: R_2	0: R_0	1: R_1
$(2422)_{16}$	2: STORE	42: M_{42}		2: R_2
$(0000)_{16}$	0: HALT			

Processing the instructions

Our simple computer, like most computers, uses machine cycles.

A cycle is made of three phases: **fetch**, **decode** and **execute**.

During the fetch phase, the instruction whose address is in PC is obtained from the memory and loaded into the IR. The PC is then incremented to point to the next instruction.

During the decode phase, the instruction in IR is decoded and the required operands are fetched from the register or from memory.

During the execute phase, the instruction is executed and the results are placed in the appropriate memory location or the register.

Once the third phase is completed, the control unit starts the cycle again, but now the PC is pointing to the next instruction. The process continues until the CPU reaches a HALT instruction.

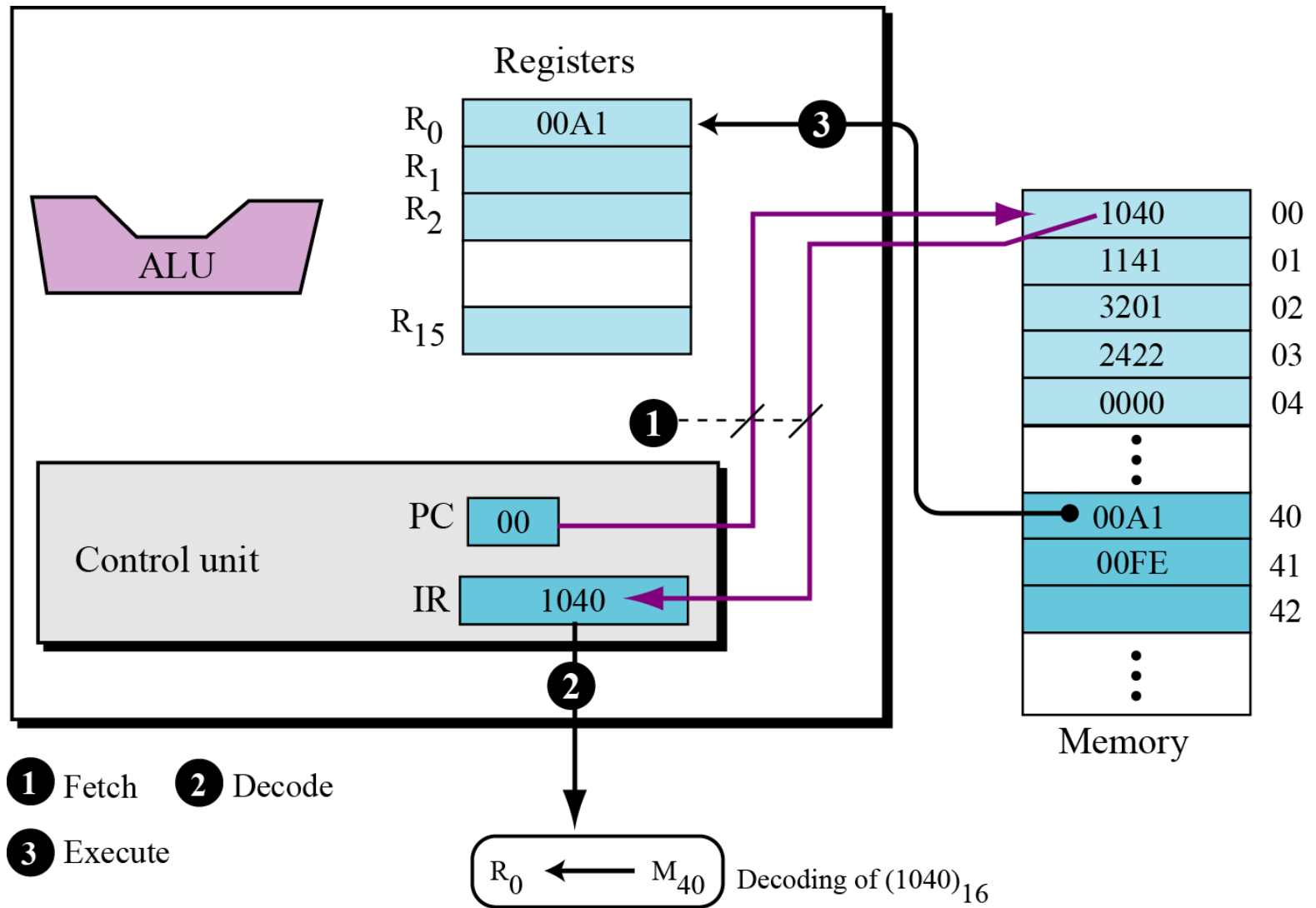


Figure Status of cycle 1

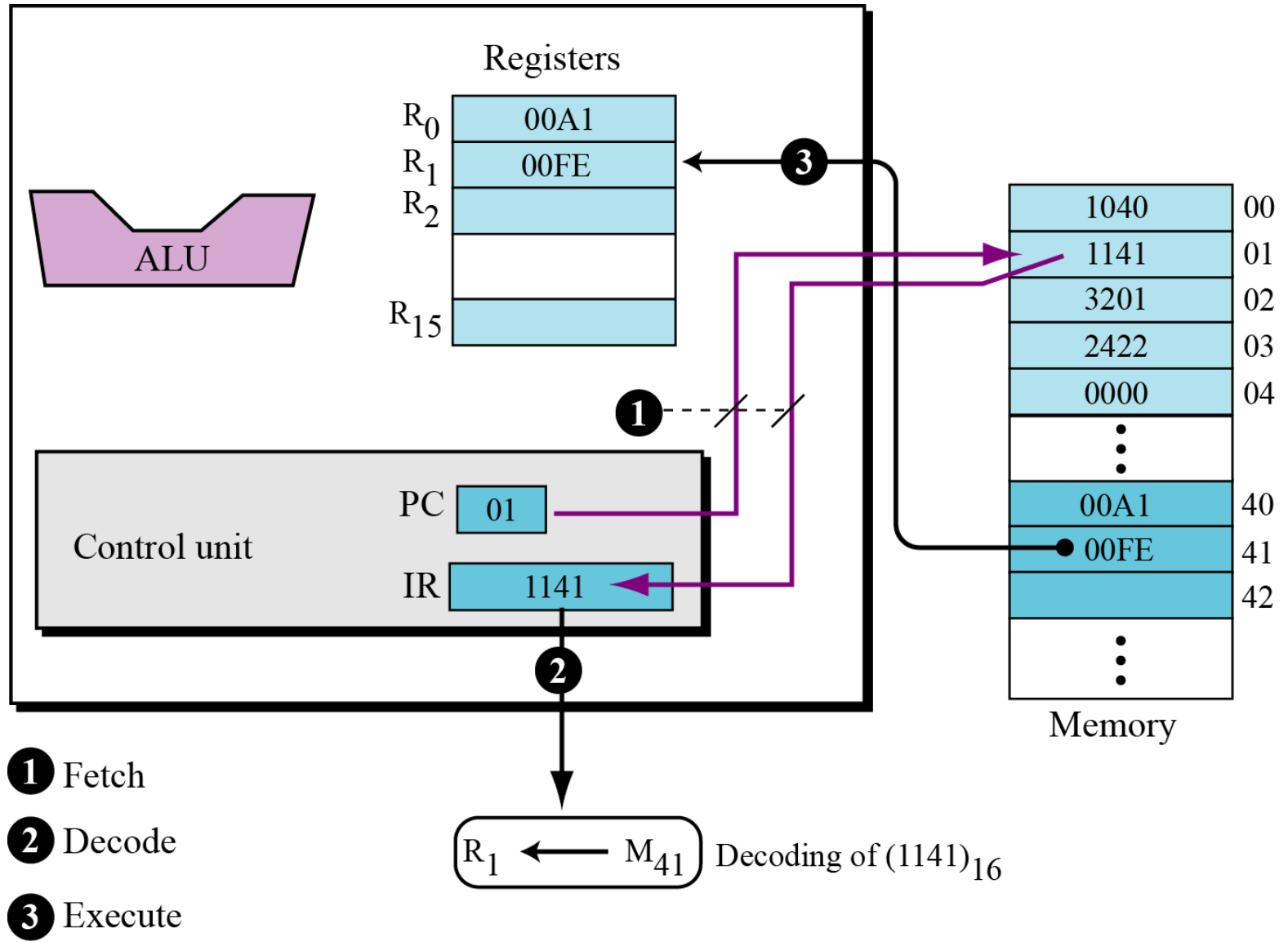


Figure Status of cycle 2

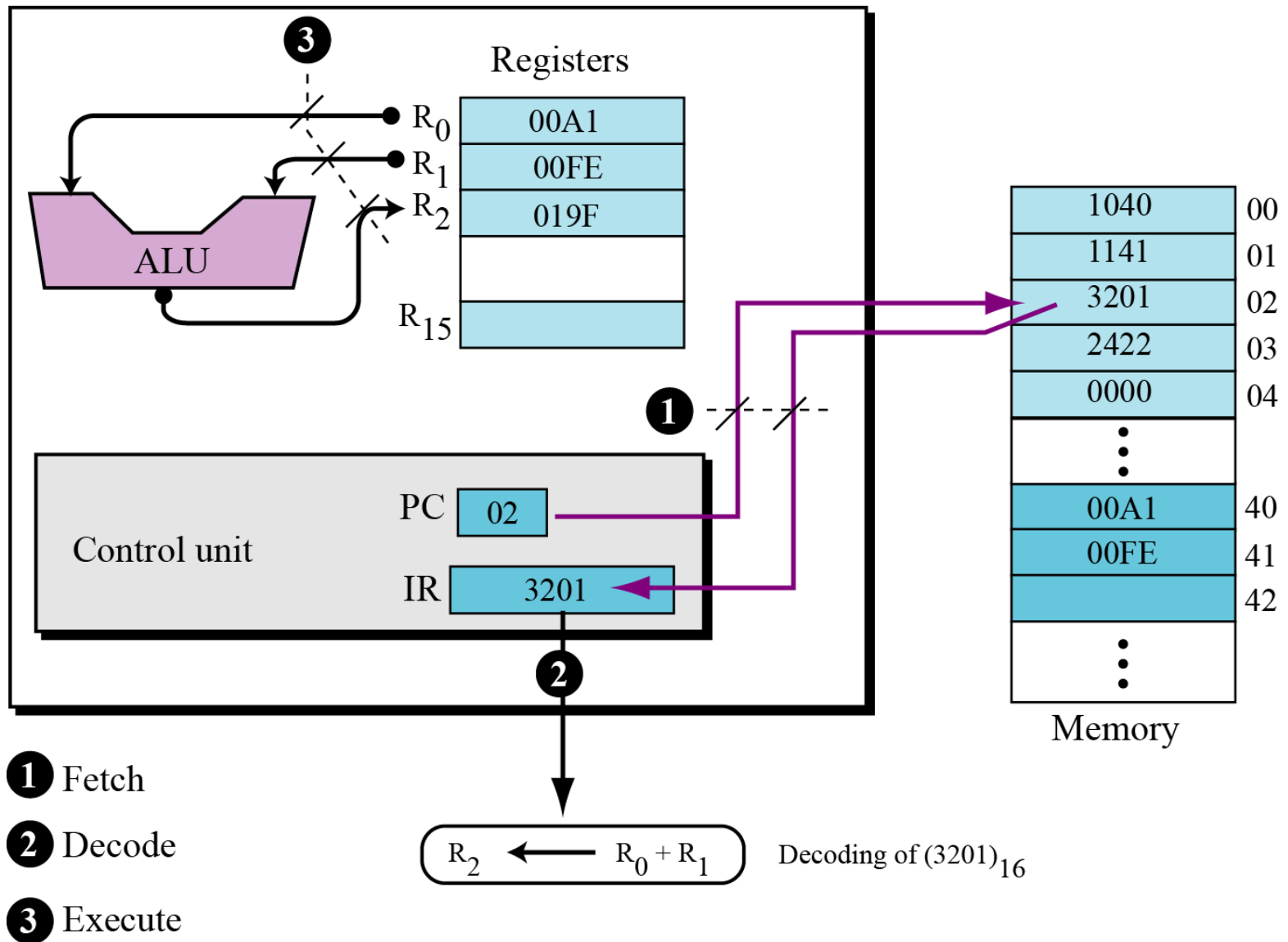


Figure: Status of cycle 3

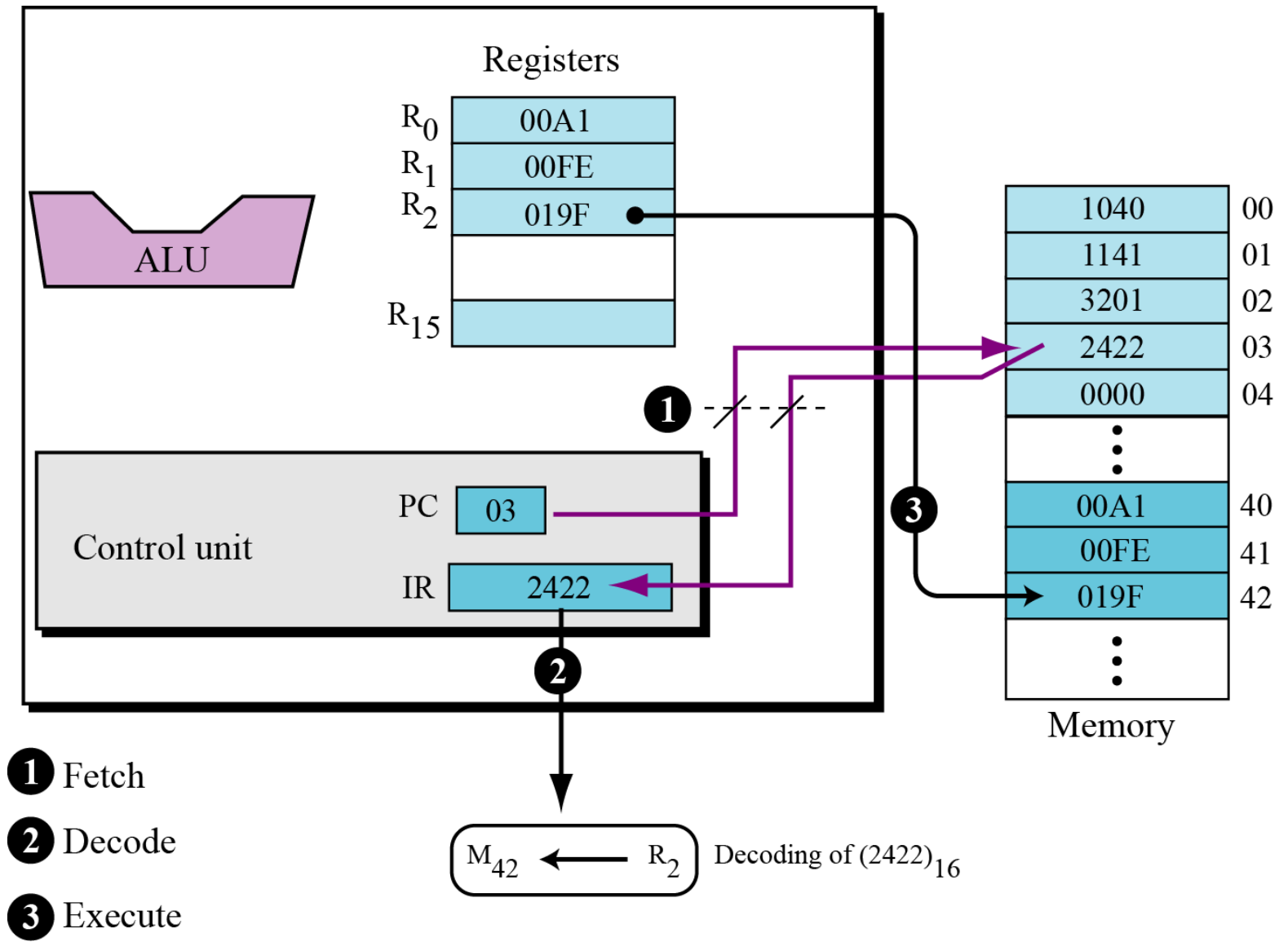


Figure: Status of cycle 4

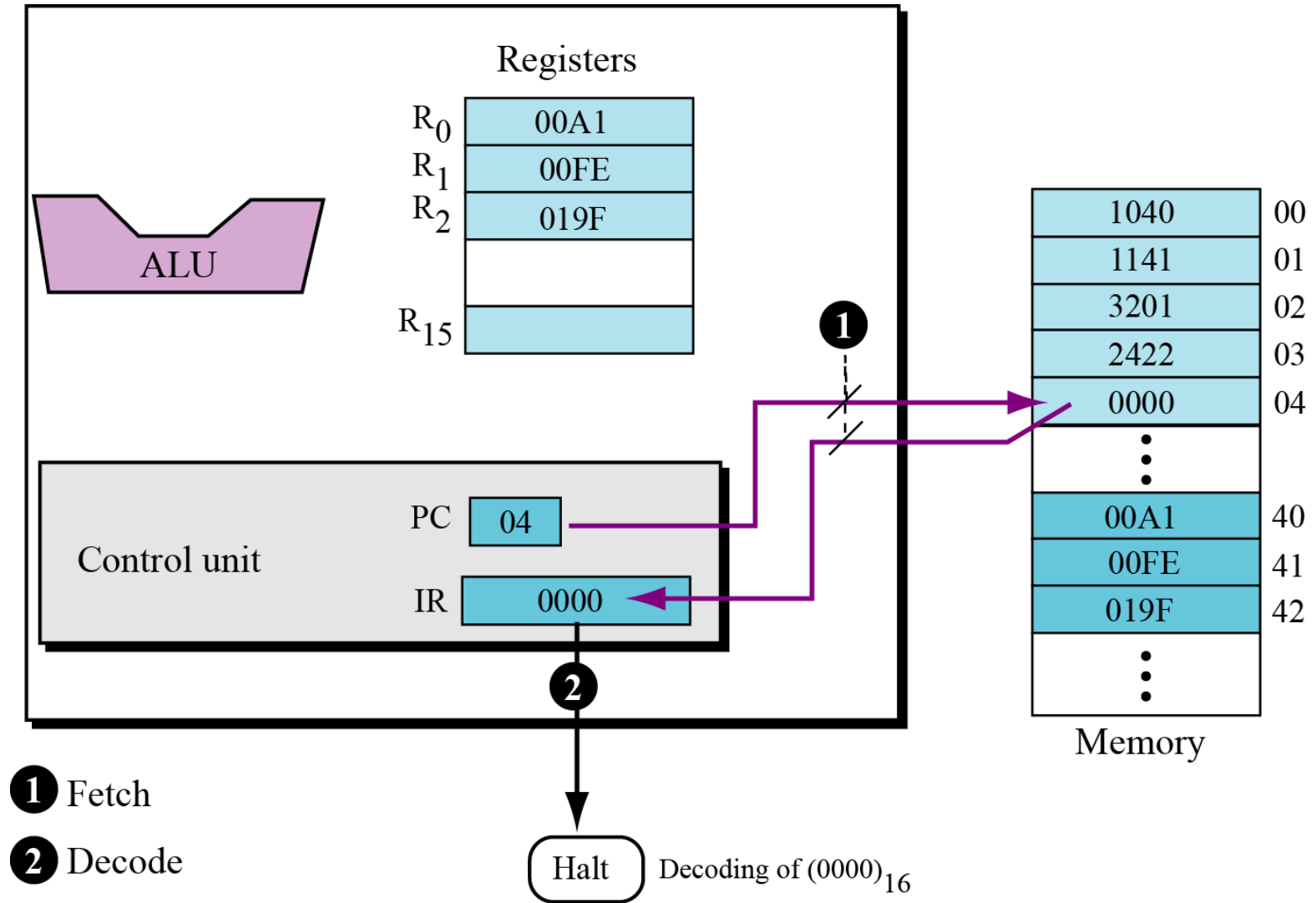


Figure: Status of cycle 5

Unstructured Programming

Limitations

- As code increases, complexity increases
 - Hard to manage large programs
 - Hard to debug
- No re-usability
- Statements written many times

Object-Oriented Programming

Lecture 2

Kinds of Programming Methodology

- Unstructured programming
- Procedural/Process oriented programming
 - Modular programming
- Object-oriented programming

Unstructured Programming

- Direct code (historically earliest programming paradigm)
 - Ex: Basic, Assembly Language

Limitations

- As code increases, complexity increases
 - Hard to manage large programs
 - Difficult to debug
- No re-usability
- Repeated Statements

Procedural-oriented programming

- Top-down approach
- Procedures (or Functions) are the building block

Ex: Calculator → scientific

→ non-scientific → float

→ int → add()

→ subs()

→ mult()

→ div()

} Small
functions

Ex: pascal, C - language

Procedural-oriented programming

- **Advantages:**

- Re-usability
- Easy to debug

- **Disadvantages:**

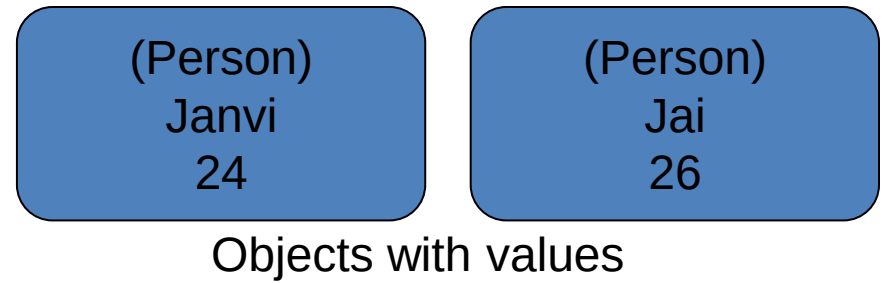
- Concentrate on what we want to do, not on who will use it
- Data does not have a owner (sharing)
- All functions are global
- No data security
 - Ex: Consider 3 functions a(),b() and c(). Let data d is used by a() and b() but how to restrict from c() [data is either global or local]
- No data integrity [Stack:{add(),del()}, Queue:{add(), del()}]
 - How to distinguish which fun associated with which data structure

Object-oriented programming

**Keep large software projects manageable by
human programmers**

- Bottom-up approach (user oriented)
- Objects are the building block
 - Each contains both methods and variables

An example



- Everything in *OOP* is grouped as self sustainable "*objects*".
- let's take your "*hand*" as another example (class/structure)
 - Your body has two objects of type hand, named left hand and right hand. Their main functions are controlled/ managed by a set of electrical signals sent through your shoulders
 - So the shoulder is an interface which your body uses to interact with your hands
 - The hand type (or class/structure) is being re-used to create self-sustainable objects: left hand and right hand

Another Example

- Ex: Institute (Student, Faculty, Admin)

Student (take course, write exam, view indiv. result)

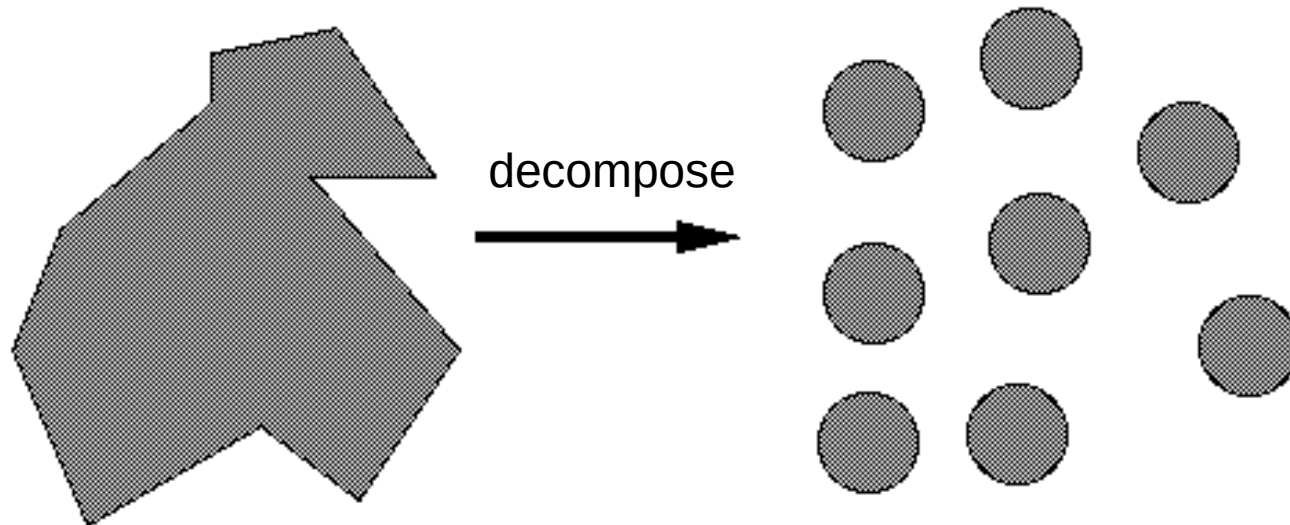
Faculty (set paper, prepare result, view results)

Admin (student admission, staff selection)

Why OOP?

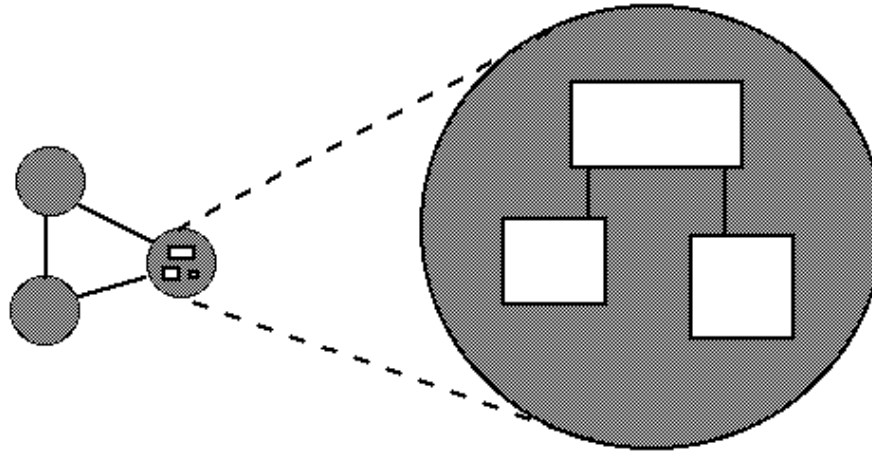
Modularization

- Decompose problem into smaller sub-problems that can be solved separately.



Why OOP?

Abstraction -- Understandability

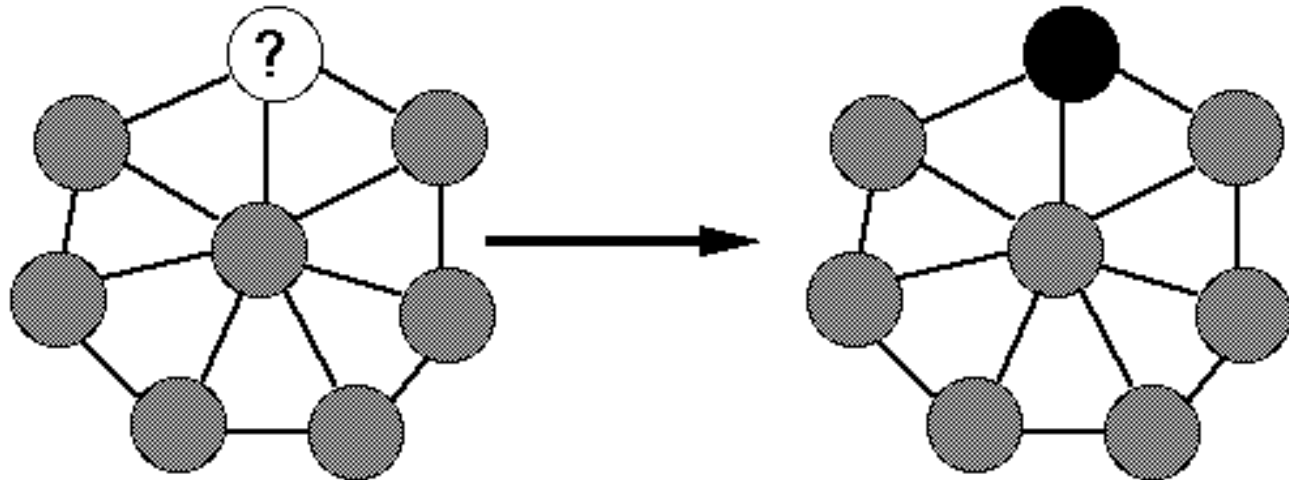


University
Admission
Account
HR
Teaching/Learning
Placement
Examination

- Individual modules are understandable by human readers.
 - ensure that the **entity** is named in a manner that will make sense – use real-world analogy

Why OOP?

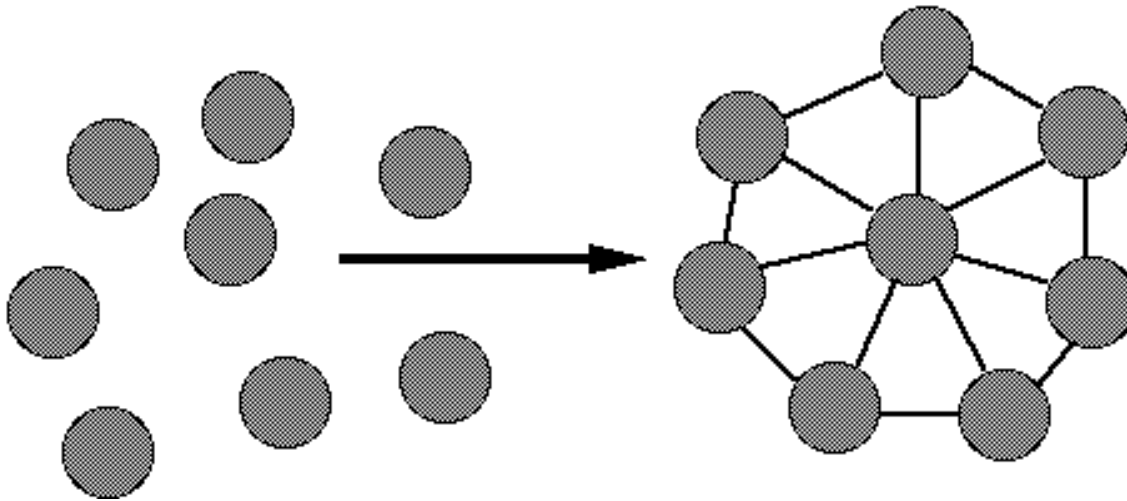
- **Encapsulation -- Information Hiding**



- Hide complexity from the user of a software. Protect low-level functionality.

Why OOP?

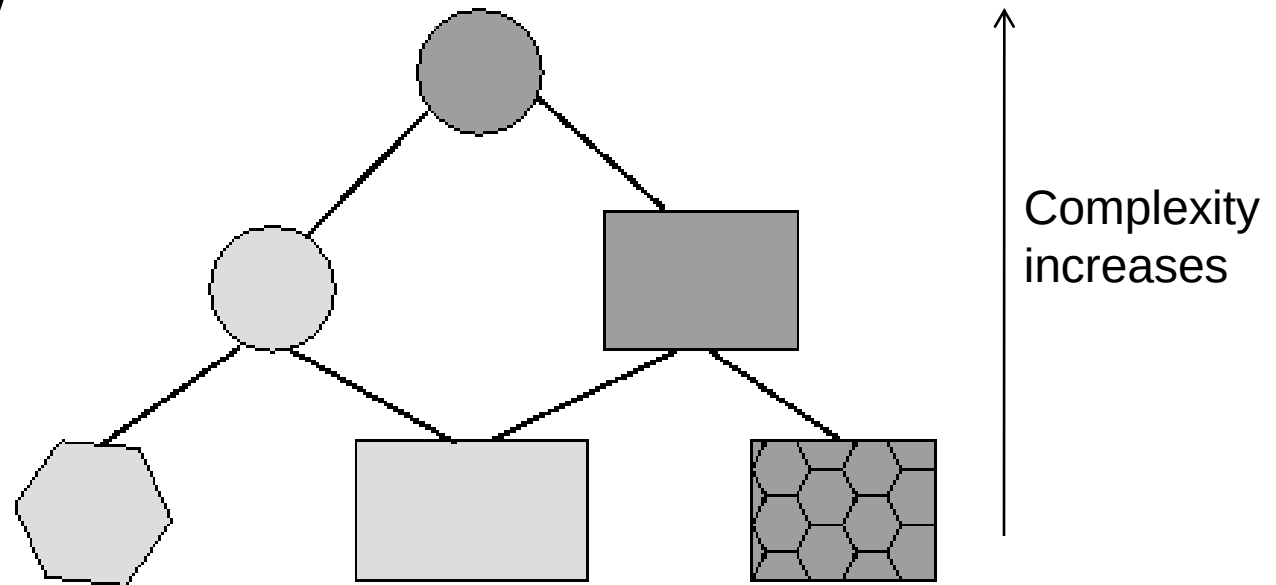
- **Composability -- Structured Design**



- Interfaces allow to freely combine modules to produce new systems.

Why OOP?

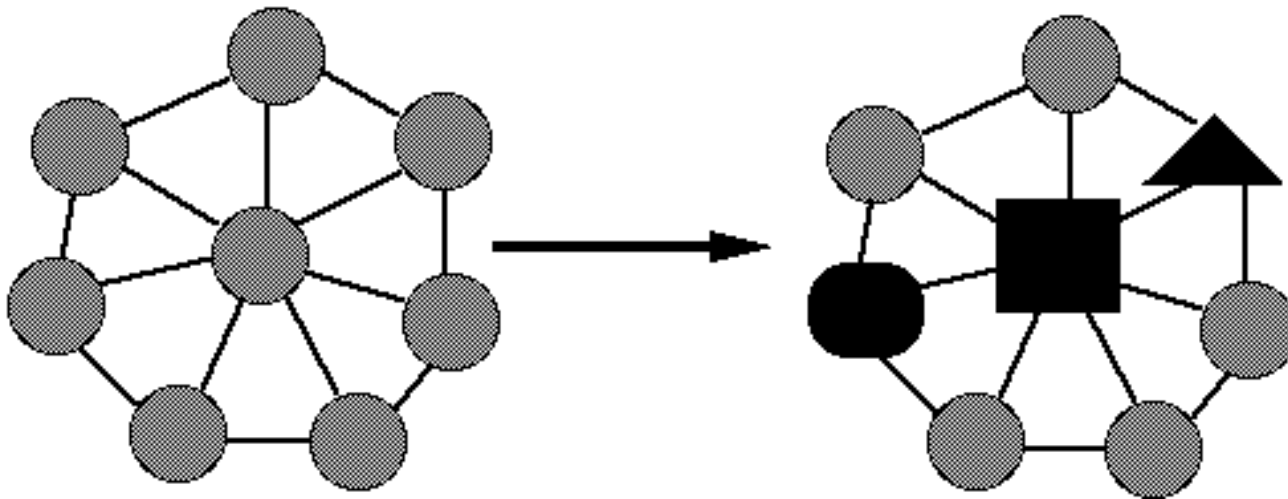
- **Hierarchy**



- Incremental development from small and simple to more complex modules.

Why OOP?

- **Continuity**



- Changes and maintenance in only a few modules does not affect the architecture.

Imp. Features of a OOP Language

- **Classes:** Modularization, structure.
- **Public/ Protected/ Private:** Encapsulation.
- **Interfaces (Abstract classes):** Composability.
- **Inheritance:** Hierarchy of modules, incremental development.
- **Polymorphism**

Introduction to Java

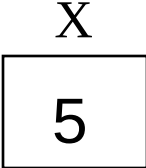
OOP languages

- *Both Java and C++ are most popular object-oriented programming languages*
- *C++ was created at AT&T Bell Labs in 1979*
- *Java was born in Sun Microsystems in 1990*

Variables for built-in types

In both C++ and Java

int x;



A diagram showing a variable 'x' represented by a box containing the number '5'. The 'x' is positioned above the box.

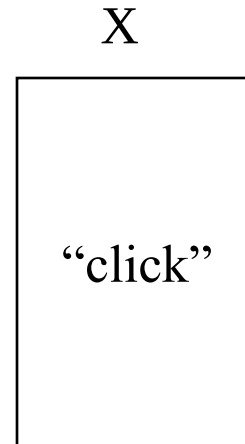
...

x = 5;

Variables that “hold” objects (in C++)

- Declaration of an object variable allocates space for the object

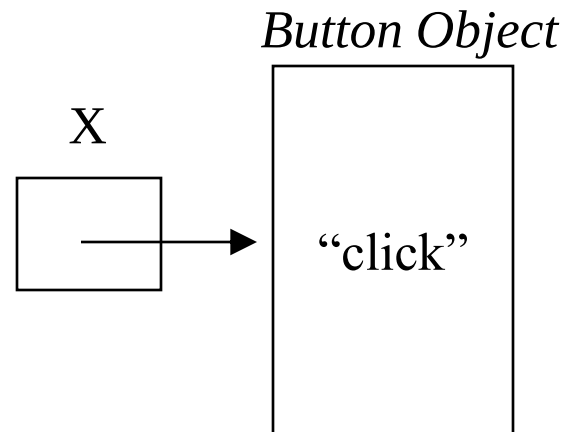
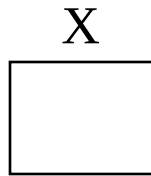
Button x(“Click”);



Pointers (in C++)

- Variables can be explicitly declared as pointers to objects

```
Button *x;  
...  
x = new Button("click");
```



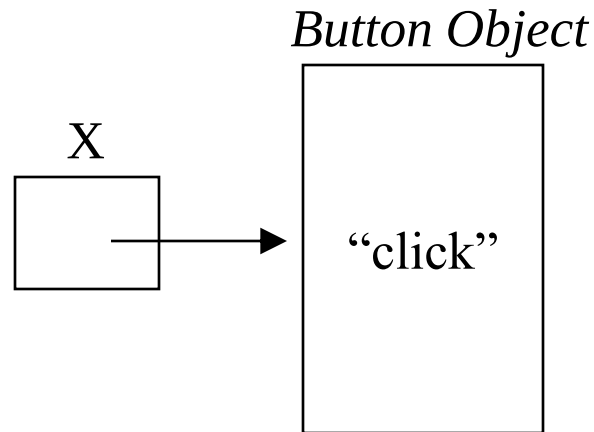
Reference variables (in Java)

- Reference type variables

Button x;

...

x = new Button("click");



Copy and assignment

- In C++, the semantics of “a = b” (assignment) can be specified
 - by defining the copy-assignment operator
 - both (a and b) are diff. objects but same content
 - In java it copies the address

Java basics

- Treats primitive data types and objects differently
 - `int i;` `// i actually holds the value`
 - `Account ac;` `// reference to a account object`

Disposing of allocated memory

- In C++, object destruction is the programmers responsibility using the **delete** keyword
- In Java, garbage collection is automatic
 - Memory allocated to objects are reclaimed when no variables refer to them
 - Need to set reference variables to null when the object is no longer needed

Summary

Java

- *No pointer*
- *No multiple inheritance*
- *Automatic garbage collection*
- *No operator overloading*
- *No goto statement and no structure and union data structure*

C++

- *Pointer*
- *Multiple inheritance*
- *Manual garbage collection*
- *Operator overloading*
- *goto statement and structure and union data structure*

Why Java ?

- Java is a general purpose object oriented programming language
- Internet programming language

Java Book's

1. Big Java (7th Edition)
Author : Cay Horstmann
2. Java: The complete reference (12th Edition)
Author : Herbert Schildt
3. Programming with Java (6th Ed)
Author : E. Balagurusamy

Web Source:

- ✓ <http://java.sun.com/docs/books/tutorial/>
- ✓ <http://java.sun.com/javase/6/docs>

History of Java

- A general purpose OOP language
- Developed by Sun Microsystems. (James Gostling)
- Initially called “Oak” but was renamed as “Java” in 1995
- Initial motivation is to develop a platform independent language to create software to be embedded in various consumer electronics devices
- Becomes the language of internet (portability and security)

Compiled and Interpreted

- Java works in two stage
 - Java compiler translate the *source code* into *byte code*
 - Java interpreter converts the byte code into machine level representation

Byte Code:

- A highly optimized set of instructions to be executed by the java runtime system, known as java virtual machine (JVM)
- Not executable code

JVM:

- Need to be implemented for each platform
- Although the details vary from machine to machine, all JVM understand the same byte code

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java is partially modeled on C++, but more simplified and improved (with more functionality and fewer negative aspects)

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java is designed from the start to be object-oriented.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- **Java Is Distributed**
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Distributed computing involves several computers working together on a network.

Java is designed to make distributed computing (e.g. *Web Services*) easy. Since networking capability is inherently integrated into Java, writing network programs is like sending and receiving data to and from a file.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- **Java Is Interpreted**
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

You need an interpreter to run Java programs.

The programs are compiled into bytecode. The bytecode is machine-independent and can run on any machine that has a Java interpreter, which is part of the JVM

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- **Java Is Robust**
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java compiler can detect many problems that would first show up at execution time in other languages

Java has eliminated certain types of error-prone programming constructs found in other languages (default value, memory leak)

Java has a runtime exception-handling feature to provide programming support for robustness (nulpointerexp.)

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- **Java Is Secure**
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Java implements several security mechanisms to protect your system

- enforce array bound check(buffer overflow)
- not allow to perform unsafe operations such as pointer arithmetic or unchecked type casts
- not allow manual control over memory allocation and deallocation; users rely on automatic garbage collection
- Java Library provides APIs related to security, such as standard cryptographic algo's, Auth., and secure comm protocols.
- Exception handling

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

Write once, run anywhere

With a Java Virtual Machine (JVM),
you can write one program that will
run on any platform

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- **Java Is Portable**
- Java's Performance
- Java Is Multithreaded
- Java Is Dynamic

The execution of bytecode by the JVM makes java programs portable

Because Java is architecture neutral, Java programs are portable. They can be run on any platform without being recompiled

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- **Java's Performance**
 - **Java performance is slower than C**
 - **Can sometimes be even faster than compiled C code!**
 - **Incorporation of multithreading enhance the overall execution speed**
 - **Just-in-Time (JIT) can compile the byte code into machine code**
- Java Is Multithreaded
- Java Is Dynamic

JIT compiler turns Java bytecode into instructions that can be sent directly to a particular hardware platform's processor.

Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- **Java Is Multithreaded**
- Java Is Dynamic

Multithread programming is smoothly integrated in Java,

whereas in other languages you have to call procedures specific to the operating system to enable multithreading

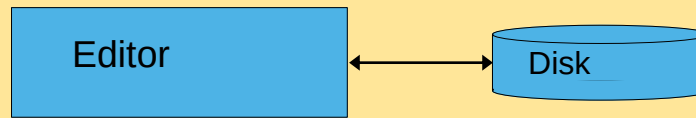
Characteristics of Java

- Java Is Simple
- Java Is Object-Oriented
- Java Is Distributed
- Java Is Interpreted
- Java Is Robust
- Java Is Secure
- Java Is Architecture-Neutral
- Java Is Portable
- Java's Performance
- Java Is Multithreaded
- **Java Is Dynamic**
 - Capable of dynamically linking a new class libraries, methods and objects.
 - Java can use efficient functions available in C/C++.
 - Installing new version of library automatically updates all programs

Basics of Java Environments

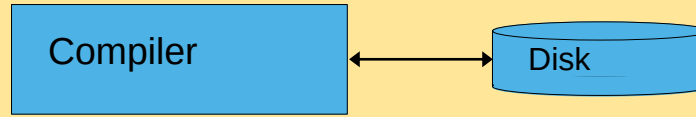
- Java programs normally undergo five phases
 - Edit
 - Programmer writes program (and stores program on disk)
 - Compile
 - Compiler creates *bytecode* from program
 - Load
 - Class loader stores bytecodes in primary memory
 - Verify
 - Verifier ensures bytecode do not violate security requirements
 - Execute
 - Interpreter translates bytecode into machine language

Phase 1



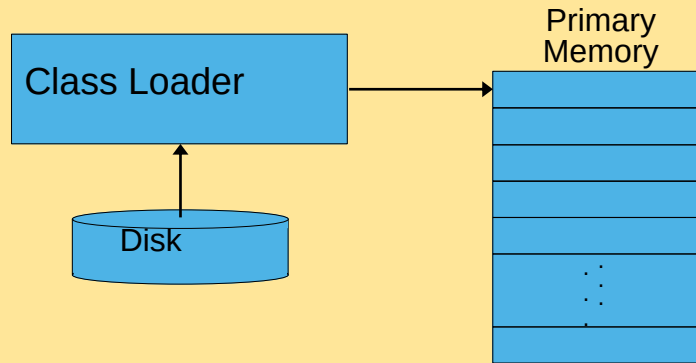
Program is created in an editor and stored on disk in a file ending with `.java`.

Phase 2



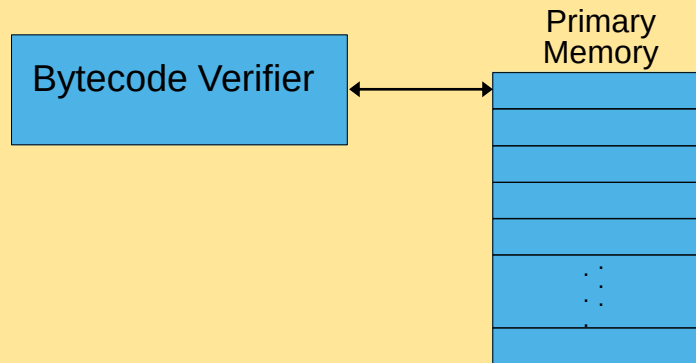
Compiler creates bytecodes and stores them on disk in a file ending with `.class`.

Phase 3



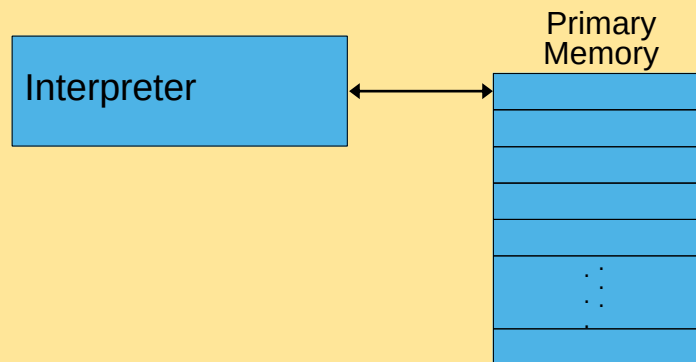
Class loader reads `.class` files containing bytecodes from disk and puts those bytecodes in memory.

Phase 4



Bytecode verifier confirms that all bytecodes are valid and do not violate Java's security restrictions.

Phase 5



Interpreter reads bytecodes and translates them into a language that the computer can understand, possibly storing data values as the program executes.

Java Environment

- Development tools-part of Java Development Kit (JDK)
- Classes and methods-part of Java Standard Library (JSL), also known as Application Programming Interface (API)

1. JDK:

- Appletviewer (for viewing applets)
- Javac (Compiler)
- Java (Interpreter)
- Javah (for C header files)
- Javadoc (for creating HTML description)

Java Environment

2. API

Contains hundreds of classes and methods grouped into several functional packages:

- **Language** Support Package
- **Utility** Packages (random num. gen., sys. date manipulation)
- **Input/Output** Packages
- **Networking** Packages (implementing networking app.)
- **AWT** Package (classes for painting graphics and images)
- **Applet** Package (web page using java)

The Evolution of Java

- Java 1.0 (96)
- Java 1.1 (97)(Add new library, redefine applet handling and reconfigured many features.)
- Java 2 (98)(Second generation). Version no:1.2 (Internal version number of java library). Also known as J2SE [Java 2 Platform Standard Edition].
 - Add swing, the collection framework, enhanced JVM etc.
- J2SE 1.3 (2000), J2SE 1.4 (2002), J2SE 5 (2004)
- J2SE 6 (2006), 7-(2011), 8(2014), 9(2017), ...
- Java SE 20 (Mar 2023), Java SE 21 (Sept. 2023),
- Java SE 22 (Mar 2024)

Comments

In Java, comments are preceded by two slashes (//) in a line, or enclosed between /* and */ in one or multiple lines

When the compiler sees //, it ignores all text after // in the same line

When it sees /*, it scans for the next */ and ignores any text between /* and */

Example

```
/* Traditional "Hello World!" program. */  
// package pack1;  
// import java.lang.System;  
class prog1  
{  
    public static void main (String args[])  
    {  
        System.out.println("Hello World!");  
    }  
}
```

Save program as **prog1.java**

Java Program Structure

Package Statement

- Javac command compiles the source code prog1.java then, generates **prog1.class** and store it under a directory which is called as name of the package
- package statement if used must be the first statement in a compilation unit. Its syntax is:

package packageName;

❑ For example:

package pack1;

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        System.out.println("Hello World!");
```

```
    }
```

```
}
```

Import Statement

- The import statements are similar to #include statements in C and C++
- In the above program, System class of java.lang package is imported into all Java programs by default. The syntax of import statement is as:

```
import fullClassName;
```

- For example, the following import statement imports the System class from java.lang:

```
import java.lang.System;  
import java.lang.*;
```

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        System.out.println("Hello World!");
```

```
    }
```

```
}
```

Classes

- Class declarations contain a keyword **class** and an identifier
- Class members are enclosed within braces. The syntax of defining a class is shown below:

```
class prog1  
    { // program code  
    }
```

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
public static void main (String args[])
```

```
{
```

```
    System.out.println("Hello World!");
```

```
}
```

```
}
```

Main method

- To execute a class, it must contain a valid main method
- It is the first method that automatically gets invoked when the program executed

```
public static void main (String args[])  
    { //instructions  
    }
```

Main method

```
public static void main (String args[])  
{  
    //instructions  
}
```

- The **main** method must always be defined as **public**: to make it publicly accessible,
- **static**: to declare it as a class member and
- **void**: returns no value
- **args[]**: parameter is an array of class String. It provides access to command line parameters

Example

```
/* Traditional "Hello World!" program. */
```

```
// package pack1;
```

```
// import java.lang.System;
```

```
class prog1
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        System.out.println("Hello World!");
```

```
    }
```

```
}
```

System class

```
System.out.println("Hello World!");
```

- invokes `println` method on object named `out` variable (of type `java.io.PrintStream`), which is a member of `System` class. The `println` method takes a String parameter and displays it on the console

Introduction to class constructor



Example

```
/* Traditional "Hello World!" program. */

// package pack1;
// import java.lang.System;
class prog1
{
    public static void main (String args[])
    {
        System.out.println("Hello World!");
    }
}
```

Main method

```
public static void main (String args[])  
{  
    //instructions  
}
```

- The **main** method must always be defined as **public**: to make it publicly accessible,
- **static**: to declare it as a class member and
- **void**: returns no value
- **args[]**: parameter is an array of class String. It provides access to command line parameters

System class

```
System.out.println("Hello World!");
```

- invokes `println` method on object named `out` (variable of type `java.io.PrintStream`), which is a member of `System` class.
- The `println` method takes a `String` parameter and displays it on the console

Class definition

```
class C1
{
    int i;
    char ch;

    public void name()
    {...}

    int get(int b)
    {...}
}
```

Access members of a class

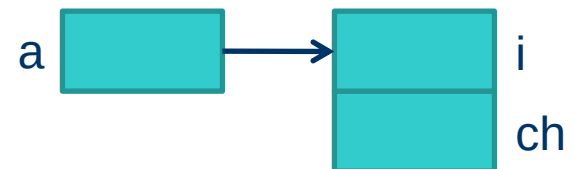
Class C1

```
{  
    int i;  
    char ch;  
    void set()  
        { i=20; }  
    int get()  
        { return i; }  
}
```

How to create two objects of same class
with different data elements?

How to access member of a class ?

```
C1 a= new C1();  
a.i;  
a.ch;  
a.set();
```



Constructor

- Similar as a method in a class
- Same name as class name
- No return type as methods

Why we need constructors?

- Initialize the members

Default cons (if we not defined)

- No parameter

- Ex: `C1() {
 }
 }`

Parameterized constructor

```
C1(int n)
{
    i=n;
}
```

```
C1(int n, char c)
{
    i=n;
    ch=c;
}
```

- Called when object initialized
- Can define any number of constructors

Example 2: two classes

```
class C1
{
    int i;
    char ch;
    void set()
        { i=20;}

    int get()
        {return i;}
}
```

```
public class C2
{
    public static void main(String args[])
    {
        C1 obj= new C1();
        int b;
        obj.set();
        b= obj.get();
        System.out.println("i="+ obj.i);
        System.out.println("i="+ b);
    }
}
```

Example 3: two classes uses cons.

```
class C1
{
    int i;
    char ch;
    void set()
        { i=20;}
    int get()
        {return i;}
}
```

```
C1 (int in, char c)
{ i=in; ch=c;
}
}
```

```
public class C2
{
    public static void main(String args[])
    {
        C1 obj= new C1(20,'g');
        System.out.println("i="+ obj.i);
        System.out.println("i="+ obj.ch);
    }
}
```

Example 4: single class

```
public class C1
{
    int i;
    char ch;
    void set()
        { i=20;}

    int get()
        {return i;}
```

```
public static void main(String args[])
{
    C1 a= new C1();
    int b;
    a.set();
    b=a.get();
    System.out.println("i="+ a.i);
    System.out.println("i="+ b);
}
}
```

Applet Example 1

```
/*  
<APPLET CODE="app1.class" WIDTH=150 HEIGHT=100>  
</APPLET>  
*/  
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class app1 extends Applet {  
    public void paint (Graphics g) {  
        g.drawString("Hello!",50,20);  
    }  
}
```

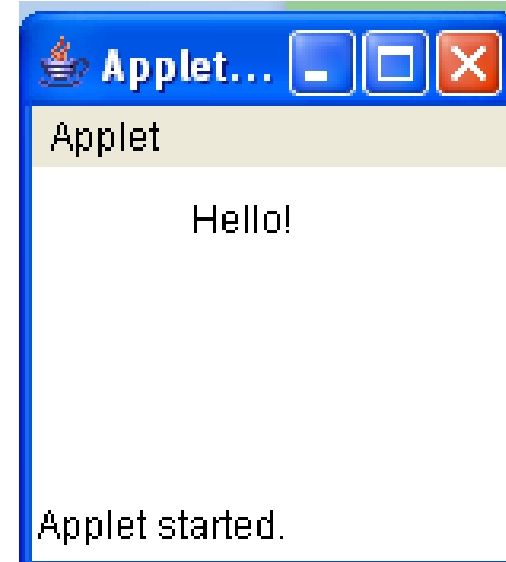
Applet program execution

Compile

```
javac app1.java
```

Execution

```
appletviewer app1.java
```



Execution through HTML file

```
<HTML>
  <HEAD>
    <TITLE> A simple Program</TITLE>
  </HEAD>
  <BODY> Here is the output:
    <APPLET CODE="app1.class" WIDTH=150 HEIGHT=100>
    </APPLET>
  <BODY>
</HTML>
```

Store with name app1.htm

Execute from browser: C:\java\app1.htm

Java Data types and Variables



Primitive Data Types

	Type	Size (in Bits)
● Integer		
	byte	8
	short	16
	int	32
	long	64
● Float	float	32
	double	64
● Character		
● Boolean	1/0	

Types and Variables

- Every value has a type
- Variable declaration examples:
String greeting = "Hello, World!";
PrintStream printer = System.out;
int Number = 13;
float num = 21.0;
- Variables
 - Store values
 - Can be used in place of the objects they store

Variable Definition

- *typeName variableName = value;*
or
typeName variableName;

Example:

- String greeting = "Hello, Dave!";

Purpose:

- To define a new variable of a particular type and optionally supply an initial value

Identifiers

- Identifier: name of a variable, method, or class
- Rules for identifiers in Java:
 - Can be made up of letters, digits, and the underscore (`_`) character
 - Cannot start with a digit
 - Cannot use other symbols such as `?` or `%`
 - Spaces are not permitted inside identifiers
 - You cannot use reserved words
 - They are case sensitive
- By convention, variable names start with a lowercase letter
- By convention, class names start with an uppercase letter

Self Check

- What is the type of the values 0 and "0"?

int and String

- Which of the following are legal identifiers?

Greeting1

g

void

101dalmatians

Hello, World

<greeting>

Only the first two are legal identifiers

```
String myName = "John Q. Public";
```

- Define a variable to hold your name

The Assignment Operator

- Assignment operator: `=`
- Not used as a statement about equality
- Used to change the value of a variable
`int luckyNumber = 13;`
`luckyNumber = 12;`

Uninitialized Variables

- `int luckyNumber;`
`System.out.println(luckyNumber);` `????`
– `// ERROR - uninitialized variable`

Syntax : Assignment

- *variableName = value;*

Self Check

- Is `12 = 12` a valid expression in the Java language?
- How do you change the value of the greeting variable to "Hello"?

Answers

- No, the left-hand side of the `=` operator must be a variable
- `greeting = "Hello";`
Note that
`String greeting = "Hello";`
is not the right answer—that statement defines a new variable

String Methods

- **length**: counts the number of characters in a string
`String greeting = "Hello, World!";`
`int n = greeting.length();` // sets n to 13
- **toUpperCase**: creates another String object that contains the characters of the original string, with lowercase letters converted to uppercase
`String river = "Mississippi";` // sets bigRiver to "MISSISSIPPI"
`String bigRiver = river.toUpperCase();`
- When applying a method to an object, make sure method is defined in the appropriate class `System.out.length();`
// This method call is an error

```
greeting = "Hello, World!";  
river = "Mississippi";
```

Self Check

- How can you compute the length of the string "Mississippi"?
- How can you print out the uppercase version of "Hello, World!"?
- Is it legal to call `river.println()`? Why or why not?

Answers

- `river.length()` or `"Mississippi".length()`
- `System.out.println(greeting.toUpperCase());`
- It is not legal. The variable `river` has type `String`. The `println` method is not a method of the `String` class.

Implicit and Explicit Parameters

- Parameter (explicit parameter):
 - Input to a method.
 - `System.out.println(greeting)`
 - Not all methods have explicit parameters.
 - `greeting.length()` // has no explicit parameter
- Implicit parameter: The object on which a method is invoked
`greeting.length()`

Return Values

- Return value: A result that the method has computed, for use

```
int n = greeting.length();
```

```
// return value stored in n
```

Passing Return Values

- You can also use the return value as a parameter of another method:
`System.out.println(greeting.length());`
- Not all methods return values
 - Example: `println`

A More Complex Call

- Let, String river="Mississippi";
- replace method carries out a search-and-replace operation

```
river.replace("issipp", "our")
```

```
// constructs a new string ("Missouri")
```

- This method call has
 - one implicit parameter: the string "Mississippi"
 - two explicit parameters: the strings "issipp" and "our"
 - a return value: the string "Missouri"

Method Definitions

- Method definition specifies types of explicit parameters and return value
- Type of implicit parameter (is object); not mentioned in method definition
- Example: Class String defines,

```
public int length()
```

```
// return type: int
```

```
// no explicit parameter
```

```
public String replace(String target, String replacement)
```

```
// return type: String;
```

```
// two explicit parameters of type String
```

Method Definitions

- If method returns no value, the return type is declared as **void**
public void println(String output) // in class
PrintStream
- A method name is **overloaded** if a class has more than one method with the same name (but different parameter types)
public void println(String output)
public void println(int output)

Self Check

```
river="Mississippi";
```

```
greeting = "Hello, World!";
```

- What are the implicit parameters, explicit parameters, and return values in the method call `river.length()`?
- What is the result of the call `river.replace("p", "s")`?
- What is the result of the call `greeting.replace("World", "Dave").length()`?
- How is the `toUpperCase` method defined in the `String` class?

- **Answers**

- The implicit parameter is `river`. There is no explicit parameter. The return value is 11
- "Missississi"
- 12
- As `public String toUpperCase()`, with no explicit parameter and return type `String`.

Number Types

- Integers: short, int, long
13
- Floating point numbers: float, double
1.3
0.00013
- When a floating-point number is multiplied or divided by 10, only the position of the decimal point changes; it "floats". This representation is related to the "scientific" notation 1.3×10^{-4} .
1.3E-4 // 1.3×10^{-4} written in Java
- Numbers are not objects; numbers types are primitive types

Arithmetic Operations

- Operators: + - *

$10 + n$

$n - 1$

$10 * n$ // $10 \times n$

- As in mathematics, the * operator binds more strongly than the + operator

$x + y * 2$ // means the sum of x and $y * 2$

$(x + y) * 2$ // multiplies the sum of x and y with 2

Self Check

- Which number type would you use for storing the area of a circle?
- Is the expression `13.println()` ok?
- Write an expression to compute the average of the values `x` and `y`.

Answers

- `double`
- An `int` is not an object, and you cannot call a method on it
- $(x + y) * 0.5$

Rectangular Shapes and Rectangle Objects

- Objects of type **Rectangle** *describe* rectangular shapes
- A Rectangle object isn't a rectangular shape—it is an object that contains a set of numbers that describe the rectangle

Constructing Objects

- `new Rectangle(5, 10, 20, 30)`

Detail:

- The new operator makes a Rectangle object
- It uses the parameters (in this case, 5, 10, 20, and 30) to initialize the data of the object
- It returns the object
- Usually the output of the new operator is stored in a variable

`Rectangle box = new Rectangle(5, 10, 20, 30);`

Constructing Objects

- The process of creating a new object is called *construction*
- The four values 5, 10, 20, and 30 are called the *construction parameters*

Self Check

- How do you construct a square with center (100, 100) and side length 20?
- What does the following statement print?
`System.out.println(new Rectangle().getWidth());`

Answers

- `new Rectangle(90, 90, 20, 20)`
- 0.0

Random, String and Math classes



Learning so far ...

- Difference between unstructured, procedural and OOP Languages
- Understood a computing machine and how a program get executed in a machine
- Difference between basic programming languages: C++ and Java (proper OOP Lang.)
- Characteristics of Java – Java environment – program structure
- Packages, static keyword, nested classes, Applet (web apps)
- Characteristics of OOP paradigm
- Classes and Objects
- Access specifiers: Public, private
- Constructors – default and parametrized constructors
- Implicit and Explicit parameters

Accessor and Mutator Methods

- Accessor method: does not change the state of its implicit parameter
`double width = box.getWidth();`
- Mutator method: changes the state of its implicit parameter
`box.translate(15, 25);`

```
Rectangle box = new Rectangle(5, 10, 20, 30);
```

Self Check

- Is the toUpperCase method of the String class an accessor or a mutator?
- Which call to translate is needed to move the box rectangle so that its top-left corner is the origin (0,0)?

Answers

- An accessor—it doesn't modify the original string but returns a new string with uppercase letters
- `box.translate(-5, -10)`

Importing Packages

- Don't forget to include appropriate packages: Java classes are grouped into packages
- Import library classes by specifying the package and class name:
`import java.awt.Rectangle;`
- You don't need to import classes in the `java.lang` package such as String and System

Example

```
import java.awt.Rectangle;
public class MoveTester
{
    public static void main(String[] args)
    {
        Rectangle box = new Rectangle(5, 10, 20, 30);
                                   // Move the rectangle
        box.translate(15, 25);
                                   // Print information about the moved rectangle
        System.out.println("After moving, the top-left corner is:");
        System.out.println(box.getX());
        System.out.println(box.getY());
    }
}
```

Output

- After moving, the top-left corner is:
20
35

Self Check

- The `Random` class is defined in the `java.util` package. What do you need to do in order to use that class in your program?

Answers

- Add the statement `import java.util.Random;` at the top of your program

Random class

- `Random r = new Random();`
- `int i = r.nextInt(int n)` Returns random int ≥ 0 and $< n$
- `int i = r.nextInt()` Returns random int (full range)
- `long i = r.nextLong()` Returns random long (full range)
- `float f = r.nextFloat()` Returns random float ≥ 0.0 and < 1.0
- `double d = r.nextDouble()` Returns random double ≥ 0.0 and < 1.0
- `boolean b = r.nextBoolean()` Returns random bit value (true ,false)
- `double x;`
`x = Math.random();` // assigns random number to x

Object References

- Describe the location of objects
- The **new** operator returns a reference to a new object
Rectangle box = new Rectangle();
- Multiple object variables can **refer** to the same object
Rectangle box = new Rectangle(5, 10, 20, 30);
Rectangle box2 = box;
box2.translate(15, 25);
- Primitive type variables $\neq$ object variables

Self Check

- What is the effect of the assignment `greeting2 = greeting`?
- After calling `greeting2.toUpperCase()`, what are the contents of `greeting` and `greeting2`?

Answers

- Now `greeting` and `greeting2` both refer to the same `String` object.
- Both variables still refer to the same string, and the string has not been modified. Recall that the `toUpperCase` method **constructs a new string** that contains uppercase characters, leaving the original string unchanged.

Method Definition

- *accessSpecifier* *returnType* *methodName*(
 parameterType *parameterName*, . . .)
 {
 method body
 }

Example:

- **public** void deposit(**double** amount)
 { . . .
 }

Constructor Definition

- *accessSpecifier* *ClassName*(*parameterType* *parameterName*, . . .)
 {
 constructor body
 }

Example:

- public BankAccount(double initialBalance)
 {
 . . .
 }

The return Statement

- return *expression*;
or
return;

Example:

- return balance;

Primitive Types

Type	Description	Size
<code>int</code>	The integer type, with range -2,147,483,648 . . . 2,147,483,647	4 bytes
<code>byte</code>	The type describing a single byte, with range -128 . . . 127	1 byte
<code>short</code>	The short integer type, with range -32768 . . . 32767	2 bytes
<code>long</code>	The long integer type, with range -9,223,372,036,854,775,808 . . . 9,223,372,036,854,775,807	8 bytes
<code>double</code>	The double-precision floating-point type, with a range of about $\pm 10^{308}$ and about 15 significant decimal digits	8 bytes
<code>float</code>	The single-precision floating-point type, with a range of about $\pm 10^{38}$ and about 7 significant decimal digits	4 bytes
<code>char</code>	The character type, representing code units in the Unicode encoding scheme	2 bytes
<code>boolean</code>	The type with the two truth values <code>false</code> and <code>true</code>	1 bit

Cast

- *(typeName) expression*

Example:

- `(int) (balance * 100)`

Purpose:

- To convert an expression to a different type

Type Casting

Which of the following are legal?

- `int x = 3.5;`

Illegal: 3.5 is not an int

- `float x = 3;`

Legal: 3 is an int, which is also a Float

- `long i = 3;`

Legal: 3 is an int, which is also a long

- `byte x = 155;`

Illegal: 155 is too big to be a byte (> 127)

- `double d = 3.14159f;`

Legal: 3.14159f is a float, which is also a double

What is “Type Casting”?

- Type casting: automatic conversion of values from one type to another
- e.g. int → double
 - Float → double
 - Int → long
- Type casts can be:
 - Implicit: performed automatically
 - Explicit: programmed by developer

Float range from 1.40129846432481707e-45
to 3.40282346638528860e+38

Implicit Type Casting in Java

- Hierarchy of primitive numerical types

double
float
long
int
short
byte



- Idea

- Higher types have more precision
- Lower types are “subsets” of higher types

- Java only performs implicit **upcasts** (casting from lower to higher type)

e.g. int → double
 float → double
 int ↗ byte

Explicit Type Casting in Java

- To explicitly cast a value to a type `t`, use `(<t>)` value
 - `int x = (int)3.7;`
 - Assigns value 3 to `x`
 - Reason: `(int)` operator converts double to int by truncation (chopping off decimal) when double is small enough
 - This is an example of **downcasting**
 - `byte bt = (byte) 200;`
 - Assigns value -56 to `bt`
 - Reason: `(byte)` operator “wraps around” values that are too big
- This sounds confusing
 - It is!
 - Rule of thumb:
 - Only use explicit casts when you know what the answer is likely to be
 - Otherwise (e.g. in “overflow” situations) write your own type-conversion routines (we’ll see how later this semester)

Mixed Expressions

- What is result of
`float x = 3 / 4;`
 - x assigned value 0.0F
 - Why?
 - 3, 4 are ints
 - So integer / operation is used, yielding 0, before upcasting is performed
- To get floating point result, use explicit casting
`float x = (float) 3 / (float) 4;`
 - Assigns x the value 0.75F
- Can also do following
`float x = (float) 3 / 4;`
 - Why?
 - `(float) 3` returns a value type float (3.0F)
 - 4 is an int
 - In this case, Java compiler uses upcasting on “lower” type (here, `int`) to obtain values in same type before computing operation

Self Check

- Which are the most commonly used number types in Java?
- When does the cast `(long) x` yield a different result from the call `Math.round(x)`?

Answers

- int and double
- When the fractional part of x is ≥ 0.5

Constants: final

- A **final** variable is a constant
- Once its value has been set, it cannot be changed
- Named constants make programs easier to read and maintain
- Convention: use all-uppercase names for constants

```
final double QUARTER_VALUE = 0.25;  
final double DIME_VALUE = 0.1;
```

Constants: static final

- static is used with class members
- If constant values are needed in several methods, declare them together and tag them as **static** and **final**
- Give static final constants **public** access to enable other classes to use them

```
public class Math  
{
```

```
...
```

```
public static final double E = 2.7182818284590452354;  
public static final double PI = 3.14159265358979323846;  
}
```

```
double circumference = Math.PI * diameter;   (call without object)
```


Syntax : Constant Definition

- In a method:
final typeName variableName = expression ;
- In a class:
accessSpecifier static final typeName variableName = expression;

Example:

- *final double NICKEL_VALUE = 0.05;*
- *public static final double LITERS_PER_GALLON = 3.785;*

Increment, and Decrement

- `items++` is the same as `items = items + 1`
- `items--` subtracts 1 from items
 - `++a;` Let, `a=10`
 - `--a;`
 - `a++;`
 - `a--;`
- 11,10,10,11..... (Top to Bottom)
- 10, 9, 9,10.....(Bottom to Top)

Arithmetic Operations

- $/$ is the division operator
- If both arguments are integers, the result is an integer. The remainder is discarded
 - $7.0 / 4$ yields ? 1.75
 - $7 / 4$ yields ? 1
- Get the remainder with $\%$ (pronounced "modulo")
 $7 \% 4$ is ? 3

The Math class

- Math class: contains methods like `sqrt` and `pow`
- To compute x^n , you write `Math.pow(x, n)`
- However, to compute x^2 it is significantly more efficient simply to compute `x * x`
- To take the square root of a number `x`, use the `Math.sqrt`;
for example,
`Math.sqrt(x)`

Mathematical Methods in Java

<code>Math.sqrt(x)</code>	square root
<code>Math.pow(x, y)</code>	power x^y
<code>Math.exp(x)</code>	e^x
<code>Math.log(x)</code>	natural log
<code>Math.sin(x), Math.cos(x), Math.tan(x)</code>	sine, cosine, tangent (x in radian)
<code>Math.round(x)</code>	closest integer to x
<code>Math.min(x, y), Math.max(x, y)</code>	minimum, maximum

java.lang.Math methods

- E, PI (constants)
- sin(_), cos(_), abs(_), tan(_), ceil(_), floor(_), log(_), max(_,_), min(_,_), pow(_,_), sqrt(_), round(_), random(), toDegrees(_), toRadians(_)

Self Check

- What is the difference between the following two statements?
`final double CM_PER_INCH = 2.54;`
and
`public static final double CM_PER_INCH = 2.54;`
- What is wrong with the following statement?
`double circumference = 3.14 * diameter;`

Answers

- The first definition is used **inside a method**, the second **inside a class**
- (1) You should use a named constant, not 3.14
(2) 3.14 is not an accurate representation of π

Self Check

- What is the value of $1729 / 100$? and $1729 \% 100$?
- What does the following statement compute ?
`double average = s1 + s2 + s3 / 3;` `[s1=10,s2=20,s3=30]`
- What is the value of `Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2))` in mathematical notation?

Answers

- 17 and 29
- 40: Only s3 is divided by 3. To get the correct result, use parentheses. Moreover, if s1, s2, and s3 are integers, you must divide by 3.0 to avoid integer division:
 $(s1 + s2 + s3) / 3.0$

Calling Static Methods

- A static method does not operate on an object
Double x = 4;
Double root = x.sqrt(); // Error
- Naming convention: Classes start with an uppercase letter; objects start with a lowercase letter
Math
System.out

Static Method Call

- *ClassName. methodName(parameters)*

Example:

- `Math.sqrt(4)`

Purpose:

- To invoke a static method (a method that does not operate on an object) and supply its parameters

double x=10.0

Self Check

- Why can't you call `x.pow(y)` to compute x^y ?
- Is the call `System.out.println(4)` a static method call?

Answers

- x is a number, not an object, and you cannot invoke methods on numbers
- No—the `println` method is called on the object `System.out`

Strings

- A string is a sequence of characters
- Strings are objects of the String class
- String constants:
 "Hello, World!"
- String variables:
 String message = "Hello, World!";
- String length:
 int n = message.length();
- Empty string: ""

Concatenation

- Use the + operator:

```
String name = "Dave";  
String message = "Hello, " + name;  
// message is "Hello, Dave"
```

- If one of the arguments of the + operator is a string, the other is converted to a string

```
String a = "Agent";  
int n = 7;  
String bond = a + n; // bond is Agent7
```

Converting between Strings and Numbers

- Convert to number:

```
int n = Integer.parseInt(str);  
double x = Double.parseDouble(str);
```

- Convert to string:

```
String str = "" + n;  
str = Integer.toString(n);
```

Substrings

- String greeting = "Hello, World!";
String sub = **greeting.substring(0, 5);**
// sub is "Hello"
- Supply start and end positions
- First position is at 0

Self Check

- Assuming the String variable `s` holds the value "Super", what is the effect of the assignment `s = s + s.length()` ?
- Assuming the String variable `river` holds the value "Mississippi", what is the value of
`river.substring(1, 2)?` and
`river.substring(2, river.length() - 3)?`

Answers

- `s` is set to the string Super5
- The strings "i" and
- "ssissi"

New learning

- Accessor and Mutator Methods
- Random class – Util package
- Type casting
- Constants – Final and static final
- Static methods
- Math class methods
- Converting strings and numbers, etc.

If-else and loop Statements



The if Statement

- The **if** statement lets a program carry out different actions depending on a condition

```
if (balance >= price)
{
    purchase();
    balance = balance - price;
}
```

If / else Statement

```
if (balance >= price)
    { purchase();
      balance = balance - price;
    }
else
    { overdraft();
      purchase();
    }
```

If / else Statement

- *if(condition)*
statement
- *if (condition)*
statement1
else
statement2

Comparing Values: Relational Operators

Java	Math Notation	Description
>	>	Greater than
>=	≥	Greater than or equal
<	<	Less than
<=	≤	Less than or equal
==	=	Equal
!=	≠	Not equal

Multiple Alternatives: Sequences of Comparisons

- **if** (*condition1*)
 statement1;
 else if (*condition2*)
 statement2;
 ...
 else
 statement4;

Multiple Alternatives: Nested Branches

- Branch inside another branch

```
if (condition1)
{
    if (condition1a)
        statement1a;
    else
        statement1b;
}
else
    statement2;
```


Using Boolean Expressions: Predicate Method

- A predicate method returns a boolean value

```
public boolean isOverdrawn()
{
    return balance < 0;           // return (balance < 0);
}
```
- Use in conditions

```
if (Check.isOverdrawn()) . . .
```

Using Boolean Expressions: Predicate Method

- Useful predicate methods in **Character** class:
isDigit(Char c)
isLetter(Char c)
isUpperCase(Char c)
isLowerCase(Char c), isSpace(Char c)
- if (Character.isUpperCase(c)) . . .
- Useful predicate methods in **Scanner** class: hasNextInt() and hasNextDouble()
if (in.hasNextInt()) n = in.nextInt();

Using Boolean Expressions: The Boolean Operators

- `&&` and
- `||` or
- `!` not
- `if (0 < amount && amount < 1000) . . .`
- `if (input.equals("S") || input.equals("M")) . . .`

Self Check

- When does the statement `system.out.println (x > 0 || x < 0);` print false?

Answers

- When x is zero

while Loops

- Executes a block of code repeatedly
- A condition controls how often loop is executed
while (condition)
statement;
- Most commonly, the statement is a block statement (set of statements delimited by { })
while (condition) { statement; }

while Loop Flowchart

*while (condition)
statement;*

How to print int less then 10?

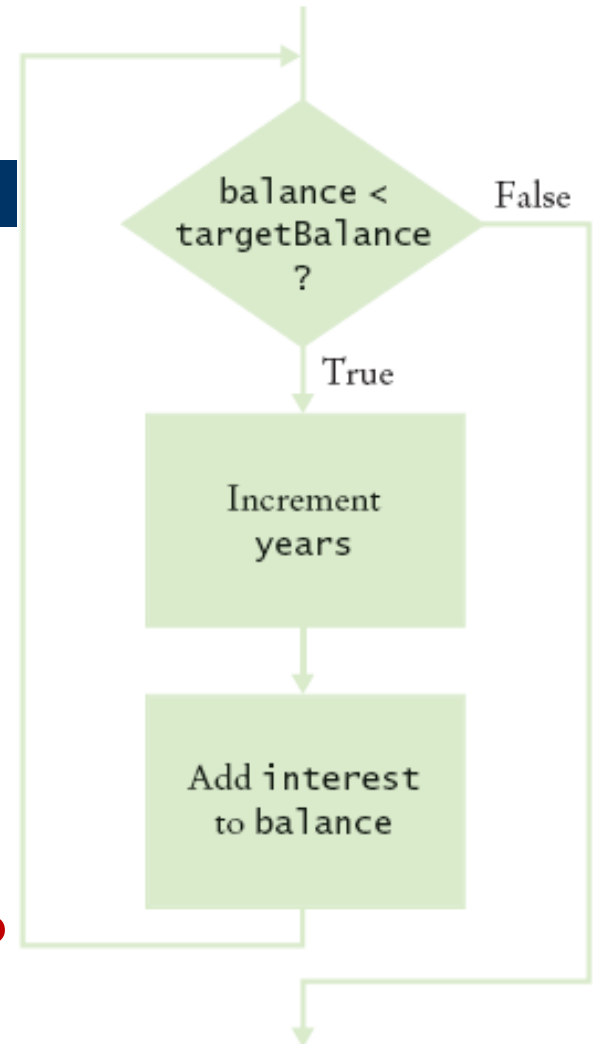


Figure 1 Flowchart of a while Loop

Self Check

- How many times is the statement in the loop *while (false) statement;* executed?

Answers

- Never

What if false condition is replaced by 0, 2, 10, 100, -5 , -1?

Common Error:

- ```
years = 0;
while (years < 20)
{
 interest = balance * rate / 100;
 balance = balance + interest;
}
```

 // should have years++
- ```
years = 20;
while (years > 0)
{
    years++;
    interest = balance * rate / 100;
    balance = balance + interest;
}
```

 // should have years--
- Loops run forever—must kill program

Do-while Loop

- Executes loop body at least once:

```
do{  
    statement;  
}  
while (condition);
```

Do-while Loop

Example: Validate input

- **double value;**
do
 {
 System.out.print("Please enter a positive number: ");
 value = in.nextDouble();
 }
while (value <= 0);

Alternative:

- **boolean done = false;**
while (!done)
 {
 System.out.print("Please enter a positive number: ");
 value = in.nextDouble();
 if (value > 0) done = true;
 }

For loop and Arrays



for Loop

- for (*initialization; condition; update*)
 { *statement;* }

- Example:

```
for (int i = 1; i <= n; i++)  
{  
    double interest = balance * rate / 100;  
    balance = balance + interest;  
}
```

while loop as for loop

- Equivalent to

```
initialization;  
while (condition)  
{  
    statement;  
    update;  
}
```

```
for (int i = 1; i <= n; i++)  
{ statement;  
}
```

Self Check

- How many times does the following for loop execute?
for (i = 0; i <= 10; i++)
System.out.println(i);
- Rewrite the above for loop as a while loop

Answers

- 11 times
- ```
int i = 0;
while (i <= 10)
{
 System.out.println(i); // { System.out.println(i++); }
 i++;
}
```

# Common Errors

- Example:

```
sum = 0;
for (i = 1; i <= 10; i++);
 sum = sum + i;
 System.out.println(sum);
```

**Delete semi colon**

**Use parenthesis if more than one statement**

# Nested Loops

- Some triangle patterns

\*

\*\*

\*\*\*

\*\*\*\*

(1)

\*

\*\*\*

\*\*\*\*\*

\*\*\*\*\*

(2)

\*

\*\*\*

\*\*\*\*\*

\*\*\*\*\*

\*\*\*\*\*

\*\*\*

\*

(3)



# Self Check

- What is the value of n after the following nested loops?

```
int n = 0;
for (int i = 1; i <= 5; i++)
 for (int j = 0; j < i; j++)
 n = n + j;
```

## Answers

- 20

# Array

- Sequence of values of the same type
- Construct array:  
`new double[10]`
- Store in variable of type `double[]`  
`double[] data = new double[10];`
- When array is created, all values are initialized depending on array type:
  - Numbers: 0
  - Boolean: false
  - Object References: null

# Element access of an array

- Use `arrayname[ i ]` to access an element  
`data[2] = 29.95;`  
`data[5] = 19.05;`  
`data[8] = 9.91;`

|      |   |   |       |   |   |       |   |   |      |
|------|---|---|-------|---|---|-------|---|---|------|
|      | 0 | 1 | 2     | 3 | 4 | 5     | 6 | 7 | 8    |
| data |   |   | 29.95 |   |   | 19.05 |   |   | 9.91 |

# Self Check

- What elements does the data array contain after the following statements?

```
int[] data = new int[10];
for (int i = 0; i < data.length; i++)
 data[i] = i * i;
```

## Answers

- 0, 1, 4, 9, 16, 25, 36, 49, 64, 81, but not 100

# Self Check

- What do the following program segments print? Or, if there is an error, describe the error and specify whether it is detected at compile-time or at run-time.
  - `double[ ] a = new double[10];`  
`System.out.println(a[0]);`
  - `double[ ] b = new double[10];`  
`System.out.println(b[10]);`
  - `double[ ] c;`  
`System.out.println(c[0]);`

## Answers

- - 0.0
  - a run-time error: array index out of bounds
  - a compile-time error: c is not initialized

# Generic classes

```
class Solution<T>
{ T data;
 public static T getData()
 {
 return data;
 } }
```

```
Solution<String> S=new Solution<String>();
```

# Array Lists

- The `ArrayList` class
  - manages a sequence of objects
  - Can grow and shrink as needed
  - supplies methods for many common tasks, such as inserting and removing elements
  - a generic class: `ArrayList<T>` collects objects of type T:  
`ArrayList<BankAccount> accounts = new  
ArrayList<BankAccount>();`  
`accounts.add(new BankAccount(1001));`  
`accounts.add(new BankAccount(1015));`  
`accounts.add(new BankAccount(1022));`
  - `size()` method yields number of elements

# Retrieving Array List Elements

- Use `get()` method
- Index starts at 0
- `BankAccount anAccount = accounts.get(2);`  
    // gets the third element of the array list
- Bounds error if index is out of range
- Most common bounds error:  
    `int i = accounts.size();`  
    `anAccount = accounts.get(i);`     // Error

    // legal index values are 0. . .i-1



# Adding and Removing elements

- **set()** method overwrites an existing value  
BankAccount anAccount = new BankAccount(1729);  
**accounts.set(2, anAccount);**
- **add()** adds a new value      **accounts.add(i, a)**  
    X.add("red");    // ["red"]  
    X.add("blue");   // ["red", "blue"]  
    X.add(1, "white"); // ["red", "white", "blue"]
- **remove** method removes an element at an index  
**accounts.remove(i)**

# Self Check

- How do you construct an array of 10 strings?
- An array list of strings?
- What is the content of names after the following statements?  

```
ArrayList<String> names = new ArrayList<String>();
names.add("A");
names.add(0, "B");
names.add("C");
names.remove(1);
```

## Answers

- `new String[10];`
- `new ArrayList<String>();`
- names contains the strings "B" and "C" at positions 0 and 1

# Designing Classes

## Choosing Classes

- A class represents a single concept from the problem domain
- Name for a class should be a noun that describes concept
- Concepts from mathematics: Point, Rectangle, Ellipse
- Concepts from real life: BankAccount, Student, Employee

# Static Methods

- Every method must be in a class
- A static method is not invoked on an object

Quest: Suppose Java had no static methods. Then all methods of the Math class would be instance methods. How would you compute the square root of x?

Ans: `Math m = new Math();`  
`y = m.sqrt(x);`

# Static Fields/variables

- A static field belongs to the class, not to any object of the class. Also called *class field*.
- ```
public class BankAccount
{
    ...
    private double balance;
    private int accountNumber;
    private static int var = 1000;    // Executed once, when class is loaded
}
```
- call as BankAccount.var
- If **var** was not static, each instance of BankAccount would have its own value of var

Static variable

- All objects of its class, shares same static variable space
- If value changes, it will be changed for every later call to the static variable (until further change)
- It can be thought as a common shared variable
 - ```
public BankAccount() // constructor
{
 // Generates next account number to be assigned
 var++; // Updates the static field
 // Assigns field to account number of this bank account
 accountNumber = var; // Sets the instance field
}
```
  - Minimize the use of static fields. (Static final fields are ok.)

# Scope of Local Variables

- Scope of variable: Region of program in which the variable can be accessed
- Scope of a local variable extends from its declaration to end of the block that encloses it
- Sometimes the same variable name is used in two methods:

```
public class RectangleTester
{
 public static double area(Rectangle rect)
 {
 double r = rect.getWidth() * rect.getHeight();
 return r;
 }
 public static void main(String[] args)
 {
 Rectangle r = new Rectangle(5, 10, 20, 30);
 double a = area(r);
 System.out.println(r);
 }
}
```

- These variables are independent from each other; their scopes are disjoint

# Scope of Local Variables

- Scope of a local variable cannot contain the definition of another variable with the same name

```
Rectangle r = new Rectangle(5, 10, 20, 30);
if (x >= 0)
{
 double r = Math.sqrt(x);
 // Error--can't declare another variable named r here
 ...
}
```

- However, can have local variables with identical names if scopes do not overlap

```
if (x >= 0)
{
 double r = Math.sqrt(x);
 ...
}
else // Scope of r ends here
{
 Rectangle r = new Rectangle(5, 10, 20, 30);
 // OK--it is legal to declare another r here ...
}
```



# Scope of Class Members

- Private members have class scope: You can access all private members in any method of the class
- Public members have outside scope

`Math.sqrt`

`harrysChecking.getBalance`

- Inside a method, no need to qualify member variables or methods that belong to the same class
  - An unqualified instance variable or method name refers to the **this** parameter
- `public class BankAccount`

```
{
 public void transfer(double amount, BankAccount other)
 {
 withdraw(amount); // i.e., this.withdraw(amount);
 other.deposit(amount);
 }
}
```

# Overlapping Scope

- A local variable can *shadow* a variable with the same name

- Local scope wins over class scope

```
public class Coin
{
 ...
 public double getExchangeValue(double exchangeRate)
 {
 double value; // Local variable
 ...
 return value;
 }
 private String name;
 private double value; // variable with the same name
}
```

- Access shadowed fields by qualifying them with the **this** reference  
value = this.value \* exchangeRate;

# An example

```
class temp1
{ public double get()
 { double value=10.0; // Local variable
 value = value + this.value;
 return value;
 }
 private String name;
 public double value=20.0; // Field with the same name
}
```

```
public class temp2
{ public static void main(String args[])
 {
 temp1 a=new temp1();
 System.out.println(a.get());
 System.out.println(a.value);
 }}
}
```

Output: 30  
20

# Organizing Related Classes into Packages

- Package: Set of related classes.
- To put classes in a package, you must place a line

`package packageName;`

- the first instruction in the source file containing the classes Package name

# Organizing Related Classes into Packages

| Package       | Purpose                                   | Sample Class            |
|---------------|-------------------------------------------|-------------------------|
| java.lang     | Language support                          | Math, String, Character |
| java.util     | Utilities                                 | Random                  |
| java.io       | Input and output                          | PrintStream             |
| java.awt      | Abstract Windowing Toolkit                | Color                   |
| java.applet   | Applets                                   | Applet                  |
| java.net      | Networking                                | Socket                  |
| java.sql      | Database Access                           | ResultSet               |
| javax.swing   | Swing user interface                      | Jbutton, JFrame         |
| org.omg.CORBA | Common Object Request Broker Architecture | IntHolder               |

# Importing Packages

- Can always use class without importing  
`java.util.Scanner in = new java.util.Scanner(System.in);`
- Tedious to use fully qualified name
- Import lets you use shorter class name  
`import java.util.Scanner;`  
...  
`Scanner in = new Scanner(System.in)`
- Can import all classes in a package  
`import java.util.*;`
- Never need to import `java.lang`
- You don't need to import other classes in the same package

# Package Names and Locating Classes

- Use packages to avoid name clashes  
`java.util.Timer` vs. `javax.swing.Timer`
- Package names should be unambiguous

# Self Check

- Which of the following are packages?
  - java
  - java.lang
  - java.util
  - java.lang.Math
- Can you write a Java program without ever using import statements?

## Answers

- No      Yes      Yes      No
- Yes—if you use fully qualified names for all classes, such as `java.util.Random` and `java.awt.Rectangle`



# Designing Classes



# Two dimensional arrays

```
double[][] a = new double[3][4];
```

```
int M = 3; int N = 4;
```

```
double[][] ar = new double[M][N];
```

```
for (int i = 0; i < M; i++)
```

```
{
```

```
 for (int j = 0; j < N; j++)
```

```
 { ar[i][j] = 0.0;
```

```
 }
```

```
}
```

# Initialize 2D array using values

```
double[][] ar = {
 { .02, .92, .02, .07, .02 },
 { .02, .03, .32, .32, .32 },
 { .04, .02, .02, .92, .08 },
 { .92, .02, .02, .04, .02 },
 { .47, .02, .47, .02, .01 },
};
```

# Matrix addition

```
double[][] c = new double[N][N];
for (int i = 0; i < N; i++)
 for (int j = 0; j < N; j++)
 c[i][j] = a[i][j] + b[i][j];
```

# The `?:` Operator

- In addition to the **if** statement, Java provides a more compact way to express conditional execution that can be extremely useful in certain situations.

$$condition \ ? \ expression_1 \ : \ expression_2$$

- When Java evaluates the `?:` operator, it first determines the value of *condition*, which must be a **boolean**. If *condition* is **true**, Java evaluates *expression<sub>1</sub>* and if *condition* is **false**, Java evaluates *expression<sub>2</sub>*
- You could use the `?:` operator to assign the larger of **x** and **y** to the variable **max** like this:

$$\text{max} = (\text{x} > \text{y}) \ ? \ \text{x} \ : \ \text{y};$$

# The `switch` Statement

The **`switch`** statement provides a convenient syntax for choosing among a set of possible paths:

```
switch (expression) {
 case v_1 :
 statements to be executed if expression = v_1
 break;
 case v_2 :
 statements to be executed if expression = v_2
 break;
 ... more case clauses if needed ...
 default:
 statements to be executed if no values match
 break;
}
```

# Command-Line Input vs. Standard Input

## Command-line inputs

- Use command-line inputs to read a few user values
- Input entered before program begins execution

## Standard input

- Flexible OS abstraction for input
- Input entered while program is executing

# Standard input.

- **StdIn** is library for reading text input.

```
public class StdIn
```

---

|         |               |                                                |
|---------|---------------|------------------------------------------------|
| boolean | isEmpty()     | <i>true if no more values, false otherwise</i> |
| int     | readInt()     | <i>read a value of type int</i>                |
| double  | readDouble()  | <i>read a value of type double</i>             |
| long    | readLong()    | <i>read a value of type long</i>               |
| boolean | readBoolean() | <i>read a value of type boolean</i>            |
| char    | readChar()    | <i>read a value of type char</i>               |
| String  | readString()  | <i>read a value of type String</i>             |
| String  | readLine()    | <i>read the rest of the line</i>               |
| String  | readAll()     | <i>read the rest of the text</i>               |



# Standard output

- **StdOut** is library for writing text output

```
public class StdOut
```

---

```
void print(String s)
```

*print s*

```
void println(String s)
```

*print s, followed by newline*

```
void println()
```

*print a new line*

```
void printf(String f, ...)
```

*formatted print*

# A example: reading input string

```
import java.io.*;
public class Ball {
 public static void main (String[] args) {
 BufferedReader stdin = new BufferedReader (new
 InputStreamReader(System.in));
 String message; // Creates a variable called message for input
 System.out.print ("Enter the message : ");

 message = stdin.readLine();
 System.out.println("You entered : " + message);
 } // method main
} // end of class
```

# An example: read integer

```
public class Add {
 public static void main(String[] args) {
 BufferedReader stdin = new BufferedReader (new
 InputStreamReader(System.in));
 System.out.print("Type the first integer: ");
 String number1 = stdin.readLine();
 int x = Integer.parseInt(number1);
 System.out.print("Type the second integer: ");
 String number2 = stdin.readLine();
 int y = Integer.parseInt(number2);
 int sum = x + y;
 System.out.println("Their sum is " + sum); } }
```

# Another example

```
public class Average {
 public static void main(String[] args) {
 BufferedReader stdin = new BufferedReader (new
 InputStreamReader(System.in));
 double sum = 0.0; // cumulative total
 int n = 0; // number of values
 while (!stdin.isEmpty()) {
 // String number = stdin.readLine();
 // double y = Double.parseDouble(number);
 double x = stdin.readDouble();
 sum = sum + x;
 n++; }
 StdOut.println(sum / n);
 }
}
```

Ctrl z to exit (Windows)  
Ctrl d (Linux,etc)

# Designing Classes

Access modifiers

Encapsulation, this

Inheritance

Function overloading & overriding



# Class

```
class Car
{
 // member variables
 // member methods
}
```

## **Example:**

```
class Car { //these are member variables
 String manufacturer;
 String model;
 int year;
 int passengers;
}
```

# Encapsulation

- One of advantages of OOP is encapsulation
- ability to define classes that hide their impl. details, exposing only public methods to other classes
- Support for encapsulation in Java comes from three keywords: **public**, **private**, and **protected**
- these access modifiers specify where a member is accessible
- **public** is entirely accessible to all other classes (whether in same package or outside)

# Public access modifier

```
class Car // all are public
{
 public String manufacturer;
 public String model;
 public int year;
 public int passengers;
}
```



# Private access modifier

- Declaring all members as public, does not achieve the goal of encapsulation since it permits direct access to the class members from outside the class
- To restrict access to a field, use the keyword **private**
- A private member is available only within the same class

# Private access modifier cont.

Suppose the Car class is intended for use in a used car sales application. In this case, in order to hide the cost of car from potential buyers:

```
class Car {
 public String manufacturer;
 public String model;
 public int year;
 public int passengers;
 private float cost; }
```

# Protected access modifier

- members declared as **protected** are accessible within the same class and classes derived from the class, but not to other classes
- The ability to derive a class from another class will be discussed later

# static member

- In addition to private, protected and public members, a Java class can also have static members
- A static member is often referred to as class members or class instances since they are accessed with reference to class name instead of instance (object) name.

# Member Methods

- Similar to member variables, access to member methods can be controlled with the `public`, `protected` and `private` modifiers

```
[methodModifiers] returnType methodName
 ([param1],[param2]...) [throws exceptionList]
{ // method body
}
```

# Member Methods(modifiers)

| Modifier            | Purpose                                                                                                                                      |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Public</b>       | Accessible outside the class in which it is declared.                                                                                        |
| <b>Protected</b>    | Accessible by the class in which it is declared and by subclasses of that class.                                                             |
| <b>Private</b>      | Accessible only by the class in which it is declared.                                                                                        |
| <b>Static</b>       | A method of the class rather than of a particular instance of the class.                                                                     |
| <b>Abstract</b>     | Not implemented in this class.                                                                                                               |
| <b>Final</b>        | Cannot be overridden in subclasses.                                                                                                          |
| <b>Native</b>       | A platform-dependent implementation of the method in another language, typically C or assembly.                                              |
| <b>Synchronized</b> | Used to indicate a critical method, that will lock the object to prevent execution of other methods, while the synchronized method executes. |

# Return type of a method

- can be one of the primitive types (for example, int, float, char), another class, or void

# Overloaded Methods

- Use the same method name for more than one methods

## Example:

```
private float cost;
```

```
public float CalculateSalePrice() {
 return cost * 1.5; }
}
```

```
public float CalculateSalePrice(double margin) {
 return cost * (1 + margin); }
}
```



# The **this** Variable

- The `this` can be used at runtime to reference the object itself

```
public class Car
{ String manufacturer; String model; int year; int passengers; float cost;
 // calculate the sale price of a car based on its cost
 public double CalculateSalePrice() {
 return cost * 1.5; }
 // a public constructor
 public Car(String manufacturer, String model, int year, int passengers, float cost)
 { this.manufacturer = manufacturer;
 this.model = model;
 this.year = year;
 this.passengers = passengers;
 this.cost = cost;
 }
}
```

# Class Inheritance

- Every class in Java is derived from another class
- A derived class is referred to as a **subclass**,
- while the class it is derived from is referred to as **superclass**.
- A class can be derived from another by using **extends** keyword as follows:

Example:

```
public class ClassicCar extends Car
{
 // member methods and variables
}
```

# Extends by default

- A class is not declared as being derived from a specific class is implicitly derived from Java base class, **Object**.
- This means that the following two class declarations are same:

```
public class Car {
 // member methods and variables
}
```

```
public class Car extends Object {
 // member methods and variables
}
```

# Overriding member methods

- While creating a subclass, all functionalities of its superclass are inherited
- The subclass can add or change this functionality as desired

```
public class Car {
 private int year;
 private float originalPrice; // calculate the sale price of a car based on its cost
 public double CalculateSalePrice() {
 double salePrice;
 if (year > 1994) salePrice = originalPrice * 0.75;
 else if (year > 1990) salePrice = originalPrice * 0.50;
 else salePrice = originalPrice * 0.25;
 return salePrice;
 } // a public constructor
 public Car(int year, float originalPrice) {
 this.year = year;
 this.originalPrice = originalPrice;
 }
}
```

# Subclass inherit car

```
public class ClassicCar extends Car {
 // calculate the sale price of a car based on its cost
 public double CalculateSalePrice() {
 return 10000;
 }

 // a public constructor
 public ClassicCar(int year, float originalPrice) {
 super(year, originalPrice);
 }
}
```

## Cont.

```
ClassicCar myClassic = new ClassicCar(1920, 1400);
double classicPrice = myClassic.CalculateSalePrice();
```

```
Car myCar = new Car(1990, 12000);
double price = myCar.CalculateSalePrice();
```

- Variable myClassic is of type ClassicCar. Because myClassic is a ClassicCar, the sale price will be \$10,000.
- Next, myCar is constructed as a new object of type Car. Because myCar is a Car, its sale price will be based on the year it was made (1990) and will be 25% of \$12,000, or \$3,000.

# Inheritance

# INHERITANCE BASICS

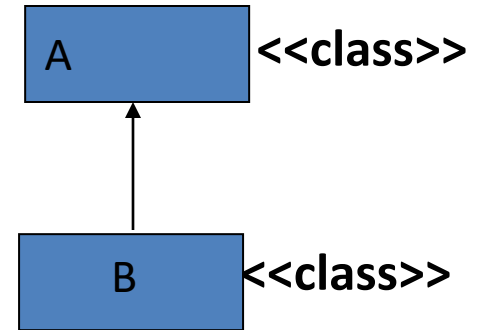
1. Reusability is achieved by **INHERITANCE**
2. Java classes Can be Reused by extending a class. Extending an existing class is for reusing properties of the existing classes.
3. The class whose properties are extended is known as ***super or base or parent class.***
4. The class which extends the properties of super class is known as ***sub or derived or child class***
5. ***A class can either extends another class or can implement an interface***



**class B extends A { ..... }**

**A super class**

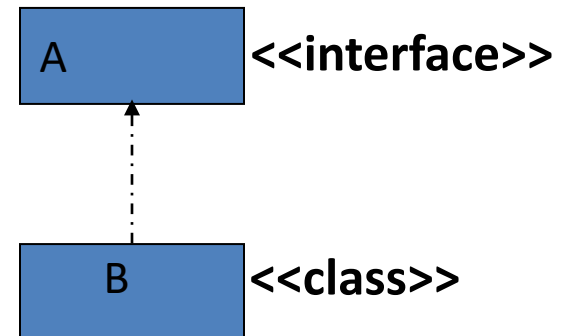
**B sub class**



**class B implements A { ..... }**

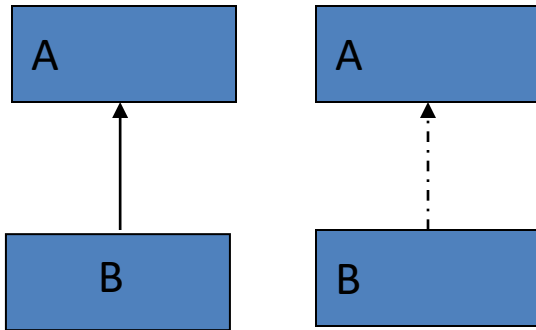
**A interface**

**B sub class**

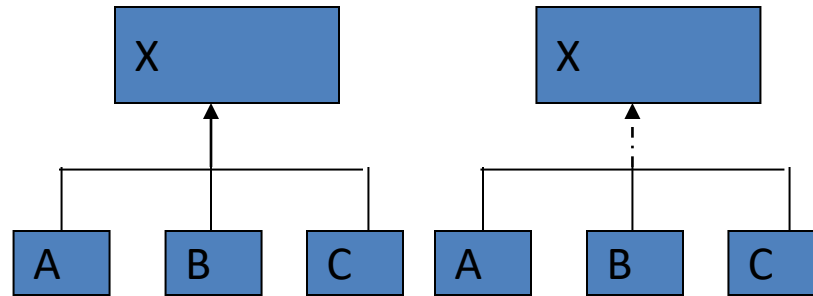


# Various Forms of Inheritance

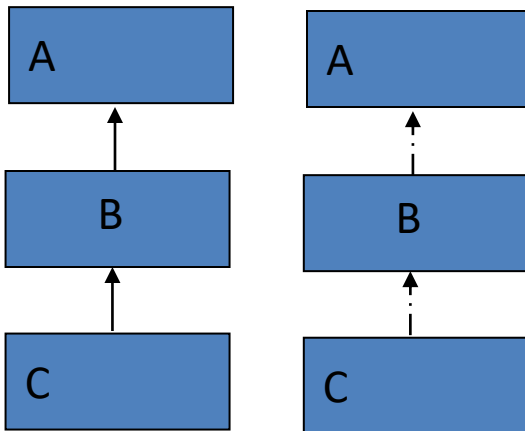
Single Inheritance



Hierarchical Inheritance

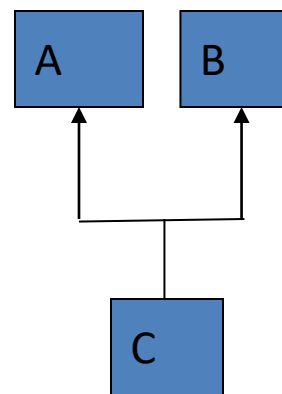


MultiLevel Inheritance

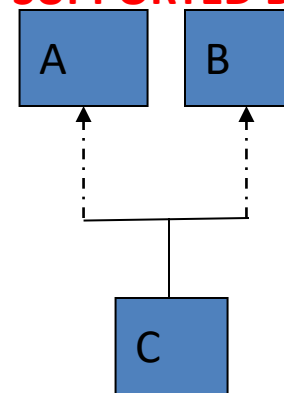


**NOT SUPPORTED BY JAVA**

Multiple Inheritance



**SUPPORTED BY JAVA**



# Forms of Inheritance

- Multiple Inheritance can be implemented by implementing multiple interfaces not by extending multiple classes

Example :

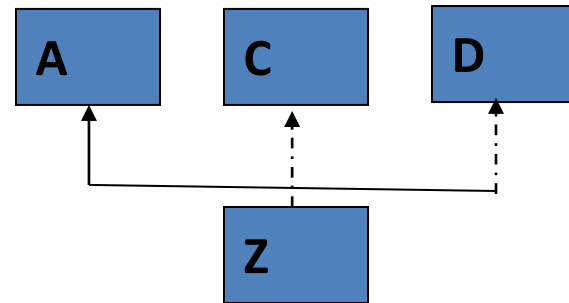
```
class Z extends A implements C , D
{ }
```

**OK**

```
class Z extends A ,B
{
```

**WRONG**

```
}
```



```
class Z extends A extends B
{
```

**WRONG**

```
}
```

**OR**

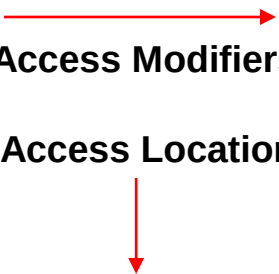
# Defining a Subclass

## Syntax :

```
class <subclass name> extends <superclass name>
{
 variable declarations;
 method declarations;
}
```

- extends keyword signifies that properties of the super class are extended to sub class
- Sub class will not inherit private members of super class

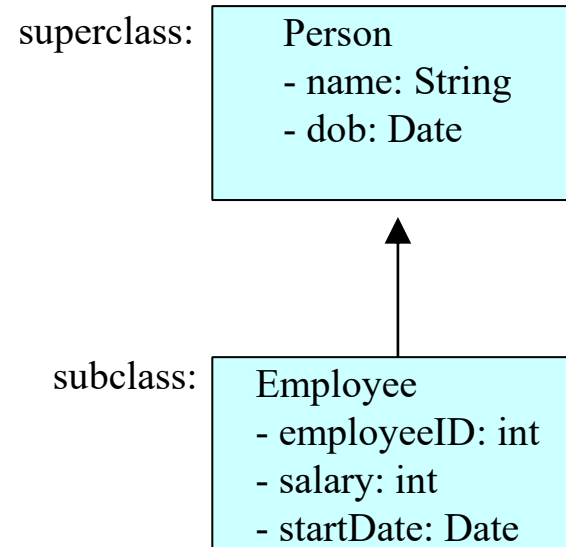
# Access Control

|  Access Modifiers<br>Access Location | public     | protected  | <u>private</u> |
|-----------------------------------------------------------------------------------------------------------------------|------------|------------|----------------|
| Same Class                                                                                                            | <u>Yes</u> | <u>Yes</u> | <u>Yes</u>     |
| sub classes in same package                                                                                           | <u>Yes</u> | <u>Yes</u> | <u>No</u>      |
| Other Classes in Same package                                                                                         | <u>Yes</u> | <u>Yes</u> | <u>No</u>      |
| Subclasses in other packages                                                                                          | <u>Yes</u> | <u>Yes</u> | <u>No</u>      |
| Non-subclasses in other packages                                                                                      | <u>Yes</u> | <u>No</u>  | <u>No</u>      |

# Terminology

---

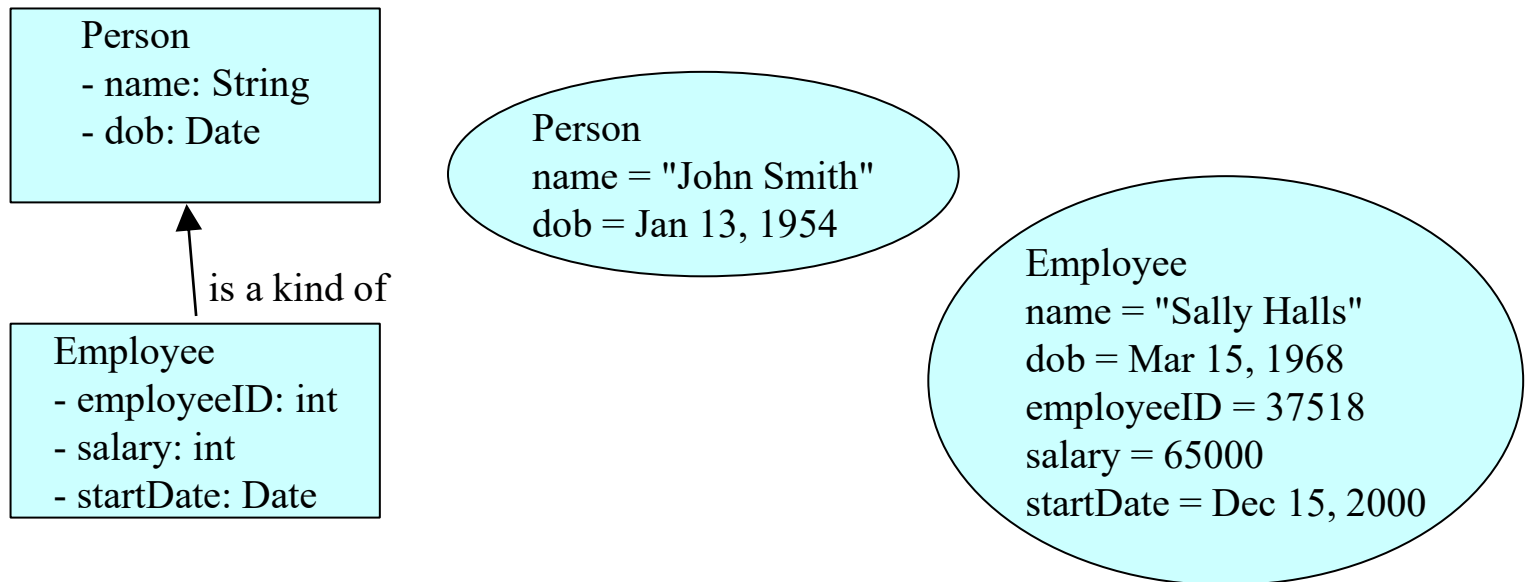
- Inheritance is a fundamental Object Oriented concept
- A class can be defined as a "subclass" of another class.
  - The subclass inherits all data attributes of its superclass
  - The subclass inherits all methods of its superclass
  - The subclass inherits all associations of its superclass
- The subclass can:
  - Add new functionality
  - Use inherited functionality
  - Override inherited functionality



# What really happens?

---

- When an object is created using new, the system must allocate enough memory to hold all its instance variables.
  - This includes any inherited instance variables
- In this example, we can say that an Employee "is a kind of" Person.
  - An Employee object inherits all of the attributes, methods and associations of Person



# Inheritance in Java

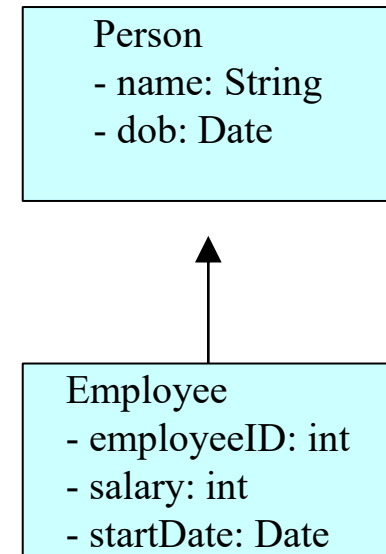
---

- Private members are not inherited in subclass

```
public class Person
{
 private String name;
 private Date dob;
 [...]
```

```
public class Employee extends Person
{
 private int employeeID;
 private int salary;
 private Date startDate;
 [...]
```

```
Employee anEmployee = new Employee();
```

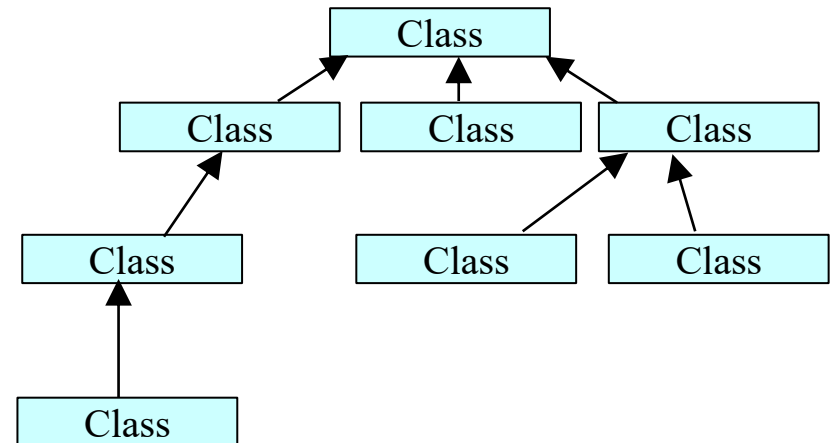




# Inheritance Hierarchy

---

- Each Java class has one (and only one) superclass.
  - C++ allows for multiple inheritance
- Inheritance creates a class hierarchy
  - Classes higher in the hierarchy are more general and more abstract
  - Classes lower in the hierarchy are more specific and concrete
- There is no limit to the number of subclasses a class can have
- There is no limit to the depth of the class tree.



# Inheritance Basics

1. Whenever a subclass object is created, super class constructor is called first.
2. If super class constructor does not have any constructor of its own OR has an unparametrized constructor then it is automatically called by Java Run Time by using call **super()**
3. If a super class has a parameterized constructor then it is the responsibility of the sub class constructor to call the super class constructor by call  
**super(<parameters required by super class>)**
4. Call to super class constructor must be the first statement in sub class constructor

# Inheritance Basics

**When super class has a Unparametrized constructor**

```
class A
{
A()
{
System.out.println("This is constructor of class A");
}
} // End of class A

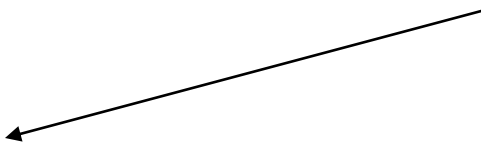
class B extends A
{
B()
{
super();
System.out.println("This is constructor of class B");
}
} // End of class B
```

```
class inhtest
{ public static void main(String args[])
{ B b1 = new B();
} }
```

## OUTPUT

This is constructor of class A  
This is constructor of class B

**Optional**



```
class A
{
A()
{
System.out.println("This is class A");
}}
```

**Without super() call**

```
class B extends A
{
B()
{
System.out.println("This is class B");
}}
```

output

This is class A  
This is class B

```
class inherit1
{
public static void main(String args[])
{
B b1 = new B();
}}
```

## private constructor

```
class A
{
 private A()
 {
 System.out.println("This is class A");
 }
}
```



class B extends A

```
{
 B()
 {
 System.out.println("This is class B");
 }
}
```

class inherit2

```
{
 public static void main(String args[])
 {
 B b1 = new B();
 }
}
```

/\*

C:\Java>javac inherit2.java

inherit2.java:12: A() has private access in A

{

^

1 error

\*/

```

class A
{
private A()
{
System.out.println("This is class A");
}
A()
{
System.out.println("This is class A");
}
}
class B extends A
{
B()
{
System.out.println("This is class B");
}
}
class inherit2
{
public static void main(String args[])
{
B b1 = new B();
}}

```

```

/*
E:\Java>javac inherit2.java
inherit2.java:7: A() is already defined in A
A()
^
inherit2.java:16: A() has private access in A
{
^
2 errors
*/

```

## When Super class has a parametrized constructor.

```
class A
{
private int a;
A(int a)
{
this.a =a;
System.out.println("This is constructor of class A");
}
}

class B extends A
{
private int b;
private double c;
B(int b,double c)
{
this.b=b;
this.c=c;
System.out.println("This is constructor of class B");
}
}
```

**B b1 = new B(10,8.6);**

```
D:\java\bin>javac inhtest.java
inhtest.java:15: cannot find
symbol
symbol : constructor A()
location: class A
{
^
1 errors
```

```
class A
{
private int a;
A(int a)
{
this.a =a;
System.out.println("This is constructor of
class A");
}}
```

**class B extends A**

```
{
private int b;
private double c;
B(int a,int b,double c)
```

```
{
```

```
super(a);
```

```
this.b=b;
```

```
this.c=c;
```

```
System.out.println("This is constructor of
class B");
```

```
}}
```

**B b1 = new B(8,10,8.6);**

## **OUTPUT**

**This is constructor of class A**

**This is constructor of class B**



```

class A
{
private int a;
protected String name;
A(int a, String n)
{
this.a = a;
this.name = n;
}
void print()
{
System.out.println("a="+a);
}
}

```

'a' is private in class A

Call to print() from  
super class A

```

class B extends A
{
int b;
double c;
B(int a,String n,int b,double c)
{
super(a,n);
this.b=b;
this.c=c;
}
void show()
{
//System.out.println("a="+a);
print();
System.out.println("name="+name);
System.out.println("b="+b);
System.out.println("c="+c);
}
}

```

Accessing name field from super  
class (super.name)

## OutPut

```

E:\Java>java xyz3
a=10
name=OOP
b=8
c=10.56

```

```

class xyz3
{ public static void main(String args[])
{ B b1 = new B(10,"OOP",8,10.56);
 b1.show();
} }

```

# USE OF super KEYWORD

- Can be used to call super class constructor

`super();`

`super(<parameter-list>);`

- Can refer to super class instance variables/Methods

`super.<super class instance variable/Method>`

```

class A
{
private int a;
A(int a)
{
this.a =a;
System.out.println("This is constructor of class
A");
}
void print()
{ System.out.println("a="+a);
}
void display()
{
System.out.println("hello This is Display in A");
} } // End of class A

```

```

class inhtest1
{
public static void main(String args[])
{ B b1 = new B(10,8,4.5);
 b1.show();
} }

```

```

class B extends A
{
private int b;
private double c;
B(int a,int b,double c)
{
super(a);
this.b=b;
this.c=c;
System.out.println("This is constructor of
class B"); }
void show()
{
print();
System.out.println("b="+b);
System.out.println("c="+c);
} } // End of class B

```

```

/* OutPut
D:\java\bin>java inhtest1
This is constructor of class A
This is constructor of class B
a=10
b=8
c=4.5 */

```

```

class A
{
private int a;
A(int a)
{
this.a =a;
System.out.println("This is constructor of
class A");
}
void show()
{ System.out.println("a="+a); }
void display()
{
System.out.println("hello This is Display in
A");
}}

```

```

class inhtest1
{ public static void main(String args[])
{ B b1 = new B(10,8,4.5);
 b1.show();
} }

```

```

class B extends A
{
private int b;
private double c;
B(int a,int b,double c)
{ super(a);
 this.b=b;
 this.c=c;
 System.out.println("This is constructor of
class B"); }
void show()
{ super.show();
 System.out.println("b="+b);
 System.out.println("c="+c);
 display(); // no need super keyword
} }

```

```

/* OutPut
D:\java\bin>java inhtest1
This is constructor of class A
This is constructor of class B
a=10
b=8
c=4.5
hello This is Display in A */

```

```
class A
{
int a;
A(int a)
{ this.a =a; }
void show()
{
System.out.println("a="+a);
}
void display()
{
System.out.println("hello This is Display in A");
}
}
```

```
class inhtest2
{
public static void main(String args[])
{
B b1 = new B(10,20,8.4);
b1.show();
}
}
```

```
class B extends A
{
int b;
double c;
B(int a,int b,double c)
{
super(a);
this.b=b;
this.c=c;
}
void show()
{
//super.show();
System.out.println("a="+a);
System.out.println("b="+b);
System.out.println("c="+c);
}
}

/*
D:\java\bin>java inhtest2
a=10
b=20
c=8.4
*/
```

```
class A
{
int a;
A(int a)
{ this.a =a; }
}
```



**class B extends A**

```
{
// super class variable 'a' hides here
int a;
int b;
double c;
B(int a,int b,double c)
{
super(100);
this.a = a;
this.b=b;
this.c=c;
}
void show()
{
System.out.println("Super class a="+super.a);
System.out.println("a="+a);
System.out.println("b="+b);
System.out.println("c="+c);
}
}
```

**class inhtest2**

```
{
public static void main(String args[])
{
B b1 = new B(10,20,8.4);
b1.show();
}
}
```

**/\* Out Put**

**D:\java\bin>java inhtest2**

**Super class a=100**

**a=10**

**b=20**

**c=8.4**

**\*/**

# Date and Time

- Two constructors

**Date()** - initializes the object with the current date and time

**Date(long millisec)** - accepts one argument that equals the number of milliseconds that have elapsed since midnight, January 1, 1970

# An example

```
import java.util.Date;
public class DateDemo
{ public static void main(String args[])
 { // Instantiate a Date object
 Date date = new Date();
 // display time and date using toString()
 System.out.println(date.toString());
 }
}
```



# Date Formatting using **SimpleDateFormat:**

```
import java.util.*;
import java.text.*;
public class DateDemo
{
 public static void main(String args[])
 {
 Date dNow = new Date();
 SimpleDateFormat ft = new SimpleDateFormat ("E yyyy.MM.dd
'at' hh:mm:ss a zzz");
 System.out.println("Current Date: " + ft.format(dNow));
 }
}
```

Current Date: Sun 2004.07.18 at 04:14:09 PM PDT

# **Abstract class and Interfaces**



# Abstract Classes

- Sometimes we might want to declare a class whose full implementation is not known
- Allows declaring the method without writing any code for it in that class
- If a method is declared abstract, then the **class must also be declared as abstract**
- **private or static method cannot be declared abstract** because,
  - a private method cannot be accessed outside and
  - all **static methods are implicitly final**, they cannot be overridden

# Abstract Example

```
abstract class Mammal
{
 abstract void a();
}
```

```
public class Human extends Mammal
{
 public void a()
 {
 // body comes here
 }
}
```

```
public class GangMember extends
Mammal
{
 public void a()
 {
 // definition
 } }
```

```
public class Dog extends Mammal
{
 public void a()
 {
 // definition
 }
}
```

# Implementing Interfaces

- An **abstract** class will have some methods that are declared as abstract and some that are not
- If an entire class is to be declared abstract, **interface** is the best choice
- An **interface** is an entirely abstract class
- Interface can also be derived in a manner as deriving a subclass from another class

# Interface Example

```
interface Clock {
 public String GetTime(int hour); }

class Cuckoo implements Clock {
 public String GetTime(int hour) {
 StringBuffer str = new StringBuffer();
 for (int i=0; i < hour; i++)
 str.append("Cuckoo ");
 return str.toString();
 } }

 class Watch implements Clock {
 public String GetTime(int hour) {
 return new String("It is " + hour + ":00");
 } }
}
```

# Type of Errors: 3 types

*Syntax errors*: rules of lang not followed. They are detected by the compiler

- Misspelled variable and function names,
- Missing semicolons,
- Improperly matches parentheses and curly braces
- Incorrect format in selection and loop statements

*Runtime errors*: occur while program is running and if the env. detects an operation that is impossible to carry out

- divide by zero error
- trying to open a file that doesn't exist

# Type of Errors: 3 types

*Logical errors:* occur when a program doesn't perform the way it was intended to

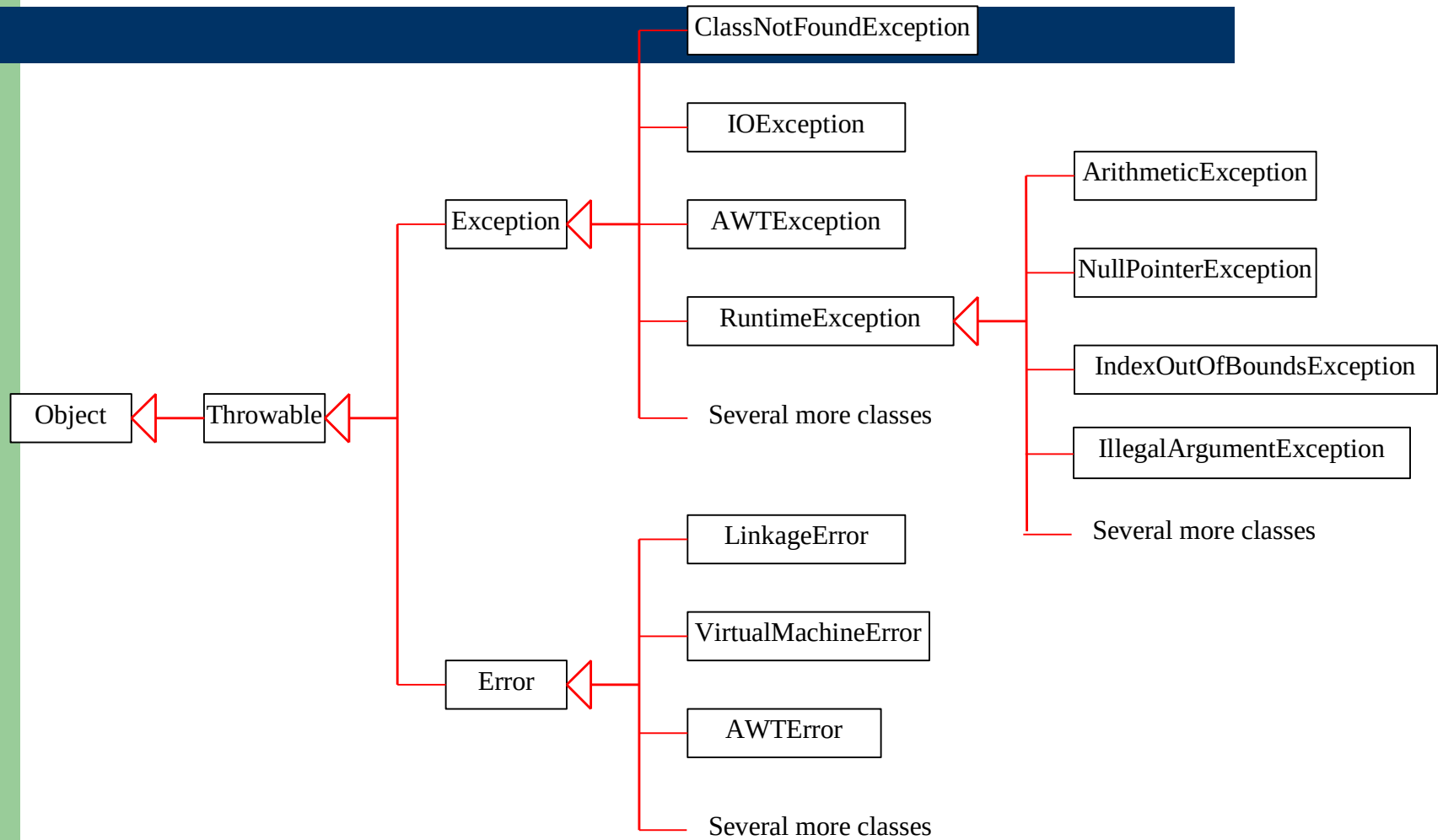
- Multiplying when you should be dividing
- Adding when you should be subtracting
- Opening and using data from the wrong file
- Displaying the wrong message



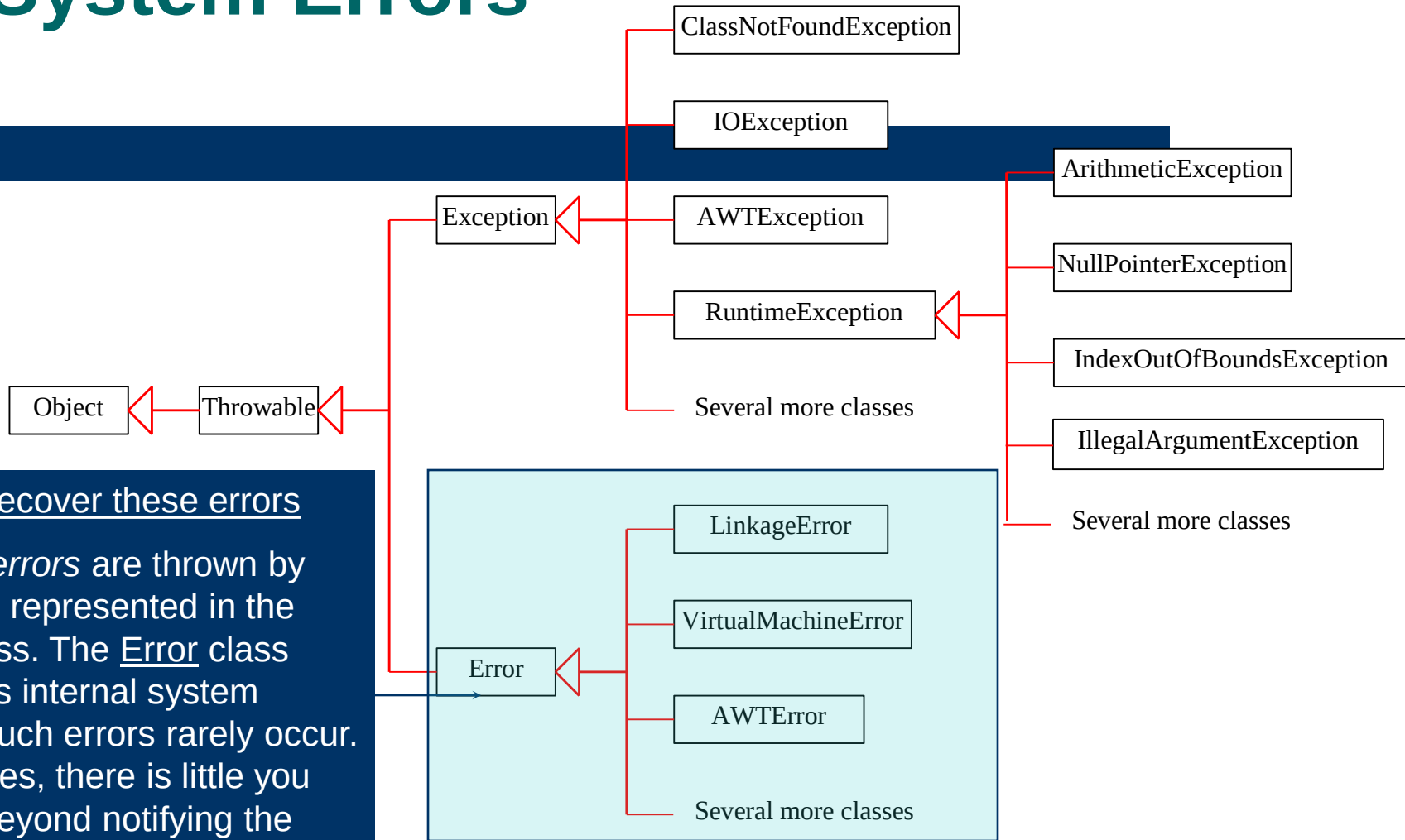
# What Is an Exception?

- The term *exception* is shorthand for the phrase "exceptional event."
- **Definition:** An *exception* is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

# Exception Classes



# System Errors

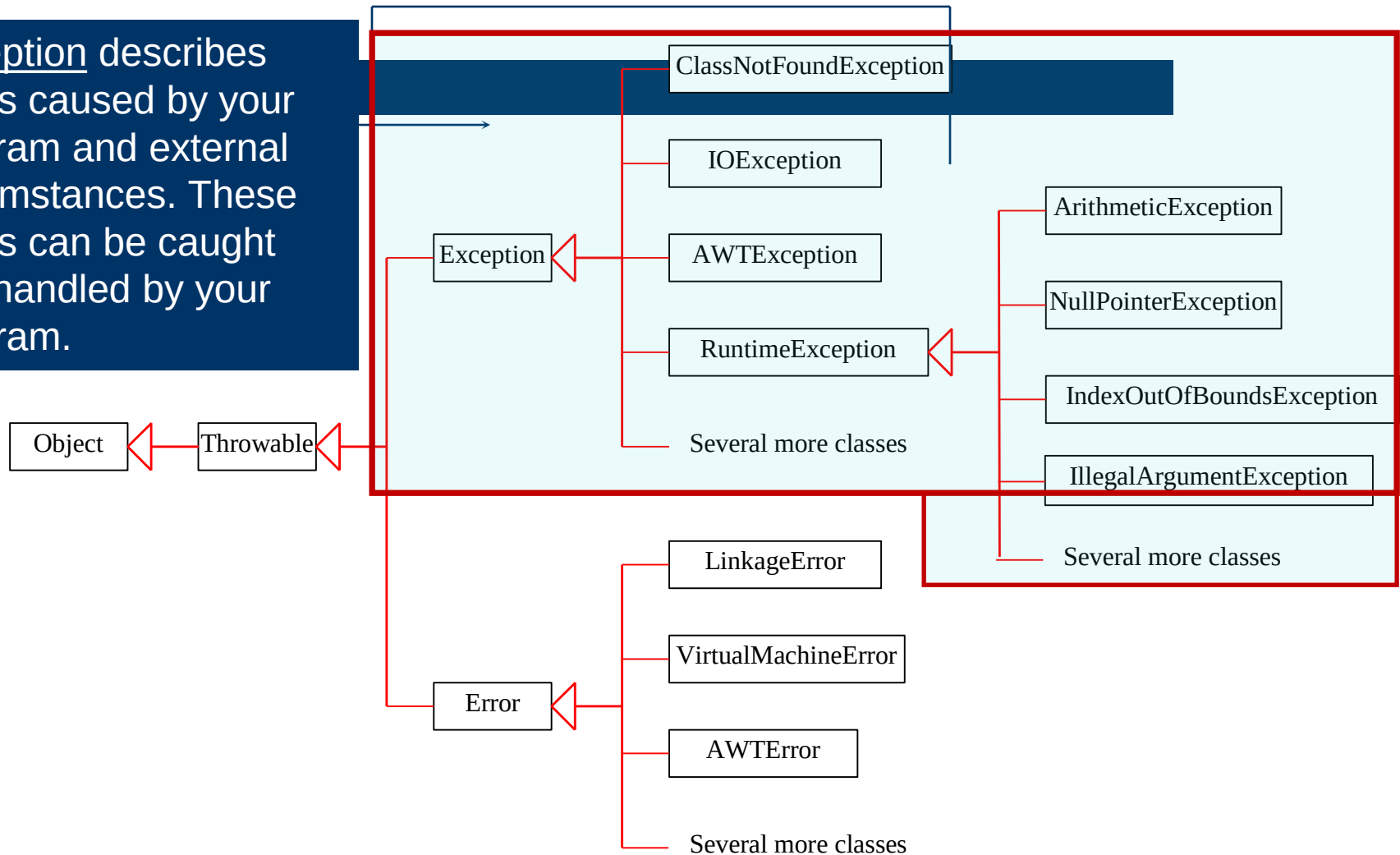


## Cannot recover these errors

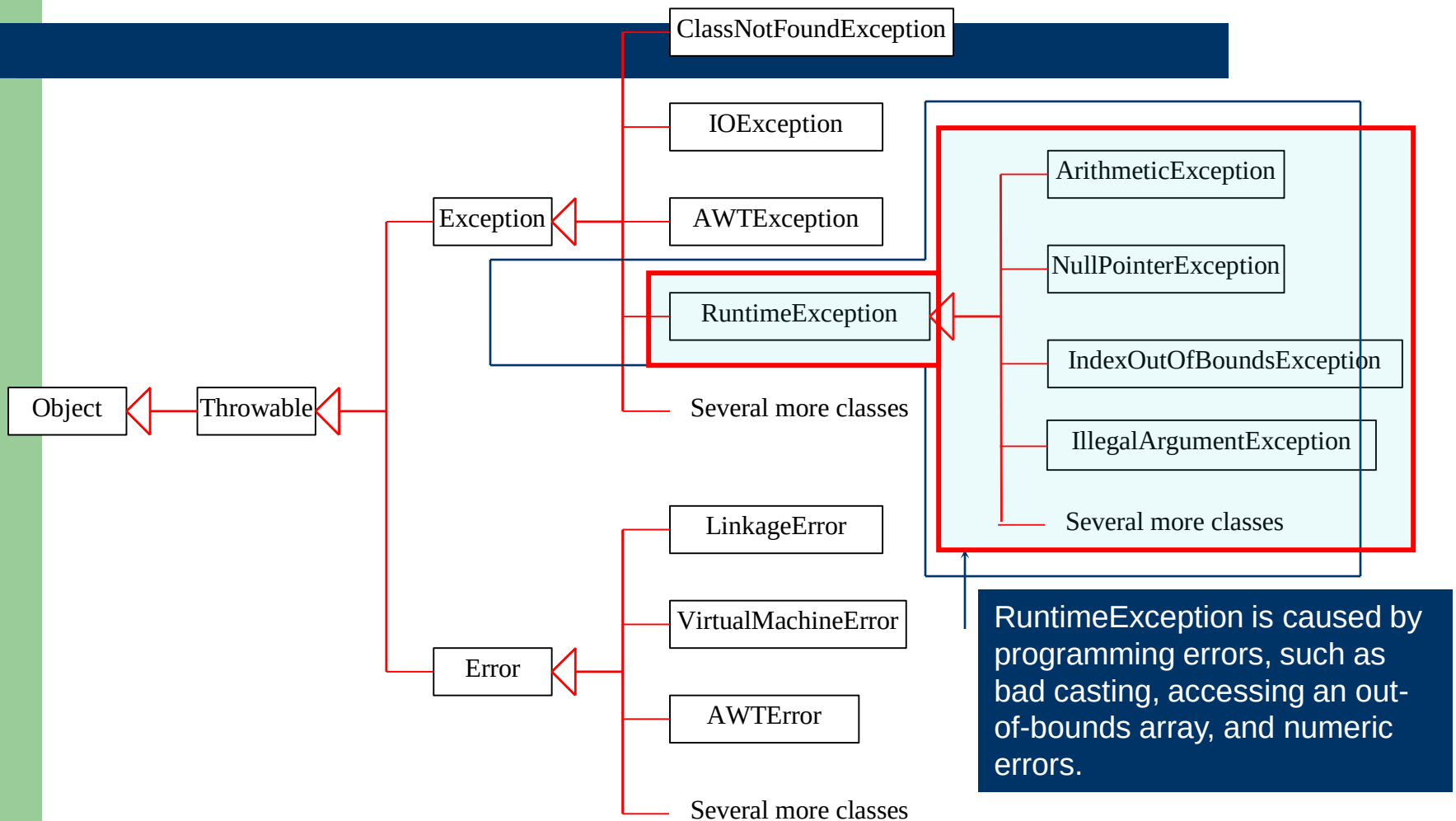
*System errors* are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

# Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



# Runtime Exceptions



# Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked* exceptions.

Ex: bad data provided by user during the user-program interaction

All other exceptions are known as *checked* exceptions, meaning that the compiler forces the programmer to check and deal with the exceptions.

These exceptions are ***checked*** at Compile time.

# Eliminating Software Errors: Exception handling

- Exception in Java is abnormal state at runtime
- Whenever an exception occurs, the program code should handle it, else the program execution is terminated by the runtime system
- Java's exception handling makes use of 5 keywords - **try**, **catch**, **throw**, **throws** and **finally**

# logic

- When an error occurs within a method,
- the method creates an object and hands it off to the runtime system.
- The object, called an exception object, contains information about the error (How?), including its type and the state of the program when the error occurred.
- Creating an exception object and handing it to the runtime system is called throwing an exception.



# Exception handling



# What is an Exception?

- The term *exception* is shorthand for the phrase "exceptional event."
- **Definition:** An *exception* is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

# The try-catch statement

- When an exception occurs in a program, an object of `java.lang.Throwable` hierarchy representing the error state is thrown(executed)
- The thrown exception object has to be caught (handled) by the program code
- If it is not caught, the program execution is not permitted to continue.
- Put the code that could generate an exception within a `try` block as shown below:

```
..... // more statements
try{
/* code to be tested */
}
..... // more statements
```

# The try-catch statement cont.

- The statements enclosed in a **try** block are tested for any possible exceptions
- Every **try** block has an associated **catch** block which actually performs processing of the exception
- The keyword **catch** is followed by a pair of parenthesis which specifies an exception object that the catch block handles
- So the above code changes to the following:

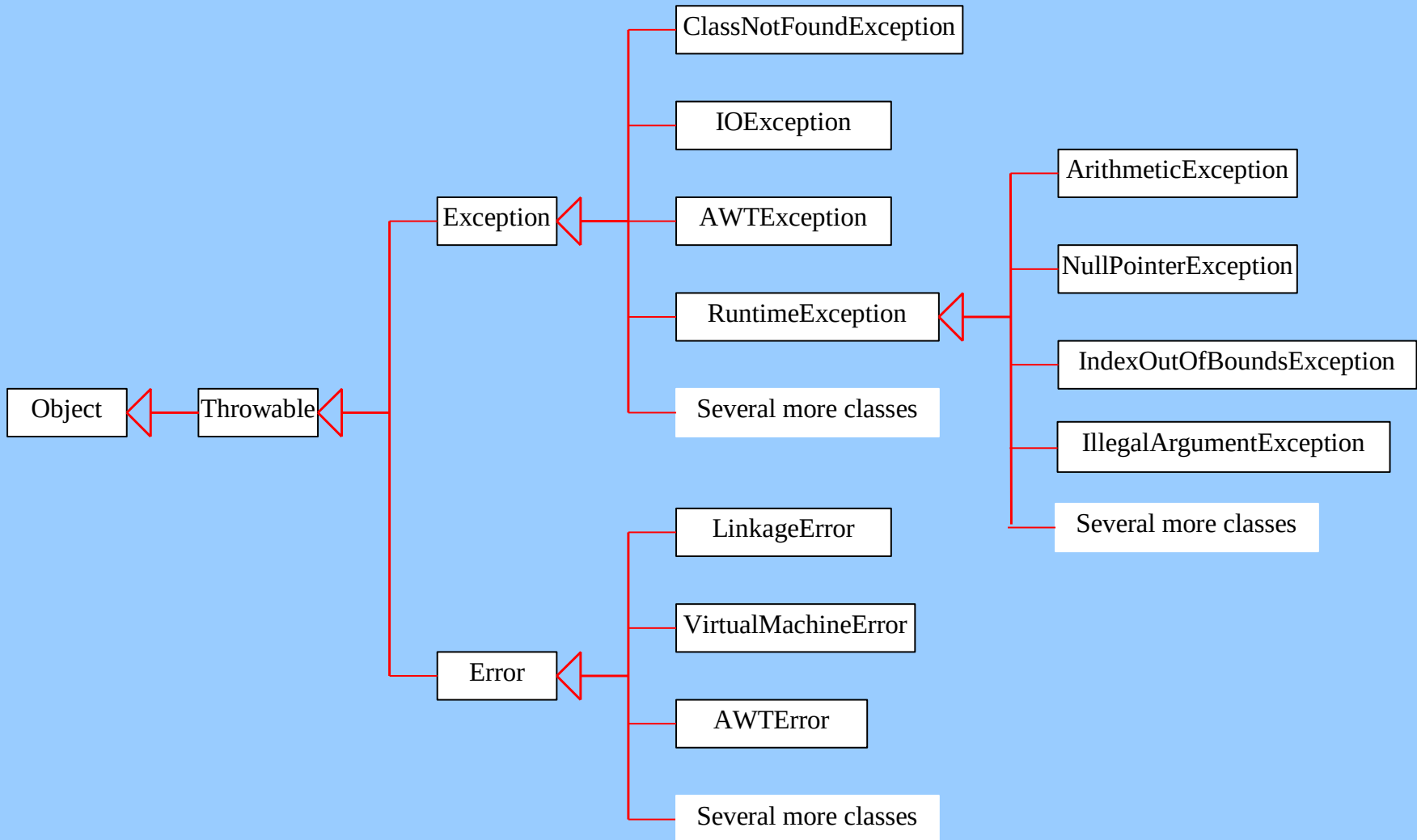
# The try-catch statement cont.

```
..... // more statements
try{
 /* code to be tested */
}
catch (Exception e)
{
 /* exception handling code here */
}
..... // more statements
```

# Throwable class

- `java.lang.Throwable` has two important subclasses, namely `java.lang.Exception` and `java.lang.Error`
- The `java.lang.Exception` and its subclasses handle the abnormalities in the program code that might arise during compile time or runtime
- `java.lang.Error` and its subclasses handle error conditions related to hardware, for example, memory not available
  - Linkage error, Virtual Machine error, etc...

# Exception Classes



# Throwable class cont.

- Hierarchy of `java.lang.Error` being associated with hardware,
- we as programmers will not use it directly, but often deal with `java.lang.Exception` and its subclasses
- One important subclass of `java.lang.Exception` is the class `java.lang.RuntimeException`.
- `RuntimeException` and its subclasses, as its name specifies, are responsible to handle scenarios where exceptions are thrown at runtime



# Exception class

- **catch** block specifies an object of one of `java.lang.Exception` hierarchy
- It indicates the type of exception being handled by the `catch` block
- Of course, if `catch` block uses `java.lang.Exception` object, all runtime and compile time exceptions are handled since all other associated exceptions classes are derived from `java.lang.Exception`.
- This might make the code simple but does not allow us to handle different exception cases individually
- To do so, multiple `catch` blocks may be used with a single **try** block as follows:

# Example

```
try
{ //code to be tested
}
catch (ArrayIndexOutOfBoundsException e1)
{ //Handling of Array Indexing problems
}
catch (ArrayStoreException e2)
{ //Handling wrong type of object reference problems
}
catch (Exception e3)
{ //Handling all other exceptions //should be at last
}
```

# Nesting try-catch statement

- The try-catch blocks can also be enclosed within another try block as shown below:

```
try{
 //code 1
 try{ //code2 }
 catch (ArithmeticException e)
 { // exception handling for arithmetic exceptions }
 //code 3
}
catch (Exception e)
{ // exception handling }
```

# The finally statement

- If an exception occurs in a try block, the remaining code (if any) within the same try block after the exception generating statement are skipped
- Also, if the associated catch block does not handle the exception, the execution control jumps to outer catch block (if any) or the program is terminated
- The code contained in a finally block is always executed irrespective of whether an exception is thrown or not.

```
try{ //code to be tested }
catch (Exception e)
{ //exception handling code here }
finally
{ // statements that should be executed
 compulsorily
}
```

# The throws keyword

- If not want to include try-catch statement in program code
- prefer handling the exception elsewhere
- exception cannot be compiled until tested with try-catch
- **throws** keyword should be used in the **method definition** as follows:

```
void function1 () throws ArithmeticException
{ // method code }
```

# The throws keyword cont.

- A method may also use multiple exception with the throws clause

```
void function1 () throws ArithmeticException,
 ArrayStateException
```

```
{
 // method code
}
```

Use ‘,’ (comma)

# The throw keyword

- all the examples discussed earlier, we saw that the exceptions are thrown either by the compiler or at runtime by JRE
- a programmer can make his program check for errors and decide to throw the exception by itself using **throw**

```
try{ //code to be tested }
catch (Exception e)
{ //exception handling code here
 throw e; }
```



# Throw in try block

```
try{
 //code to be tested
 throw new ArithmeticException();
} catch (Exception e)
{
 //exception handling code here
}
```

# Creating user defined exceptions

- exception objects used till were predefined
- developers can create their own exception classes
- We simply need to write a subclass of Exception and override the **toString()** method
- Whenever an exception object is printed, the respective objects **toString** method is invoked

# Catching Exceptions

```
main method {
 ...
 try {
 ...
 invoke method1;
 statement1;
 }
 catch (Exception1 ex1) {
 Process ex1;
 }
 statement2;
}
```

```
method1 {
 ...
 try {
 ...
 invoke method2;
 statement3;
 }
 catch (Exception2 ex2) {
 Process ex2;
 }
 statement4;
}
```

```
method2 {
 ...
 try {
 ...
 invoke method3;
 statement5;
 }
 catch (Exception3 ex3) {
 Process ex3;
 }
 statement6;
}
```

An exception  
is thrown in  
method3

# Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).



```
void p1() {
 try {
 p2();
 }
 catch (IOException ex) {
 ...
 }
}
```

(a)

```
void p1() throws IOException {
 p2();
}
```

(b)

# The `finally` Clause

```
try {
 statements;
}
catch (TheException ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

# Trace a Program Execution

Suppose no exceptions in the statements

```
try {
 statements;
}
catch (TheException ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

Next statement;

# Trace a Program Execution

```
try {
 statements;
}
catch (TheException ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

The final block is always executed

Next statement;

# Trace a Program Execution

```
try {
 statements;
}
catch (TheException ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

Next statement;

Next statement in the  
method is executed



# Trace a Program Execution

```
try {
 statement1;
 statement2;
 statement3;
}
catch (Exception1 ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

Suppose an exception of  
type Exception1 is thrown in  
statement2

Next statement;

# Trace a Program Execution

```
try {
 statement1;
 statement2;
 statement3;
}
catch (Exception1 ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

The exception is handled.

Next statement;

# Trace a Program Execution

```
try {
 statement1;
 statement2;
 statement3;
}
catch (Exception1 ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

The final block is always executed.

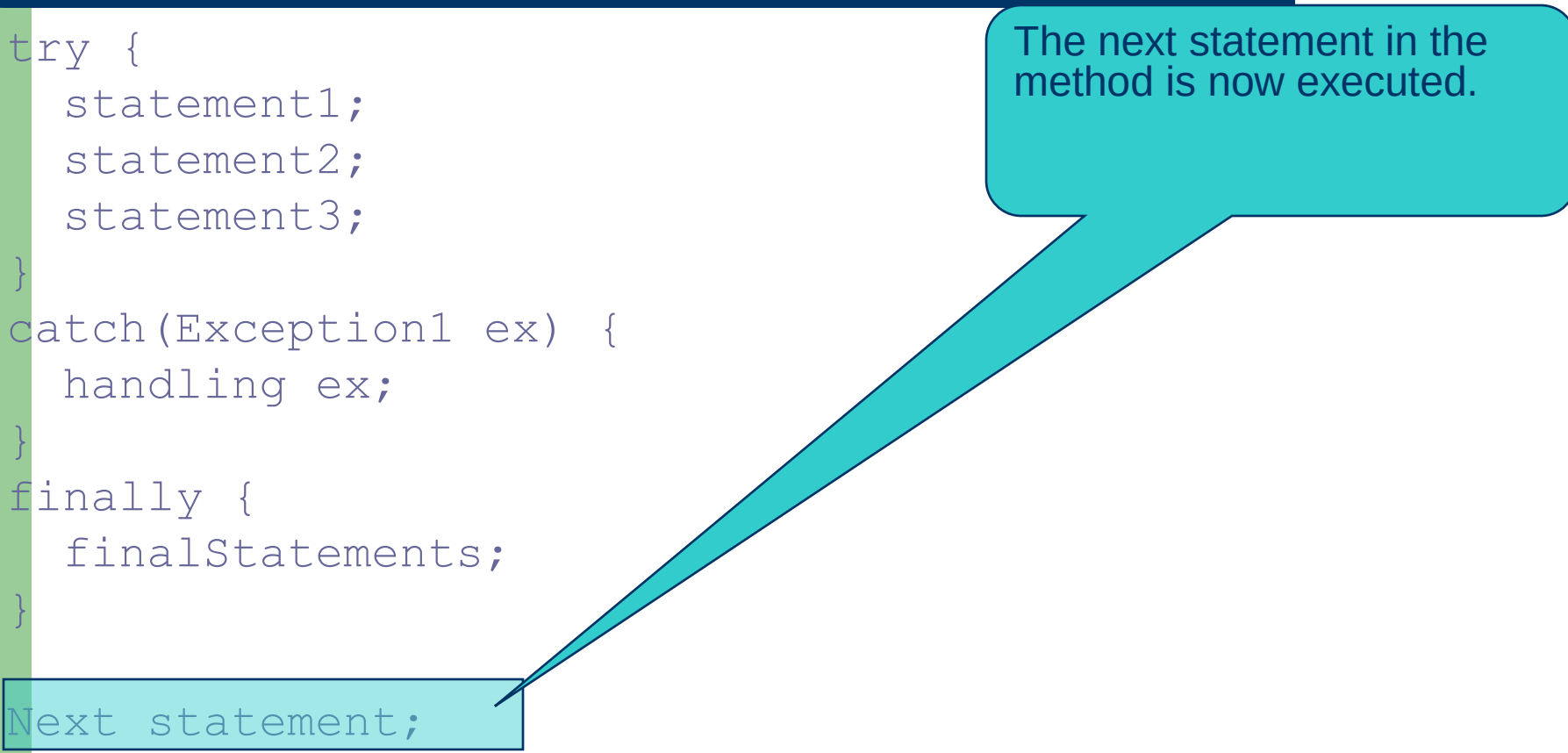
Next statement;

# Trace a Program Execution

```
try {
 statement1;
 statement2;
 statement3;
}
catch (Exception1 ex) {
 handling ex;
}
finally {
 finalStatements;
}
```

The next statement in the method is now executed.

Next statement;

A diagram illustrating program execution flow. A teal callout bubble at the top right contains the text "The next statement in the method is now executed." A teal arrow points from this bubble to a teal rectangular box at the bottom left containing the text "Next statement;". This box is positioned below the 'finally' block of the code snippet, indicating the next step in the execution process.

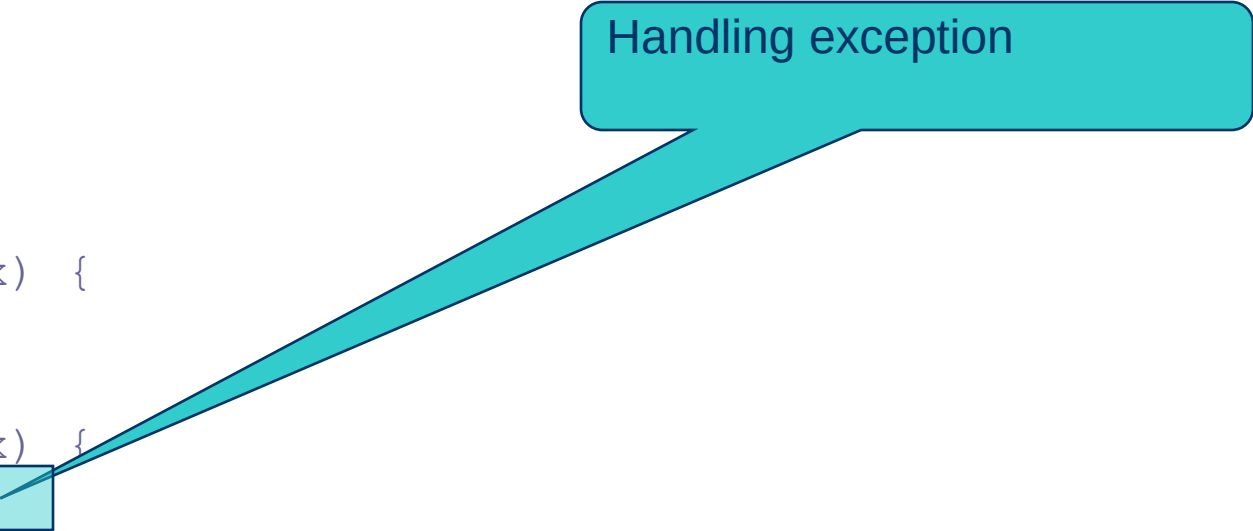
# Trace a Program Execution

```
try {
 statement1;
 statement2;
 statement3;
}
catch (Exception1 ex) {
 handling ex;
}
catch (Exception2 ex) {
 handling ex;
 throw ex;
}
finally {
 finalStatements;
}
Next statement;
```

statement2 throws an exception of type Exception2.

# Trace a Program Execution

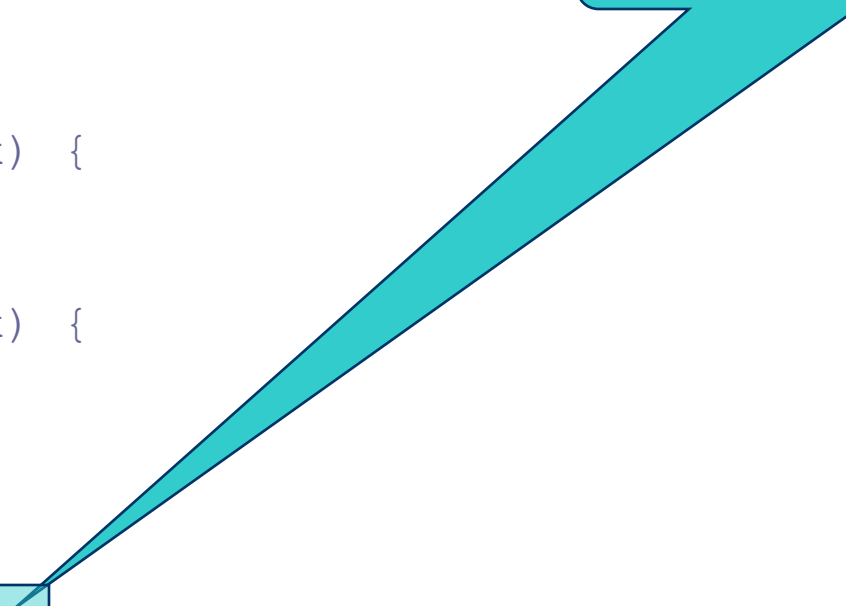
```
try {
 statement1;
 statement2;
 statement3;
}
catch(Exception1 ex) {
 handling ex;
}
catch(Exception2 ex) {
 handling ex;
 throw ex;
}
finally {
 finalStatements;
}
Next statement;
```



Handling exception

# Trace a Program Execution

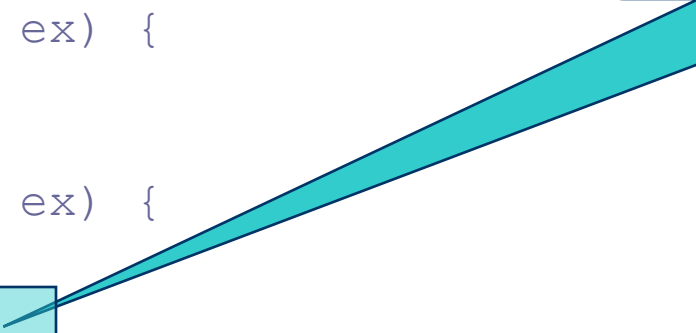
```
try {
 statement1;
 statement2;
 statement3;
}
catch(Exception1 ex) {
 handling ex;
}
catch(Exception2 ex) {
 handling ex;
 throw ex;
}
finally {
 finalStatements;
}
Next statement;
```



Execute the final block

# Trace a Program Execution

```
try {
 statement1;
 statement2;
 statement3;
}
catch (Exception1 ex) {
 handling ex;
}
catch (Exception2 ex) {
 handling ex;
 throw ex;
}
finally {
 finalStatements;
}
Next statement;
```



Rethrow the exception and control is transferred to the caller



# When to Use Exceptions

When should you use the try-catch block in the code?

You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
 System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
 System.out.println("refVar is null");
}
```

# When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
 System.out.println(refVar.toString());
else
 System.out.println("refVar is null");
```

# When to Use Exceptions

When should you use the try-catch block in the code?

You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
 System.out.println(refVar.toString());
}

catch (NullPointerException ex) {
 System.out.println("refVar is null");
}
```

# When to Use Exceptions

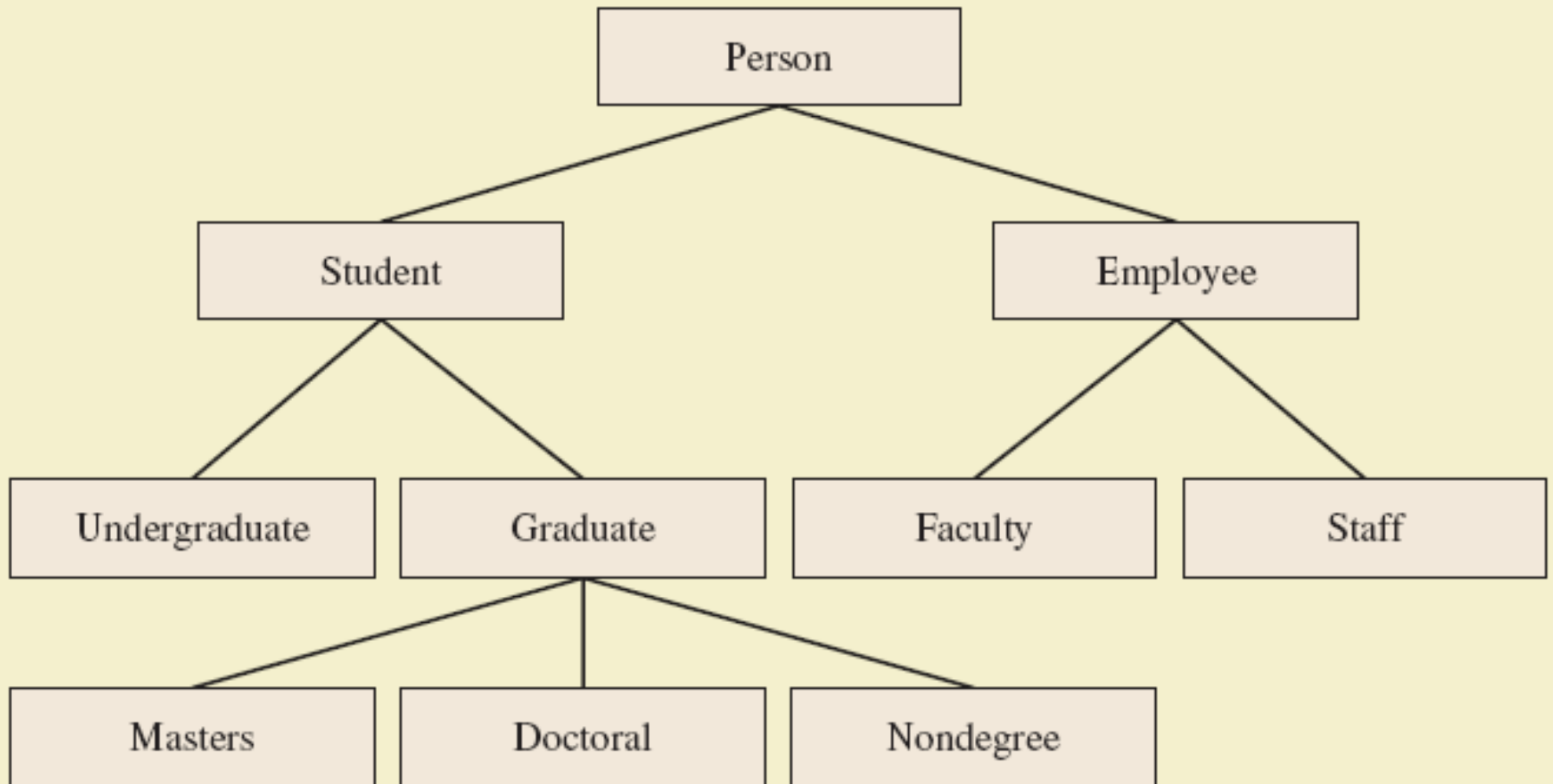
is better to be replaced by

```
if (refVar != null)
 System.out.println(refVar.toString());
else
 System.out.println("refVar is null");
```

# Polymorphism

# Derived Classes

A class hierarchy



# Inheritance Example

- **Person** is the **base** (or **parent**) **class**
- **Student** is the **derived** (or **child**) class
- Base classes are more general.
- Derived classes are more specific.
- Inheritance greatly enhances the ability to reuse existing code.
  - This simplifies the development time.
  - And makes design much simpler and cleaner.

# Polymorphism

- From the Greek
  - “poly” = many
  - “morph” = form, figure
- The ability of a **class method** to do different things based on the **object** it is acting upon.
  - Example: **Animal.makeNoise()** ;
  - **Dog**: barks
  - **Cat**: meows



# Polymorphism

## Two types of polymorphism

### 1. Method Overloading:

- Multiple implementations of the same method occur in the same class.
- Each differs in the number and types of the method arguments.
- Java invokes the closest one that matches the actual arguments being passed to the method.
  - `System.out.println ( int );`
  - `System.out.println( char );`
  - `System.out.println ( String ) ;`
  - `System.out.println ( boolean );`

**Constructors**

# Polymorphism

## Two types of polymorphism

### 2. Method Overriding:

- Multiple implementations of the same method occur in different classes along the same hierarchy.
- A derived class “overrides” the implementation of a method provided by its base class.
- Examples:
  - `Cat.makeNoise()` overrides `Animal.makeNoise()`
  - `ACat.makeNoise()` overrides `Cat.makeNoise()`

# Method Overriding

- Subclasses inherit all methods from their superclass
  - Sometimes, the implementation of the method in the superclass does not provide the functionality required by the subclass.
  - In these cases, the method must be overridden.
- To override a method, provide an implementation in the subclass.
  - The method in the subclass MUST have the exact same signature as the method it is overriding.

# Method overriding - Example

---

```
public class BankAccount
{
 private String ownersName;
 private int accountNumber;
 protected float balance;

 public void deposit(float anAmount)
 {
 if (anAmount>0.0)
 balance = balance + anAmount;
 }

 public void withdraw(float anAmount)
 {
 if ((anAmount>0.0) &&
 (balance>anAmount))
 balance = balance - anAmount;
 }

 public float getBalance()
 {
 return balance;
 }
}
```

```
public class OverdraftAccount extends
 BankAccount
{
 private float limit;

 public void withdraw(float anAmount)
 {
 if ((anAmount>0.0) &&
 (getBalance()+limit>anAmount))
 balance = balance - anAmount;
 }
}
```

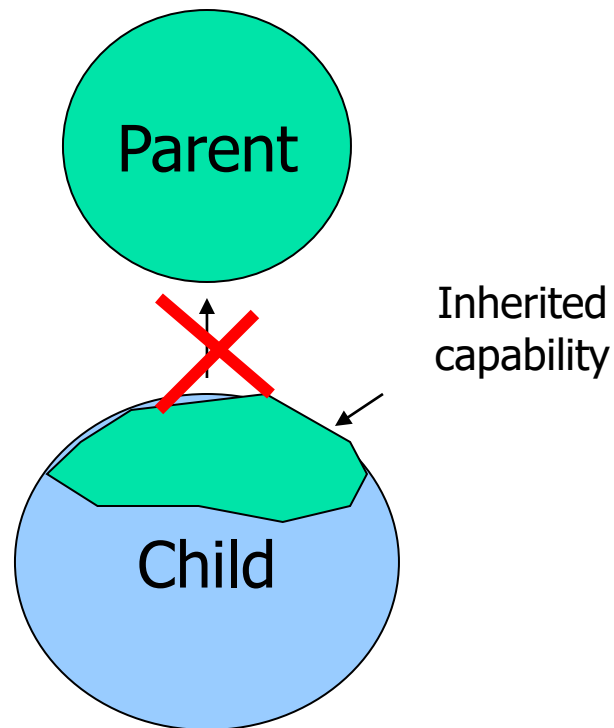
# Overriding vs. Overloading

- Do not confuse overriding with overloading
  - **Overriding** takes place in the subclass – new method with same signature
  - **Overloading** takes place in the same class – new method with different signature

# Final and Abstract Classes

# Restricting Inheritance

It is possible to specify that a method cannot be overridden in a subclass



# Final Members: A way for Preventing Overriding of Members in Subclasses

- All methods and variables can be overridden by default in subclasses.
- This can be prevented by declaring them as final using the keyword "final" as a modifier. For example:
  - `final int marks = 100;`
  - `final void display();`
- This ensures that functionality defined in this method cannot be altered. Similarly, the value of a final variable cannot be altered.



# Final Classes: A way for Preventing Classes being extended

- We can prevent an inheritance of classes by other classes by declaring them as final classes.
- This is achieved in Java by using the keyword final as follows:

```
final class Marks
```

```
{ // members
```

```
}
```

```
final class Student extends Person
```

```
{ // members
```

```
}
```

- Any attempt to inherit these classes will cause an error.

# Abstract Classes and Methods

- So far, our base classes provide an implementation **for all of its methods**.
  - This means that objects from base classes can actually be created.
  - e.g. **Person p = new Person( );**
- What if we cannot provide an implementation for a particular method?

# Abstract Classes

- When we define a class to be “final”, it cannot be extended. In certain situation, we want that properties of classes to be always extended and used. Such classes are called Abstract Classes
- An abstract class is a conceptual class.
- An Abstract class cannot be instantiated – objects cannot be created.
- Abstract classes provides a common root for a group of classes, nicely tied together in a package:

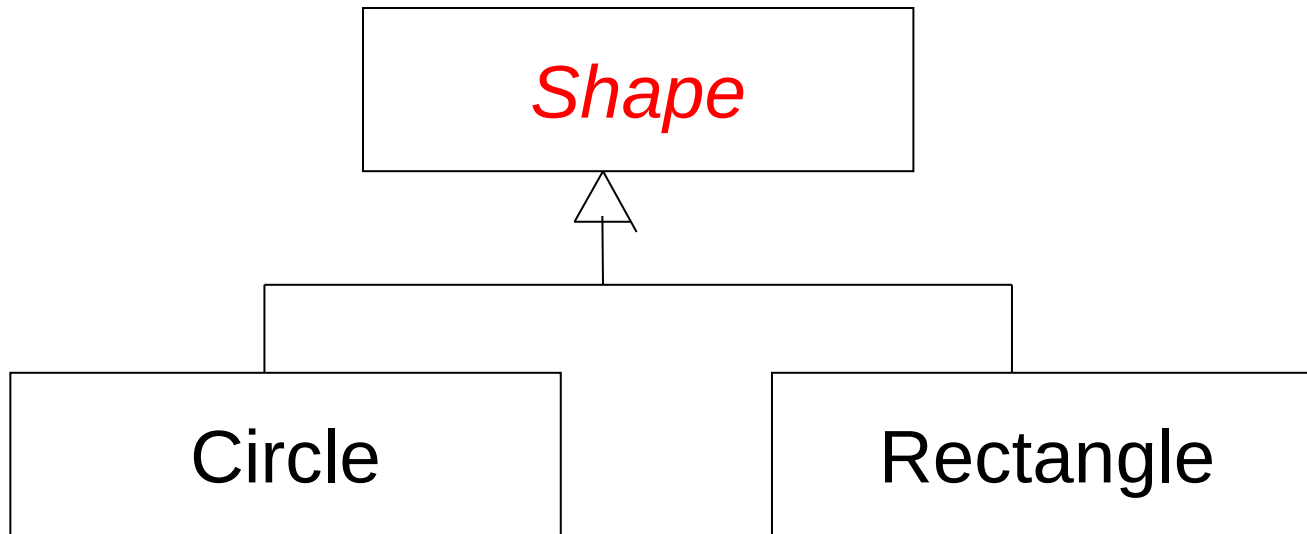
# Abstract Class Syntax

```
abstract class ClassName
{
 ...
 ...
 abstract Type MethodName1();
 ...
 ...
 Type Method2()
 {
 // method body
 }
}
```

- When a class contains one or more abstract methods, it should be declared as abstract class.
- The abstract methods of an abstract class must be defined in its subclass.
- We cannot declare abstract constructors or abstract static methods.

# Abstract Class -Example

- Shape is a abstract class.



# The Shape Abstract Class

```
public abstract class Shape {
 public abstract double area();
 public void move() { // non-abstract method
 // implementation
 }
}
```

- Is the following statement valid?
  - Shape s = new Shape();
- No. It is illegal because the Shape class is an abstract class, which cannot be instantiated to create its objects.

# Abstract Classes

```
public Circle extends Shape {
 protected double r;
 protected static final double PI = 3.1415926535;
 public Circle() { r = 1.0; }
 public double area() { return PI * r * r; }
 ... }

```

```
public Rectangle extends Shape {
 protected double w, h;
 public Rectangle() { w = 0.0; h = 0.0; }
 public double area() { return w * h; }
}
```

# Abstract Classes Properties

- A class with one or more abstract methods is automatically abstract and it cannot be instantiated.
- A class declared abstract, even with no abstract methods can not be instantiated.
- A subclass of an abstract class can be instantiated if it overrides all abstract methods by implement them.
- A subclass that does not implement all of the superclass abstract methods is itself abstract; and it cannot be instantiated.



# Summary

- If you do not want (properties of) your class to be extended or inherited by other classes, define it as a final class.
  - Java supports this is through the keyword “final”.
  - This is applied to classes.
- You can also apply the final to only methods if you do not want anyone to override them.
- If you want your class (properties/methods) to be extended by all those who want to use, then define it as an abstract class or define one or more of its methods as abstract methods.
  - Java supports this is through the keyword “abstract”.
  - This is applied to methods only.
  - Subclasses should implement abstract methods; otherwise, they cannot be instantiated.

# Interfaces

Design Abstraction  
and  
a way for loosely realizing Multiple  
Inheritance

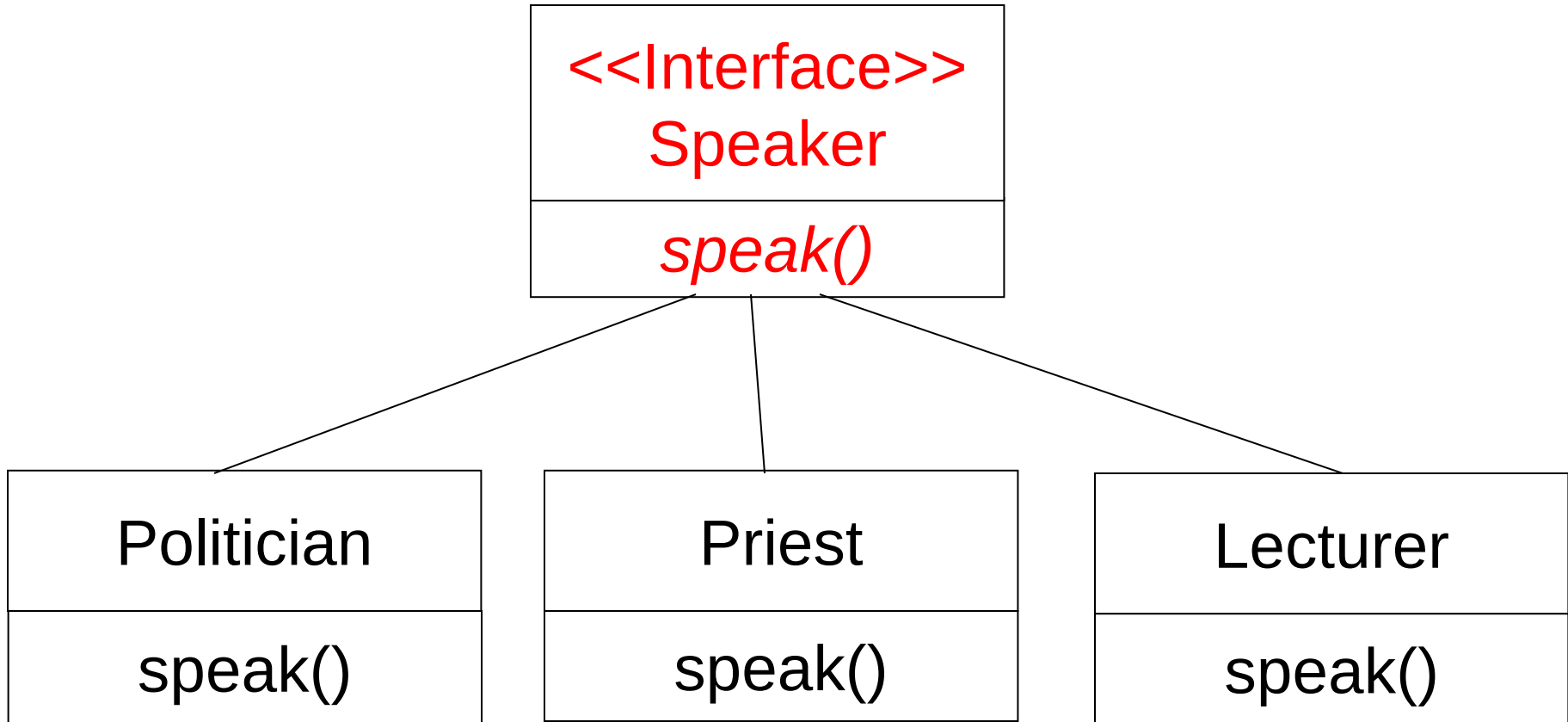
# Interfaces

- ***Interface*** is a conceptual entity similar to a Abstract class.
- Can contain only constants (final variables) and abstract method (no implementation) - Different from Abstract classes.
- Use when a number of classes share a common interface.
- Each class should implement the interface.

# Interfaces: An informal way of realising multiple inheritance

- An interface is basically a kind of class—it contains methods and variables, but they have to be only abstract methods and final variables.
- Therefore, it is the responsibility of the class that implements an interface to supply the code for methods.
- A class can implement any number of interfaces, but cannot extend more than one class at a time.
- Therefore, interfaces are considered as an informal way of realizing multiple inheritance in Java.

# Interface - Example



# Interfaces Definition

- Syntax (appears like abstract class):

```
interface InterfaceName {
 // Constant/Final Variable Declaration
 // Methods Declaration – only method body
}
```

- Example:

```
interface Speaker {
 public void speak();
}
```

# Implementing Interfaces

- Interfaces are used like super-classes whose properties are inherited by classes. This is achieved by creating a class that implements the given interface as follows:

```
class ClassName implements InterfaceName [, InterfaceName2, ...]
{
 // Body of Class
}
```

# Implementing Interfaces Example

```
class Politician implements Speaker {
 public void speak(){
 System.out.println("Talk politics");
 }
}
```

```
class Priest implements Speaker {
 public void speak(){
 System.out.println("Religious Talks");
 }
}
```

```
class Lecturer implements Speaker {
 public void speak(){
 System.out.println("Talks Object Oriented Design and Programming!");
 }
}
```



# Extending Interfaces

- Like classes, interfaces can also be extended. The new sub-interface will inherit all the members of the superinterface in the manner similar to classes. This is achieved by using the keyword `extends` as follows:

```
interface InterfaceName2 extends InterfaceName1
{
 // Body of InterfaceName2
}
```

# Inheritance and Interface Implementation

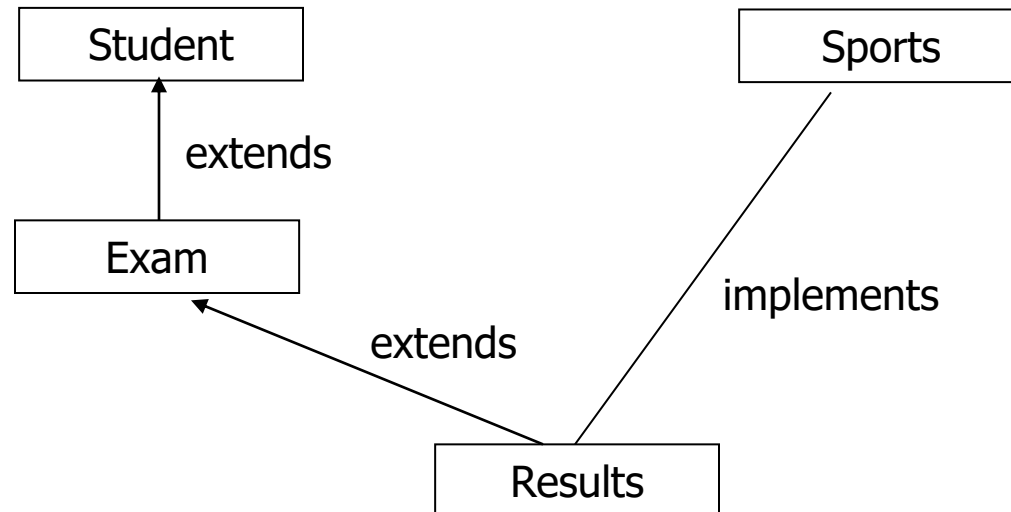
- A general form of interface implementation:

```
class ClassName extends SuperClass implements InterfaceName [,
InterfaceName2, ...]
{
 // Body of Class
}
```

- This shows a class can extended another class while implementing one or more interfaces. It appears like a multiple inheritance (if we consider interfaces as special kind of classes with certain restrictions or special features).

# Student Assessment Example

- Consider a university where students who participate in the national games or Olympics are given some grace marks. Therefore, the final marks awarded = Exam\_Marks + Sports\_Grace\_Marks. A class diagram representing this scenario is as follow:



# Software Implementation

```
class Student
{
 // student no and access methods
}
interface Sport
{
 // sports grace marks (say 5 marks) and abstract methods
}
class Exam extends Student
{
 // example marks (test1 and test 2 marks) and access methods
}
class Results extends Exam implements Sport
{
 // implementation of abstract methods of Sport interface
 // other methods to compute total marks = test1+test2+sports_grace_marks;
 // other display or final results access methods
}
```

# Interfaces and Software Engineering

- Interfaces, like abstract classes and methods, provide templates of behaviour that other classes are expected to implement.
- Separates out a design hierarchy from implementation hierarchy. This allows software designers to enforce/pass common/standard syntax for programmers implementing different classes.

# A Summary of OOP and Java Concepts Learned So Far

# Summary

- Class is a collection of data and methods that operate on that data
- An object is a particular instance of a class
- Object members accessed with the 'dot' (Class.v)
- Instance variables occur in each instance of a class
- Class variables associated with a class
- Objects created with the new keyword

# Summary

- Objects are not explicitly 'freed' or destroyed. Java automatically reclaims unused objects.
- Java provides a default constructor if none defined.
- A class may inherit the non-private methods and variables of another class by subclassing, declaring that class in its 'extends' clause.
- `java.lang.Object` is the default superclass for a class. It is the root of the Java hierarchy.



# Summary

- Method overloading is the practice of defining multiple methods which have the same name, but different argument lists
- Method overriding occurs when a class redefines a method inherited from its superclass
- static, private, and final methods cannot be overridden
- From a subclass, you can explicitly invoke an overridden method of the superclass with the 'super' keyword.

# Summary

- Data and methods may be hidden or encapsulated within a class by specifying the private or protected visibility modifiers.
- An abstract method has no method body. An abstract class contains abstract methods.
- An interface is a collection of abstract methods and constants. A class implements an interface by declaring it in its 'implements' clause, and providing a method body for each abstract method.

# Multithreaded Programming using Java Threads

# Our Goals

- Appreciate the (increasing) importance of parallel programming
- Understand fundamental concepts:
  - Parallelism, threads, multi-threading, concurrency, locks, etc.
- See some basics of this in Java

# A single threaded program

```
class ABC
```

```
{
```

```
....
```

```
 public void main(..)
```

```
 {
```

```
 ...
```

```
 ..
```

```
 }
```

```
}
```

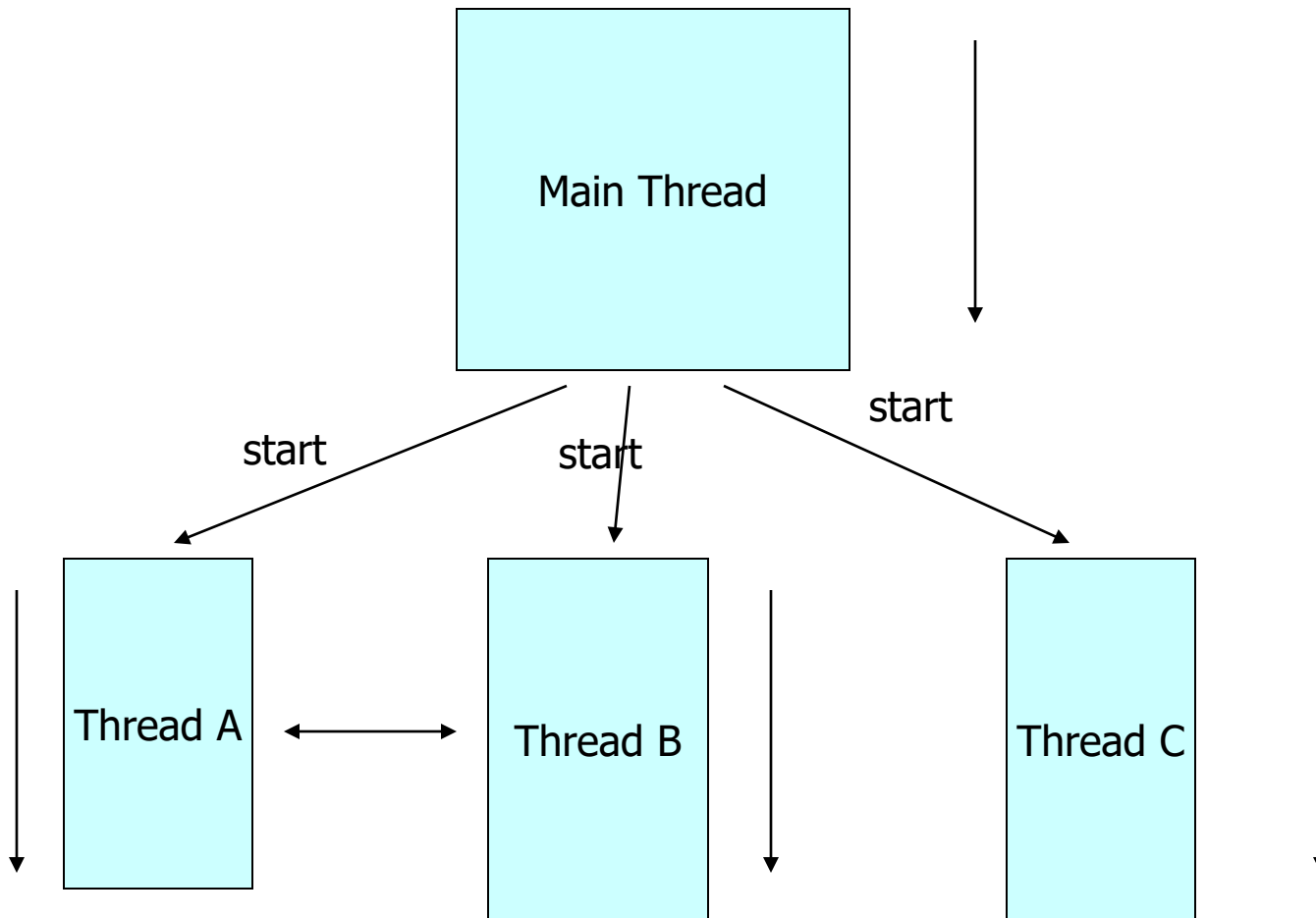
begin

body

end

Difference between a program, process, thread ?

# A Multithreaded Program



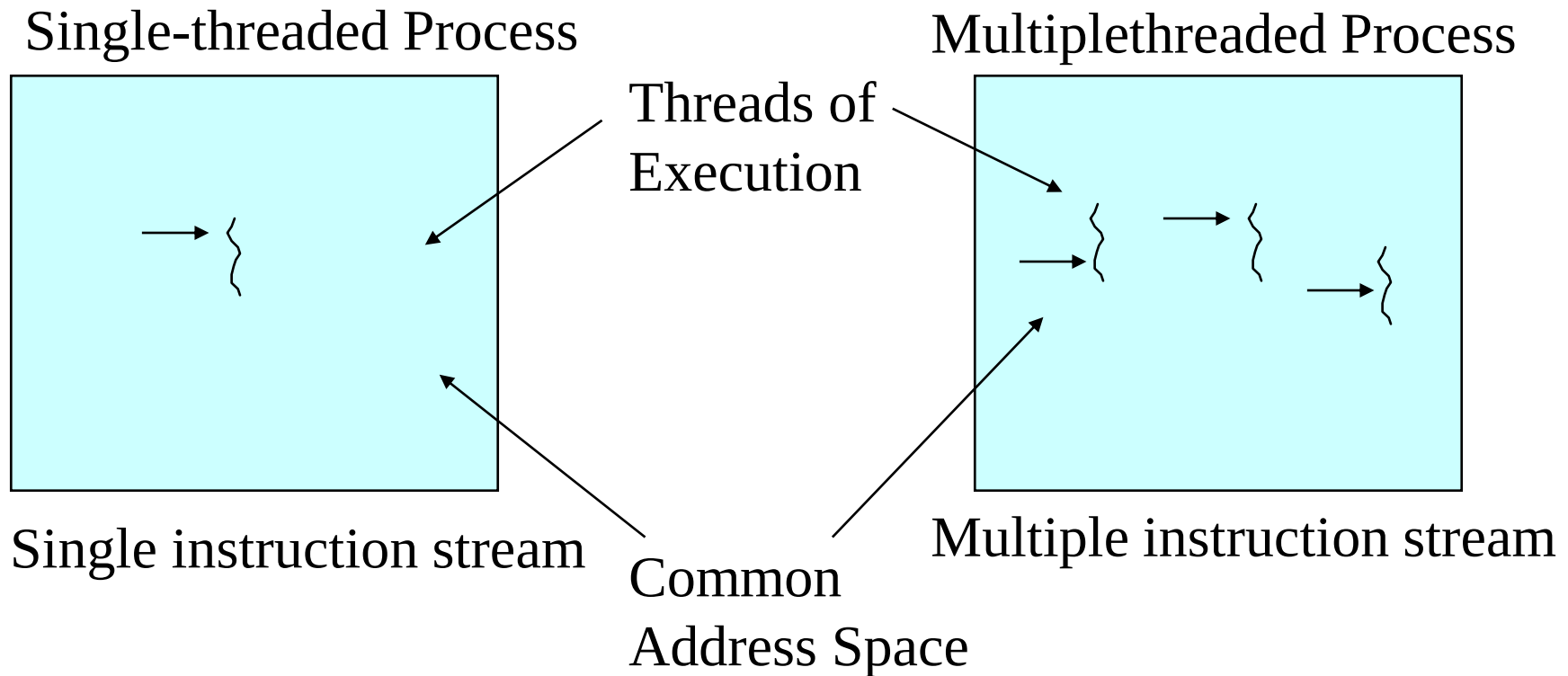
Threads may switch or exchange data/results

# Why use Multithreading?

- In a **single threaded application**, one thread of execution do everything
  - If an application has several tasks to perform, tasks will be performed sequentially
  - A single task which requires a lot of processing can make the entire application in waiting.
- In a **multithreaded application**, each task can be performed by a separate thread
  - If one thread is executing for long, it does not make the entire application wait for it to finish.
- If a multithreaded application is being executed on a system that has multiple processors, the OS may execute separate threads simultaneously on separate processors.

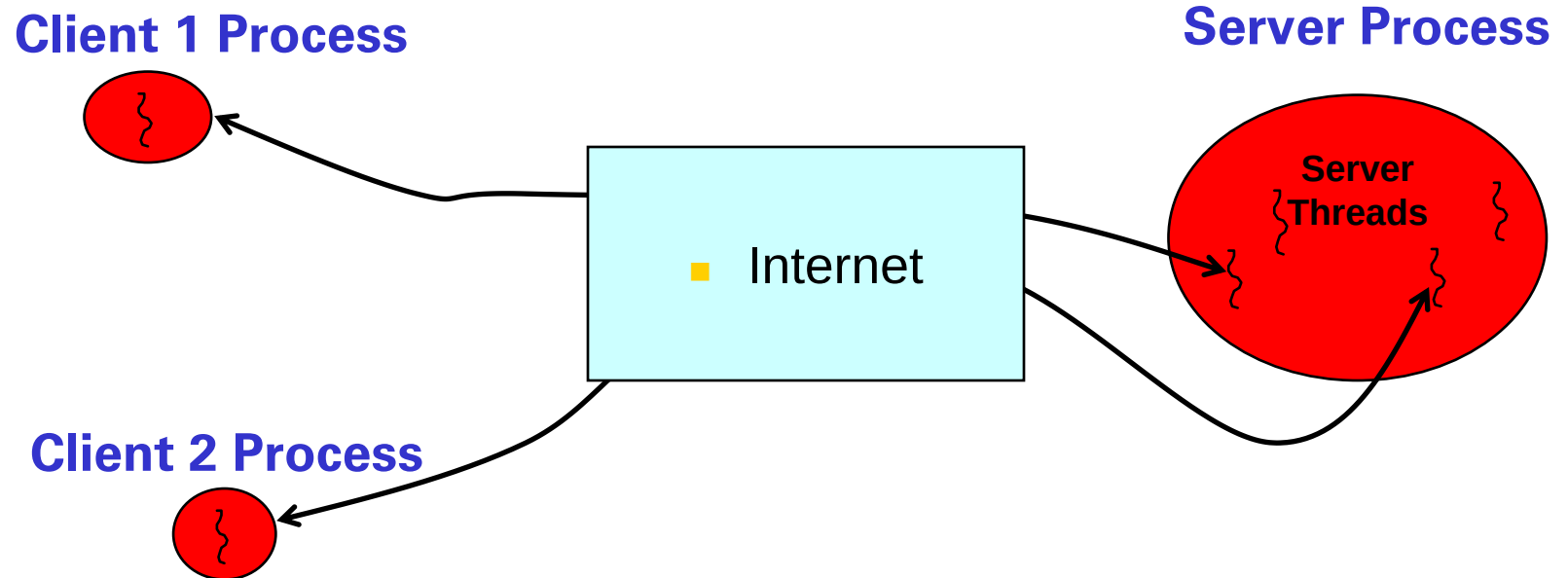
# Single and Multithreaded Processes

threads are light-weight processes within a process

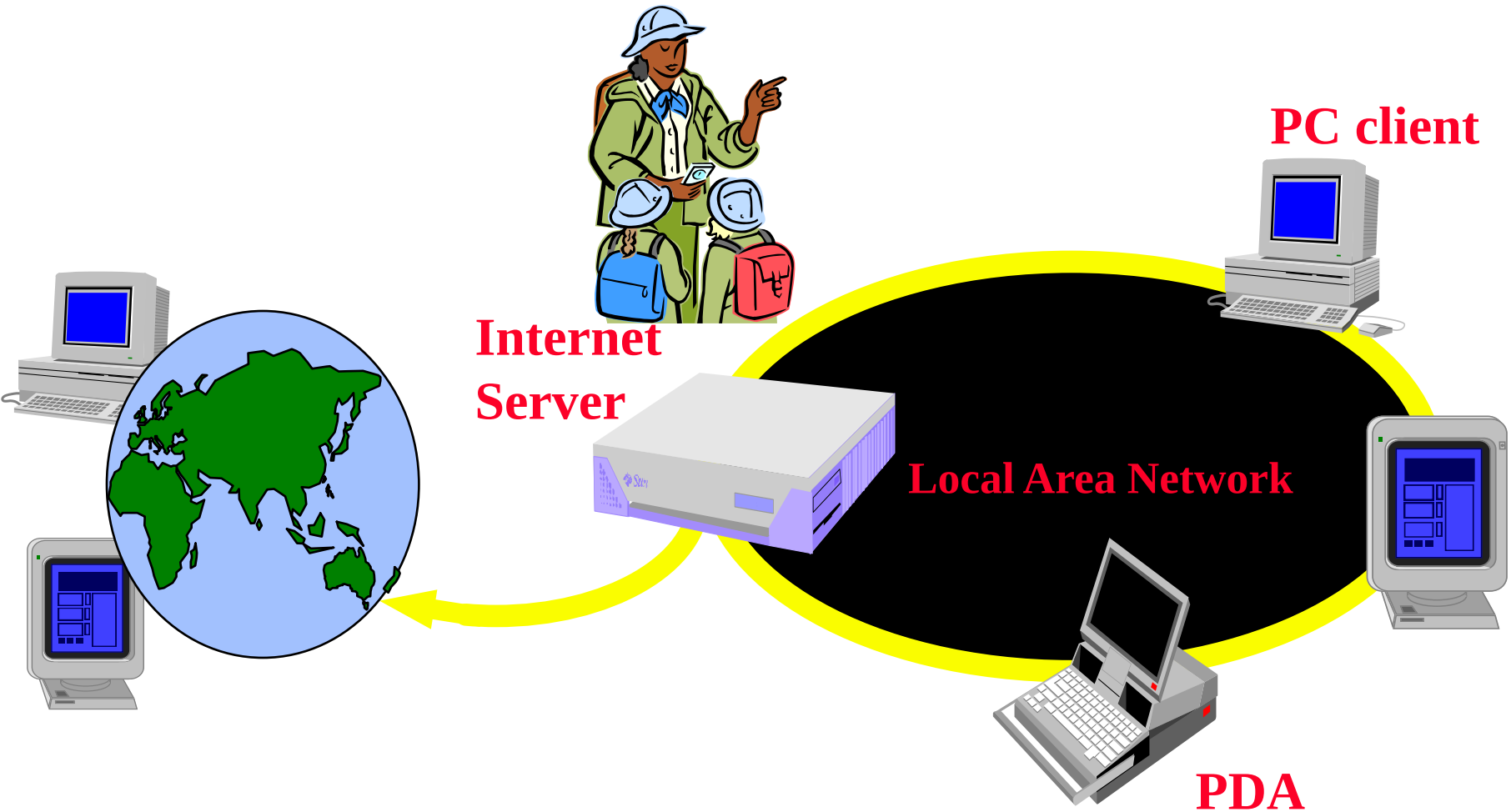




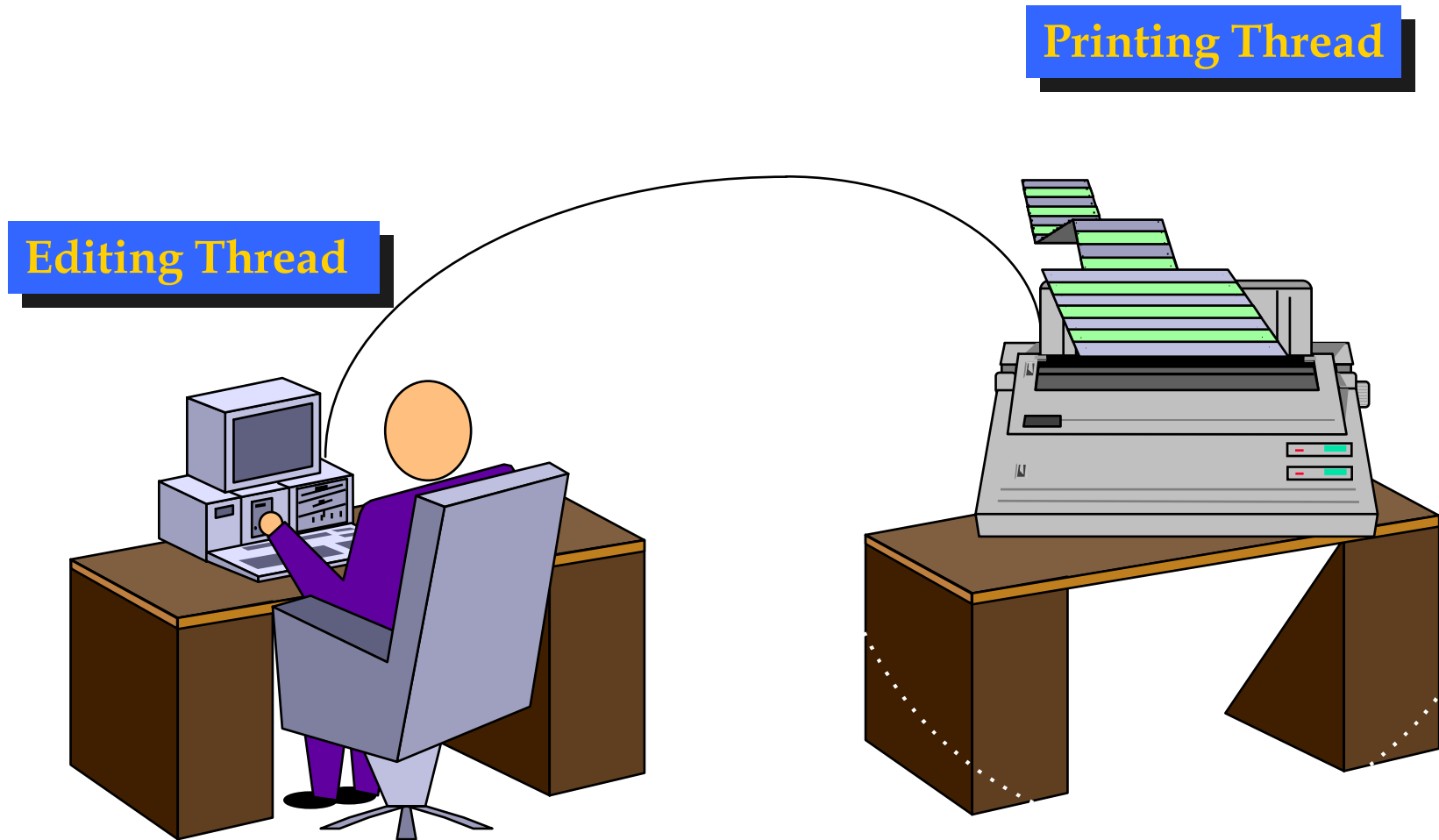
# Multithreaded Server: For Serving Multiple Clients Concurrently



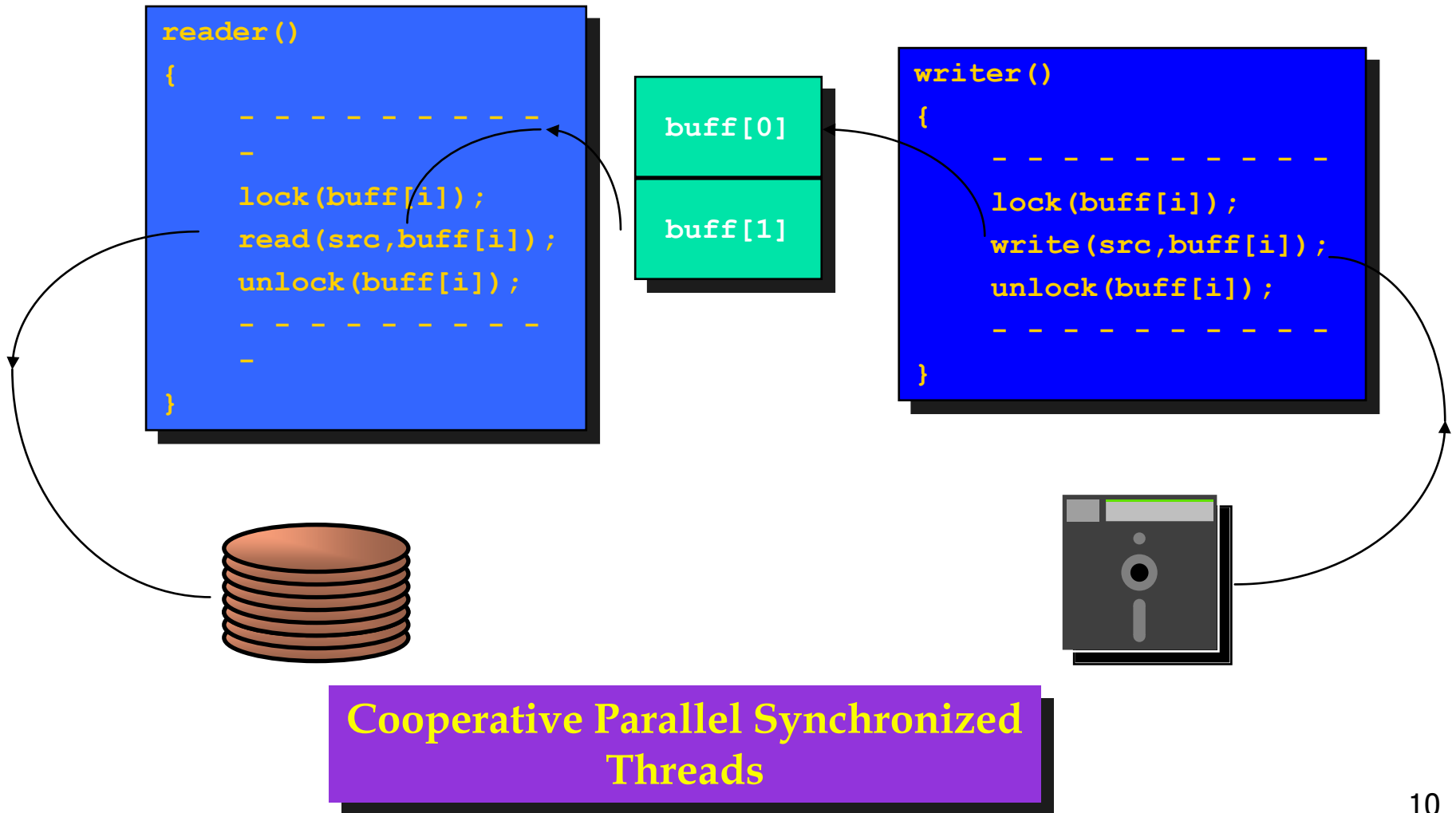
# Web/Internet Applications: Serving Many Users Simultaneously



# Modern Applications need Threads (ex1): Editing and Printing documents in background.



# Multithreaded/Parallel File Copy



# Levels of Parallelism

Sockets/

Task i-1

Task i

Task i+1

Code-Granularity

Code Item

Large grain  
(task level)

Program

Threads

```
func1 ()
{

}
```

```
func2 ()
{

}
```

```
func3 ()
{

}
```

Medium grain  
(control level)

Function (thread)

Compilers

```
a (0) =..
b (0) =..
```

```
a (1) =..
b (1) =..
```

```
a (2) =..
b (2) =..
```

Fine grain  
(data level)

Loop (Compiler)

CPU

+

X

Load

Very fine grain  
(multiple issue)

With hardware

# What are Threads?

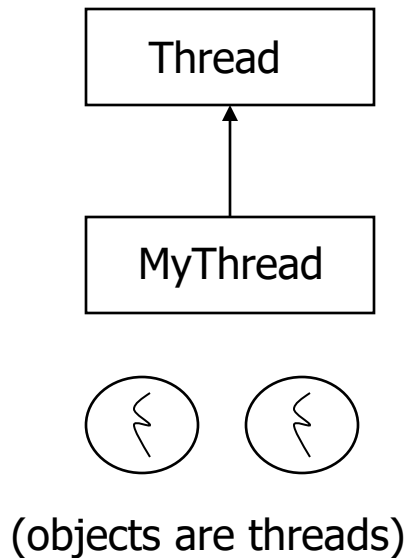
- A piece of code that run in concurrent with other threads
- Each thread is a ordered sequence of instructions
- Threads are being extensively used to express concurrency on both single and multiprocessors machines
- Programming a task having multiple threads of control called – Multithreading or Multithreaded Programming

# Java Threads

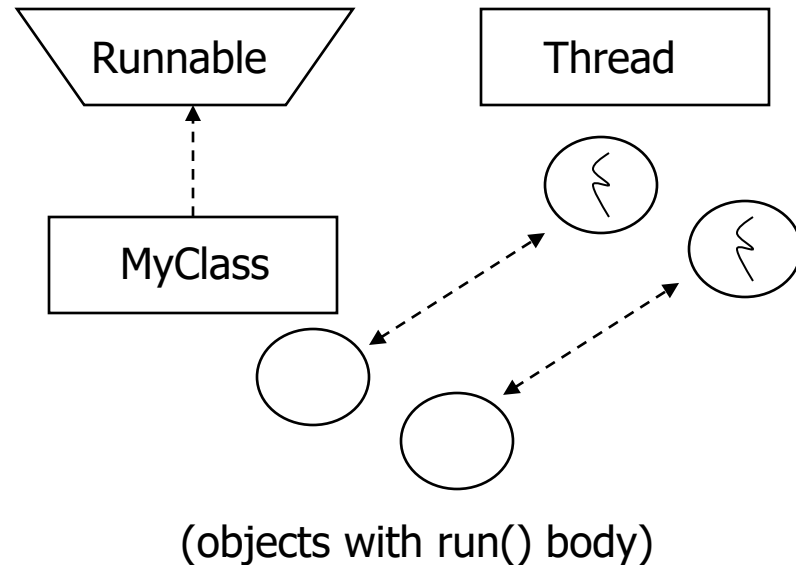
- Java has built in thread support for Multithreading
- Synchronization
- Thread Scheduling
- Inter-Thread Communication:
  - `currentThread`                      `start`                      `setPriority`
  - `yield`                                      `run`                      `getPriority`
  - `sleep`                                      `stop`                      `suspend`
  - `resume`
- Java Garbage Collector is a low-priority thread

# Threading Mechanisms...

- Create a class that extends the Thread class
- Create a class that implements the Runnable interface



[a]



[b]



# 1st method: Extending Thread class

- Create a class by extending Thread class and override run() method:

```
class MyThread extends Thread
{
 public void run()
 {
 // thread body of execution
 }
}
```

- Create a thread:

```
MyThread thr1 = new MyThread();
```

- Execute a thread (s):

```
thr1.start();
```

- Create and Execute:

```
new MyThread().start();
```

# An example

```
class MyThread extends Thread {
 public void run() {
 System.out.println(" this thread is running ... ");
 }
}
```

```
class ThreadEx1 {
 public static void main(String [] args) {
 MyThread t = new MyThread();
 t.start();
 }
}
```

# 2nd method: Threads by implementing Runnable interface

- Create a class that implements the interface Runnable and override run() method:

```
class MyThread implements Runnable
{

 public void run()
 {
 // thread body of execution
 }
}
```

- Creating Object:

```
MyThread myObject = new MyThread();
```

- Creating Thread Object:

```
Thread thr1 = new Thread(myObject);
```

- Start Execution:

```
thr1.start();
```

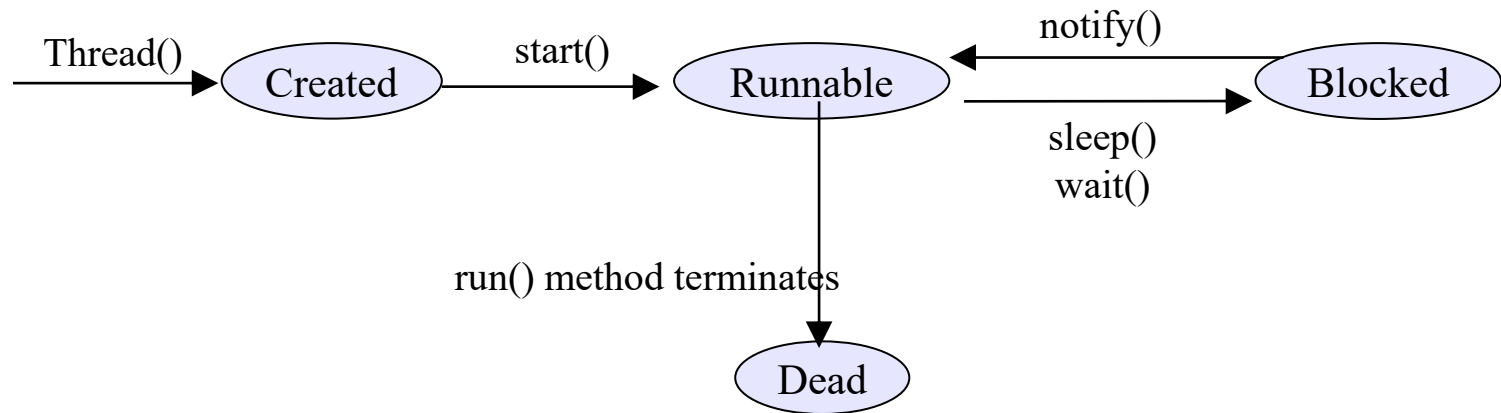
# An example

```
class MyThread implements Runnable {
 public void run() {
 System.out.println(" this thread is running ... ");
 }
}
```

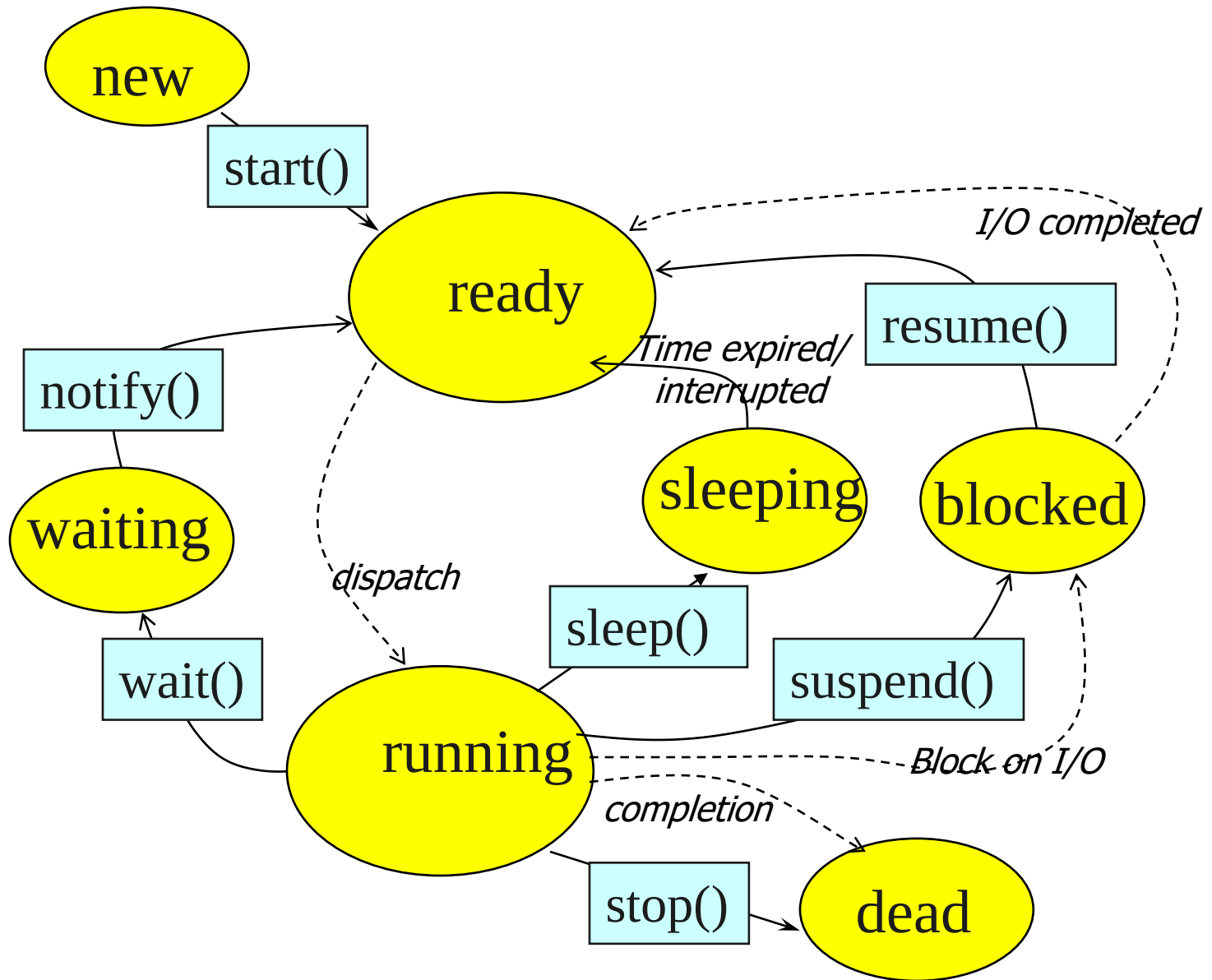
```
class ThreadEx2 {
 public static void main(String [] args) {
 Thread t = new Thread(new MyThread());
 t.start();
 }
}
```

# Thread States

- Threads can be in one of four states
  - Created, Running, Blocked, and Dead
- A thread's state changes based on:
  - Control methods such as start, sleep, yield, wait, notify
  - Termination of the run method



# Life Cycle of Thread



# A Program with Three Java Threads

- Write a program that creates 3 threads

# Three threads example

```
■ class A extends Thread
■ {
■ public void run()
■ {
■ for(int i=1;i<=5;i++)
■ {
■ System.out.println("\t From ThreadA: i= "+i);
■ }
■ System.out.println("Exit from A");
■ }
■ }

■ class B extends Thread
■ {
■ public void run()
■ {
■ for(int j=1;j<=5;j++)
■ {
■ System.out.println("\t From ThreadB: j= "+j);
■ }
■ System.out.println("Exit from B");
■ }
■ }
```



```

■ class C extends Thread
■ {
■ public void run()
■ {
■ for(int k=1;k<=5;k++)
■ {
■ System.out.println("\t From ThreadC: k= "+k);
■ }
■
■ System.out.println("Exit from C");
■ } }

■ class ThreadTest
■ {
■ public static void main(String args[])
■ {
■ new A().start();
■ new B().start();
■ new C().start();
■ }
■ }

```

# Run 1

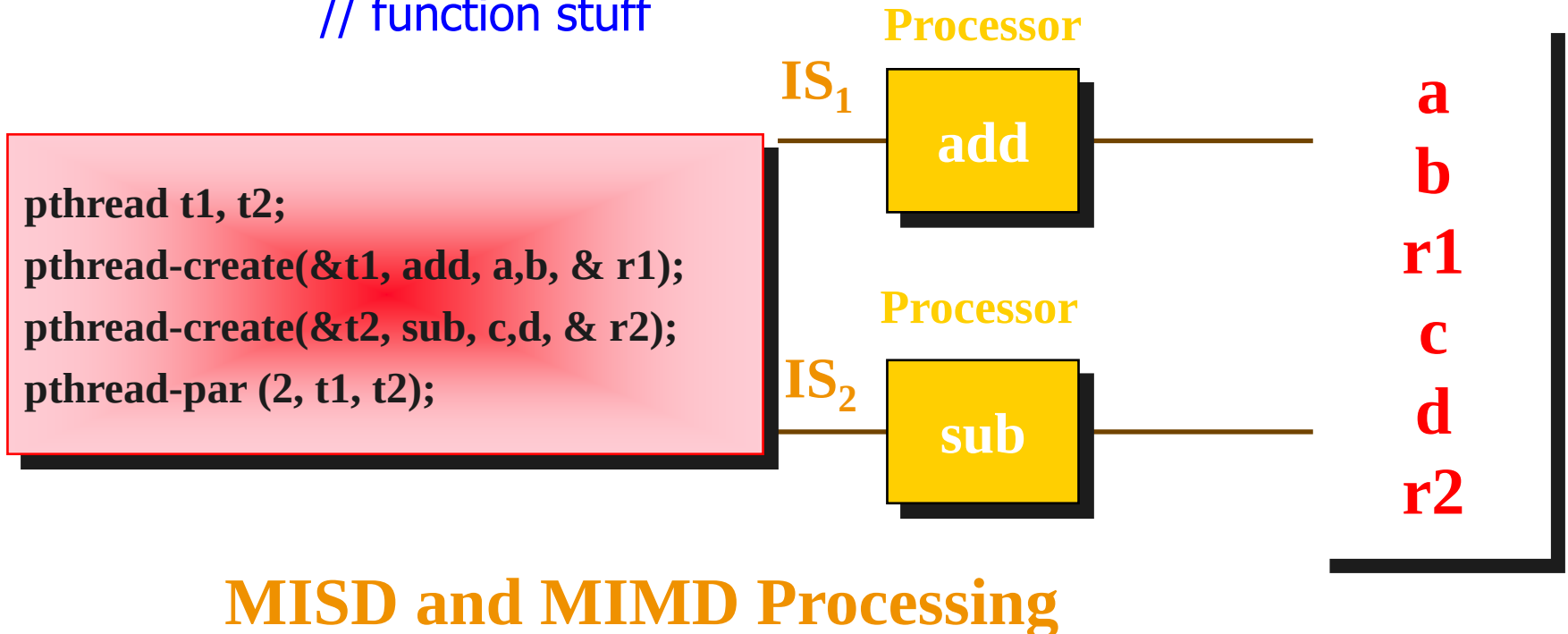
- java ThreadTest
  - From ThreadA: i= 1
  - From ThreadA: i= 2
  - From ThreadA: i= 3
  - From ThreadA: i= 4
  - From ThreadA: i= 5
  - Exit from A
    - From ThreadC: k= 1
    - From ThreadC: k= 2
    - From ThreadC: k= 3
    - From ThreadC: k= 4
    - From ThreadC: k= 5
  - Exit from C
    - From ThreadB: j= 1
    - From ThreadB: j= 2
    - From ThreadB: j= 3
    - From ThreadB: j= 4
    - From ThreadB: j= 5
  - Exit from B

# Run2

```
■ java ThreadTest
 From ThreadA: i= 1
 From ThreadA: i= 2
 From ThreadA: i= 3
 From ThreadA: i= 4
 From ThreadA: i= 5
 From ThreadC: k= 1
 From ThreadC: k= 2
 From ThreadC: k= 3
 From ThreadC: k= 4
 From ThreadC: k= 5
Exit from C
 From ThreadB: j= 1
 From ThreadB: j= 2
 From ThreadB: j= 3
 From ThreadB: j= 4
 From ThreadB: j= 5
Exit from B
Exit from A
```

# Process Parallelism

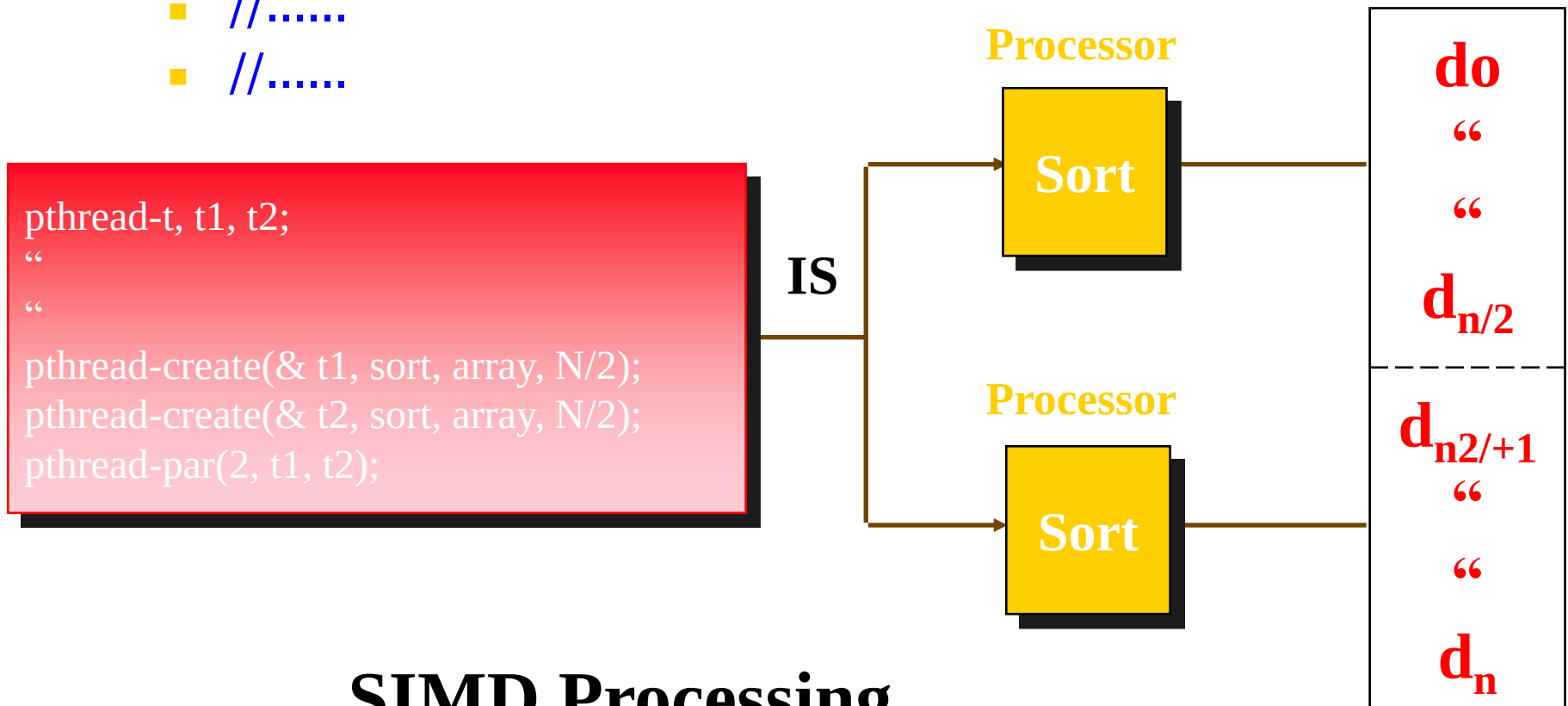
- `int add (int a, int b, int & result)`  
    `// function stuff`
- `int sub(int a, int b, int & result)`  
    `// function stuff`



# Data Parallelism

- `sort( int *array, int count)`
- `//.....`
- `//.....`

**Data**



**SIMD Processing**

# Thread Priority

- In Java, each thread is assigned priority, which affects the order in which it is scheduled for running. The threads so far had same default priority (NORM\_PRIORITY) and they are served using FCFS policy.
  - Java allows users to change priority:
    - ThreadName.setPriority(intNumber)
      - MIN\_PRIORITY = 1
      - NORM\_PRIORITY=5
      - MAX\_PRIORITY=10

# Thread Priority Example

```
class A extends Thread
{
 public void run()
 {
 System.out.println("Thread A started");
 for(int i=1;i<=4;i++)
 {
 System.out.println("\t From ThreadA: i= "+i);
 }
 System.out.println("Exit from A");
 }
}

class B extends Thread
{
 public void run()
 {
 System.out.println("Thread B started");
 for(int j=1;j<=4;j++)
 {
 System.out.println("\t From ThreadB: j= "+j);
 }
 System.out.println("Exit from B");
 }
}
```

# Thread Priority Example

```
class C extends Thread
{
 public void run()
 {
 System.out.println("Thread C started");
 for(int k=1;k<=4;k++)
 {
 System.out.println("\t From ThreadC: k= "+k);
 }
 System.out.println("Exit from C");
 }
}

class ThreadPriority
{
 public static void main(String args[])
 {
 A threadA=new A();
 B threadB=new B();
 C threadC=new C();
 threadC.setPriority(Thread.MAX_PRIORITY);
 threadB.setPriority(threadA.getPriority()+1);
 threadA.setPriority(Thread.MIN_PRIORITY);
 System.out.println("Started Thread A");
 threadA.start();
 System.out.println("Started Thread B");
 threadB.start();
 System.out.println("Started Thread C");
 threadC.start();
 System.out.println("End of main thread");
 }
}
```

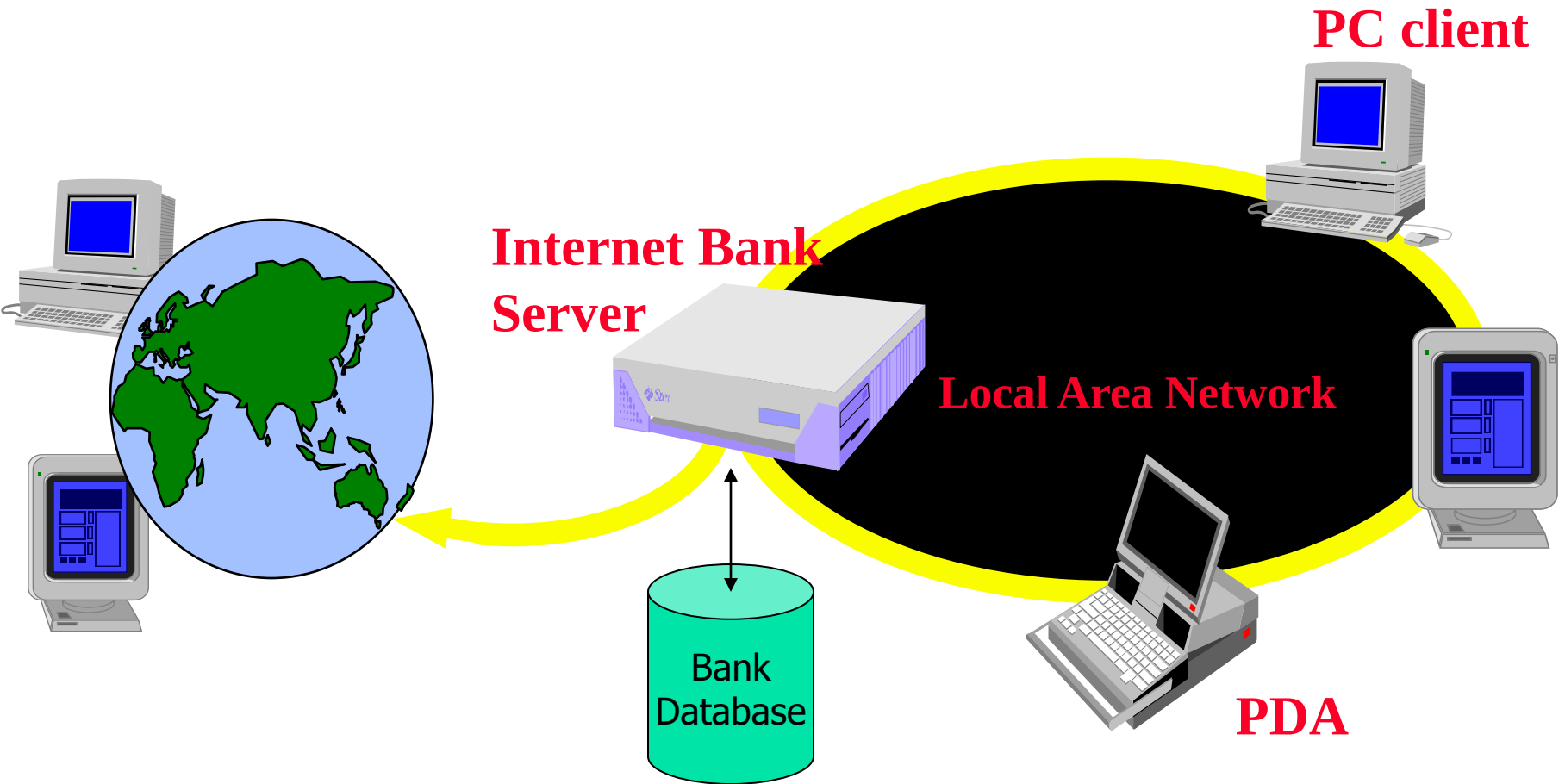
```
C:\java>java ThreadPriority
Started Thread A
Started Thread B
Thread A started
Thread B started
Started Thread C
 From ThreadB: j= 1
 From ThreadA: i= 1
Thread C started
 From ThreadB: j= 2
End of main thread
 From ThreadB: j= 3
 From ThreadC: k= 1
 From ThreadA: i= 2
 From ThreadC: k= 2
 From ThreadC: k= 3
 From ThreadB: j= 4
 From ThreadC: k= 4
 From ThreadA: i= 3
Exit from C
Exit from B
 From ThreadA: i= 4
Exit from A
```



# Accessing Shared Resources

- Applications Access to Shared Resources need to be coordinated.
  - Ex1: Printer (two person jobs cannot be printed at the same time)
  - Ex2: Simultaneous operations on your bank account.
  - Can the following operations be done at the same time on the same account?
    - Deposit()
    - Withdraw()
    - Enquire()

# Online Bank: Serving Many Customers and Operations



# Concurrent Access to Data

- Concurrent access to data can lead to data integrity problems
  - If two sources attempt to update the same data at the same time, the result of the data can be undefined.
  - The outcome is determined by how the scheduler schedules the two sources.
    - Since the schedulers activities cannot be predicted, the outcome cannot be predicted
- Databases deal with this mechanism through "**locking**"
  - If a source is going to update a table or record, it can lock the table or record until the time that the data has been successfully updated.
  - While locked, all access are blocked except to the source which holds the lock.
- Java has the equivalent mechanism. It is called **synchronization**
  - Java has a keyword called **synchronized**

# Shared Resources

- If one thread tries to read the data and other thread tries to update the same data, it leads to inconsistent state
- This can be prevented by synchronising access to the data
- Use “Synchronized” method:

```
public synchronized void update()
{
 ...
}
```

# 3 Threads sharing the same object

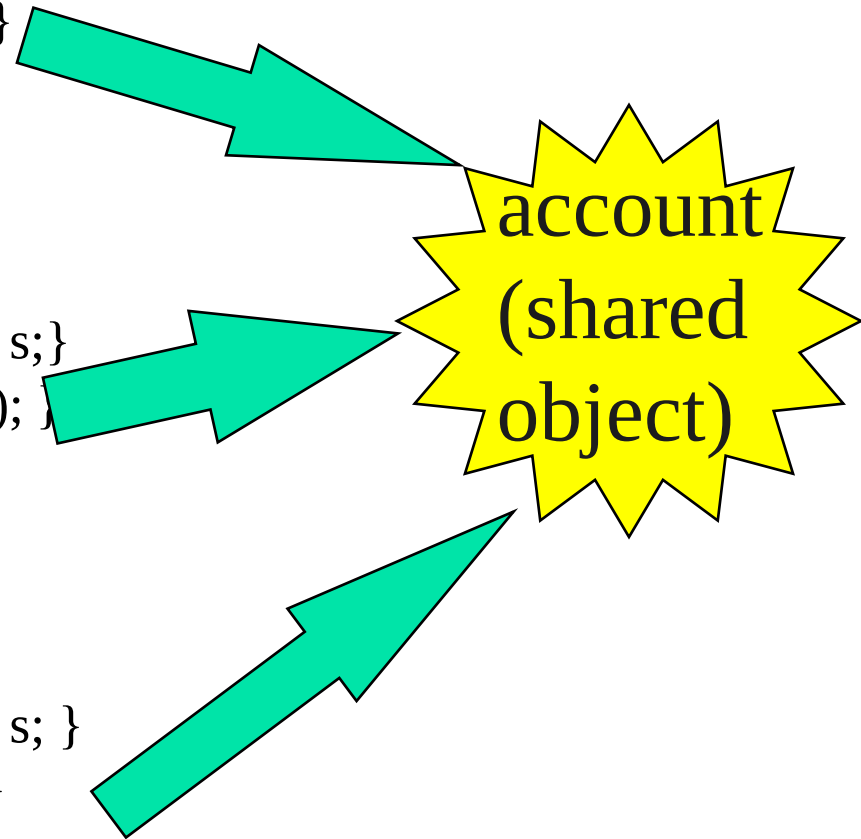
```
class InternetBankingSystem
{
 public static void main(String [] args)
 {
 Account accountObject = new Account ();
 Thread t1 = new Thread(new Thread1(accountObject));
 Thread t2 = new Thread(new Thread2(accountObject));
 Thread t3 = new Thread(new Thread3(accountObject));
 t1.start();
 t2.start();
 t3.start();
 // DO some other operation
 } // end main()
}
```

# Shared account object between 3 threads

```
class Thread1 implements Runnable {
 Account account;
 public Thread1 (Account s) { account = s;}
 public void run() { account.deposit(); }
} // end class MyThread
```

```
class Thread2 implements Runnable {
 Account account;
 public Thread2 (Account s) { account = s;}
 public void run() { account.withdraw(); }
} // end class YourThread
```

```
class Thread3 implements Runnable {
 Account account;
 public Thread3 (Account s) { account = s; }
 public void run() { account.enquire(); }
} // end class HerThread
```



# Monitor (shared object access): serializes operation on shared object

```
class Account { // the 'monitor'
 int balance;

 // if 'synchronized' is removed, the outcome is unpredictable
 public synchronized void deposit() { // METHOD BODY :
 balance = balance + deposit_amount;
 }

 public synchronized void withdraw() { // METHOD BODY:
 balance = balance - deposit_amount;
 }

 public synchronized void enquire() { // METHOD BODY:
 display balance.
 }
}
```

# Properly Terminating Threads

- In Java 1.1, the Thread class had a stop() method
  - One thread could terminate another by invoking its stop() method.
  - However, using stop() could lead to deadlocks
  - The stop() method is now not used. **DO NOT use the stop method to terminate a thread**
- The correct way to stop a thread is to have the run method terminate
  - Add a boolean variable which indicates whether the thread should continue or not
  - Provide a set method for that variable which can be invoked by another thread



# Terminating Thread Example

```
public class Test implements Runnable
{
 private Thread theThread;
 private boolean stopThread = false;

 public void start()
 {
 if (theThread == null)
 {
 theThread = new Thread(this);
 theThread.start();
 }
 }

 public void setStopThread(boolean aValue)
 {
 stopThread = aValue;
 }

 public void run()
 {
 while(true)
 {
 if (stopThread)
 break;

 //
 }
 }
}
```

## Yield() and Sleep()

- Sometimes a thread can determine that it has nothing to do
  - Sometimes the system can determine this, i.e., waiting for I/O
- **When a thread has nothing to do, it should not use CPU**
  - This is called a **busy-wait**, i.e., busy using CPU doing nothing
    - threads in busy-wait are continually checking if there is anything to do.
- It is worthwhile to run a CPU monitor program on your desktop
  - You can see that a thread is in busy-wait when the CPU monitor goes up (usually to 100%), but the application doesn't seem to be doing anything.
- Threads in busy-wait should be moved from the Run queue to the Wait queue so that they do not block the CPU
  - Use **yield()** or **sleep(time)**
  - **yield** simply tells the scheduler to schedule another thread
  - **sleep** guarantees that this thread will remain in the wait queue for the specified number of **milliseconds**.

# Java I/O Streams

# Java I/O streams

- Java programs perform I/O through streams
- A stream is an abstraction that either produces or consumes information.
- A stream is linked to a physical device by the Java I/O system.
- All streams behave in the same manner, even if the actual physical (linked) devices differ.
- Thus, the same I/O classes and methods can be applied to different types of devices.

# Java I/O streams

- An input stream can abstract many different kinds of input: from a disk file, a keyboard, or network socket
- Likewise, an output stream may refer to the console, a disk file, or network connection.
- Java implements streams within the class hierarchies defined in the `java.io` package

# Java I/O streams

- 3 Default streams in Java
  - Standard Input stream `System.in`
  - Standard output stream `System.out`
  - Standard error stream `System.err`
- Can a user define a stream?

# Byte stream & Character Stream

- Java defines two types of stream: Byte & Character
- Byte stream provide a convenient means for handling input and output of bytes. For example, when reading or writing binary data. (Any type of data, 1 byte at a time )
- Character stream provides a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. (Only text data, 2 bytes at a time)
- In some cases, character streams are more efficient than byte stream.

# Byte stream

- Byte stream are defined by using two class hierarchies.
- At the top are two abstract classes: **InputStream** and **OutputStream**
- Each of these abstract classes has several subclasses that handle the differences among various devices (such as disk files, network connection and even memory buffers.)



# Byte stream

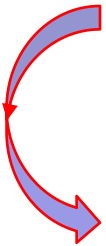
| Stream Class          | Meaning                                                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| BufferedInputStream   | Buffered input stream                                                                                               |
| BufferedOutputStream  | Buffered output stream                                                                                              |
| ByteArrayInputStream  | Input stream that reads from a byte array                                                                           |
| ByteArrayOutputStream | Output stream that writes to a byte array                                                                           |
| DataInputStream       | An input stream that contains methods for reading the Java standard data types                                      |
| DataOutputStream      | An output stream that contains methods for writing the Java standard data types                                     |
| FileInputStream       | Input stream that reads from a file                                                                                 |
| FileOutputStream      | Output stream that writes to a file                                                                                 |
| FilterInputStream     | Implements <b>InputStream</b>                                                                                       |
| FilterOutputStream    | Implements <b>OutputStream</b>                                                                                      |
| InputStream           | Abstract class that describes stream input                                                                          |
| ObjectInputStream     | Input stream for objects                                                                                            |
| ObjectOutputStream    | Output stream for objects                                                                                           |
| OutputStream          | Abstract class that describes stream output                                                                         |
| PipedInputStream      | Input pipe                                                                                                          |
| PipedOutputStream     | Output pipe                                                                                                         |
| PrintStream           | Output stream that contains <b>print( )</b> and <b>println( )</b>                                                   |
| PushbackInputStream   | Input stream that allows bytes to be returned to the input stream.                                                  |
| SequenceInputStream   | Input stream that is a combination of two or more input streams that will be read sequentially, one after the other |

**Table 13-1** The Byte Stream Classes in **java.io**

# Character stream

- Character streams are defined by using two class hierarchies.
- At the top are two abstract classes: **Reader** & **Writer**
- These abstract classes handle **Unicode (16 bit)** character streams.

# Character stream



| Stream Class       | Meaning                                                                |
|--------------------|------------------------------------------------------------------------|
| BufferedReader     | Buffered input character stream                                        |
| BufferedWriter     | Buffered output character stream                                       |
| CharArrayReader    | Input stream that reads from a character array                         |
| CharArrayWriter    | Output stream that writes to a character array                         |
| FileReader         | Input stream that reads from a file                                    |
| FileWriter         | Output stream that writes to a file                                    |
| FilterReader       | Filtered reader                                                        |
| FilterWriter       | Filtered writer                                                        |
| InputStreamReader  | Input stream that translates bytes to characters                       |
| LineNumberReader   | Input stream that counts lines                                         |
| OutputStreamWriter | Output stream that translates characters to bytes                      |
| PipedReader        | Input pipe                                                             |
| PipedWriter        | Output pipe                                                            |
| PrintWriter        | Output stream that contains <b>print( )</b> and <b>println( )</b>      |
| PushbackReader     | Input stream that allows characters to be returned to the input stream |
| Reader             | Abstract class that describes character stream input                   |
| StringReader       | Input stream that reads from a string                                  |
| StringWriter       | Output stream that writes to a string                                  |
| Writer             | Abstract class that describes character stream output                  |

Abstract class, data manipulation before reading/writing data

Buffered reader, how many lines are read

**Table 13-2** The Character Stream I/O Classes in **java.io**

# Predefined streams

- All JAVA programs automatically import the java.lang package. This package defines a class called System, which encapsulates several aspects of the run-time environment.
- System also contains three predefined stream variables: **in, out and err.**
- These fields are declared as **public, static and final** within System
- System.out refers to the standard output stream. By default, this is the console.
- System.in refers to standard input stream, which is the keyboard by default.
- System.err refers to the standard error stream, which also is the console by default.

# Reading Console Input (Character Str..)

- In Java, console input is accomplished by reading from `System.in`
- To obtain a character based stream attached to the console, wrap `System.in` in a `BufferedReader` object.

```
BufferedReader br = new BufferedReader(new
 InputStreamReader(System.in));
```

- After this statement executes, `br` is a character based stream that is linked to the console.

# Reading Characters

- To read a character from a `BufferedReader`, use `read()`.

`int read() throws IOException`

- Each time the `read()` is called, it reads a character from the input stream and returns it as an integer value.
- It returns `-1` when end of the stream is encountered.

# Reading Characters

```
import java.io.*;

class BRRead {
 public static void main(String args[]) throws IOException
 {
 char c;
 BufferedReader br = new
 BufferedReader(new InputStreamReader(System.in));
 System.out.println("Enter characters, 'q' to quit.");
 // read characters
 do {
 c = (char) br.read();
 System.out.println(c);
 } while(c != 'q');
 }
}
```

Enter characters, 'q' to quit.  
123abcq  
1  
2  
3  
a  
b  
c  
q

# Reading Strings

- To read a string from the keyboard, use the version of `readLine()` that is a member of the `BufferedReader` class.

```
String readLine() throws IOException
```

- It returns string object.



# Reading Strings

```
import java.io.*;

class BRReadLines {
 public static void main(String args[]) throws
IOException
 {
 // create a BufferedReader using System.in
 BufferedReader br = new BufferedReader(new
 InputStreamReader(System.in));

 String str;
 System.out.println("Enter lines of text.");
 System.out.println("Enter 'stop' to quit.");
 do {
 str = br.readLine();
 System.out.println(str);
 } while(!str.equals("stop"));
 }
}
```

# Reading Strings

```
// A tiny editor.
```

```
import java.io.*;
```

```
class TinyEdit {
```

```
 public static void main(String args[]) throws IOException
 {
```

```
 // create a BufferedReader using System.in
```

```
 BufferedReader br = new BufferedReader(new
 InputStreamReader(System.in));
```

```
 String str[] = new String[100];
```

```
 System.out.println("Enter lines of text.");
```

```
 System.out.println("Enter 'stop' to quit.");
```

```
 for(int i=0; i<100; i++) { // Read and Store the lines
```

```
 str[i] = br.readLine();
```

```
 if(str[i].equals("stop")) break;
```

```
 }
```

```
 System.out.println("\nHere is your file:");
```

```
 // display the lines
```

```
 for(int i=0; i<100; i++) {
```

```
 if(str[i].equals("stop")) break;
```

```
 System.out.println(str[i]);
```

```
 }
```

```
 }
```

```
}
```

# File Handling (byte stream)

- Two of the most often-used stream classes are: FileInputStream and FileOutputStream, which create byte streams linked to files.

`FileInputStream(String fileName)` throws `FileNotFoundException`

`FileOutputStream(String fileName)` throws `FileNotFoundException`

- `FileNotFoundException` is a subclass of `IOException`

# File Handling

- To close a file, use `close()` function, which is implemented by both `FileInputStream` and `FileOutputStream`

`void close() throws IOException`

- Closing a file releases the system resources allocated to the file, allowing them to be used by another file.

# File Handling - Read

- To read from a file, use `read()` function defined in `FileInputStream`.

`int read() throws IOException`

- Each time it is called, it reads a single byte from the file and returns the byte as integer value.
- `read()` returns -1 when the end of the file is encountered.

# File Handling

```
import java.io.*;

class ShowFile {
 public static void main(String args[])
 {
 int i;
 FileInputStream fin;

 // First, confirm that a file name has been specified.
 if(args.length != 1) {
 System.out.println("File not found");
 return;
 }

 // Attempt to open the file.
 try {
 fin = new FileInputStream(args[0]);
 } catch(FileNotFoundException e) {
 System.out.println("Cannot Open File");
 return;
 }
 }
}
```

# File Handling

```
// At this point, the file is open and can be read.
// The following reads characters until EOF is encountered.
 try {
 do {
 i = fin.read();
 if(i != -1) System.out.print((char) i);
 } while(i != -1);
 } catch(IOException e) {
 System.out.println("Error Reading File");
 }

 // Close the file.
 try {
 fin.close();
 } catch(IOException e) {
 System.out.println("Error Closing File");
 }
}
```

```
import java.io.*;
class ShowFile {
 public static void main(String args[])
 { int i;
 FileInputStream fin = null;
 if(args.length != 1) {
 System.out.println("File not found");
 return; }
 try {
 fin = new FileInputStream(args[0]);
 do {
 i = fin.read();
 if(i != -1) System.out.print((char) i);
 } while(i != -1);
 } catch(FileNotFoundException e) {
 System.out.println("File Not Found.");
 } catch(IOException e) {
 System.out.println("An I/O Error Occurred");
 } finally {
 try {
 if(fin != null) fin.close(); }
 catch(IOException e) {
 System.out.println("Error Closing File"); }
 }
 }
}
```



# File Handling - Write

- To write to a file, use `write()` function defined by `FileOutputStream`.

```
void write(int byteval) throws IOException
```

- This method writes the byte specified by `byteval` to the file. Although, `byteval` is declared as an integer, only the lower eight bits are written to the file.
- If an error occurs during writing, an `IOException` is thrown.

```

import java.io.*;
class CopyFile {
 public static void main(String args[]) throws IOException
 {
 int i;
 FileInputStream fin = null;
 FileOutputStream fout = null;
 // First, confirm that both files has been specified.
 if(args.length != 2) {
 System.out.println("Usage: CopyFile from to");
 return;
 }
 finally {
 // Co
 try {
 if(fin != null) fin.close();
 } catch(IOException e2) {
 System.out.println("Error Closing Input File");
 }
 do
 try {
 if(fout != null) fout.close();
 } catch(IOException e2) {
 System.out.println("Error Closing Output File");
 }
 }
 }
}

```

# File Handling

- JDK 7 added a new feature that offers another way to manage resources, such as file stream, by automating the closing process.

```
try(resource-specification) {
 // use the resource
}
```

- Here, resource-specification is a statement that declares and initializes a resource, such as a file stream
- When the try block ends, the resource is automatically released.

```
import java.io.*;
class ShowFile {
 public static void main(String args[])
 {
 int i;
 // First, confirm that a file name has been specified.
 if(args.length != 1) {
 System.out.println("Usage: ShowFile filename");
 return;
 }
 // The following code uses a try-with-resources statement to open a
 // file and then automatically close it when the try block ends.
 try(FileInputStream fin = new FileInputStream(args[0])) {
 do {
 i = fin.read();
 if(i != -1) System.out.print((char) i);
 } while(i != -1);
 } catch(FileNotFoundException e) {
 System.out.println("File Not Found.");
 } catch(IOException e) {
 System.out.println("An I/O Error Occurred");
 }
 }
}
```

# File Handling

- The scope of the resource is limited to the try-with-resource statement.
- More than one resources can be managed by semicolon in a single try statement.

```
try (FileInputStream fin = new FileInputStream(args[0]);
 FileOutputStream fout = new FileOutputStream(args[1]))
```

# File Class

- The File class does not specify how information is retrieved from or stored in files; it describes the properties of a file itself.
- A File object is used to obtain or manipulate the information associated with a disk file, such as the permissions, time, date, and directory path, and to navigate subdirectory hierarchies.
- The following constructors can be used to create File objects:  
File(String directoryPath)  
File(String directoryPath, String filename)  
File(File dirObj, String filename)

```
File f1 = new File("/");
File f2 = new File("/", "autoexec.bat");
File f3 = new File(f1, "autoexec.bat");
```

# File Class

- File defines many methods that obtain the standard properties of a File object

```
import java.io.File;
class FileDemo {
 static void p(String s) {
 System.out.println(s);
 }
 public static void main(String args[]) {
 File f1 = new File("/java/COPYRIGHT");

 p("File Name: " + f1.getName());
 p("Path: " + f1.getPath());
 p("Abs Path: " + f1.getAbsolutePath());
 p("Parent: " + f1.getParent());
 p(f1.exists() ? "exists" : "does not exist");
 p(f1.canWrite() ? "is writeable" : "is not writeable");
 p(f1.canRead() ? "is readable" : "is not readable");
 p("is " + (f1.isDirectory() ? "" : "not" + " a directory"));
 p(f1.isFile() ? "is normal file" : "might be a named pipe");
 p(f1.isAbsolute() ? "is absolute" : "is not absolute");
 p("File last modified: " + f1.lastModified());
 p("File size: " + f1.length() + " Bytes");
 }
}
```

# File Class

```
import java.io.File;
class FileDemo {
 static void p(String s) {
 System.out.println(s);
 }
 public static void main(String args[]) {
 File f1 = new File("/java/COPYRIGHT");

 p("File Name: " + f1.getName());
 p("Path: " + f1.getPath());
 p("Parent: " + f1.getParent());
 p(f1.exists() ? "exists" : "does not exist");
 p(f1.canWrite() ? "is writeable" : "is not writeable");
 p(f1.canRead() ? "is readable" : "is not readable");
 p("is " + (f1.isDirectory() ? "" : "not") + " a directory");
 p(f1.isFile() ? "is normal file" : "might be a named pipe");
 p(f1.isAbsolute() ? "is absolute" : "is not absolute");
 p("File last modified: " + f1.lastModified()); //milliseconds
 p("File size: " + f1.length() + " Bytes");
 }
}
```

```
File Name: COPYRIGHT
Path: /java/COPYRIGHT
Parent: /java
exists
is writeable
is readable
is not a directory
is normal file
is absolute
File last modified: 812465204000
File size: 695 Bytes
```



# File Class

- File also includes two useful utility methods. The first is `renameTo( )`, shown here:

```
boolean renameTo(File newName)
```

- Here, the filename specified by `newName` becomes the new name of the invoking File object. **It will return true upon success and false if the file cannot be renamed** (if you either attempt to rename a file so that it moves from one directory to another or use an existing filename, for example).
- The second utility method is `delete( )`, which deletes the disk file represented by the path of the invoking File object. It is shown here:

```
boolean delete()
```
- You can also use `delete( )` to delete a directory if the directory is empty. **`delete( )` returns true if it deletes the file and false if the file cannot be removed.**

# File Class

| Method                                                     | Description                                                                                                                                                                 |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void deleteOnExit( )</code>                          | Removes the file associated with the invoking object when the Java Virtual Machine terminates.                                                                              |
| <code>boolean isHidden( )</code>                           | Returns <b>true</b> if the invoking file is hidden. Returns <b>false</b> otherwise.                                                                                         |
| <code>boolean setLastModified(long <i>millisec</i>)</code> | Sets the time stamp on the invoking file to that specified by <i>millisec</i> , which is the number of milliseconds from January 1, 1970, Coordinated Universal Time (UTC). |
| <code>boolean setReadOnly( )</code>                        | Sets the invoking file to read-only.                                                                                                                                        |

# Directories

- A directory is a File that contains a list of other files and directories.
- When create a File object and it is a directory, the `isDirectory( )` method will return true.
- In this case, call `list( )` on that object to extract the list of other files and directories inside.

```
String[] list()
```

The list of files is returned in an array of String objects

# Directories

```
import java.io.File;

class DirList {
 public static void main(String args[]) {
 String dirname = "/java";
 File f1 = new File(dirname);

 if (f1.isDirectory()) {
 System.out.println("Directory of " + dirname);
 String s[] = f1.list();

 for (int i=0; i < s.length; i++) {
 File f = new File(dirname + "/" + s[i]);
 if (f.isDirectory()) {
 System.out.println(s[i] + " is a directory");
 } else {
 System.out.println(s[i] + " is a file");
 }
 }
 } else {
 System.out.println(dirname + " is not a directory");
 }
 }
}
```

# Directories

```
Directory of /java
bin is a directory
lib is a directory
demo is a directory
COPYRIGHT is a file
README is a file
index.html is a file
include is a directory
src.zip is a file
.hotjava is a directory
src is a directory
```

# Directories

- limit the number of files returned by the `list( )` method to include only those files that match a certain filename pattern, or filter. To do this, use a second form of `list( )`,

```
String[] list(FilenameFilter FFObj)
```

- In this form, `FFObj` is an object of a class that implements the `FilenameFilter` interface.
- `FilenameFilter` defines only a single method, `accept( )`, which is called once for each file in a list. Its general form is:

```
boolean accept(File directory, String filename)
```

- The `accept( )` method returns `true` for files in the directory specified by `directory` that should be included in the list (that is, those that match the `filename` argument), and returns `false` for those files that should be excluded.

# Directories

```
public class OnlyExt implements FilenameFilter {
 String ext;

 public OnlyExt(String ext) {
 this.ext = "." + ext;
 }

 public boolean accept(File dir, String name) {
 return name.endsWith(ext);
 }
}
```

# Directories

```
import java.io.*;
class DirListOnly {
 public static void main(String args[]) {
 String dirname = "/java";
 File f1 = new File(dirname);
 FilenameFilter only = new OnlyExt("html");
 String s[] = f1.list(only);
 for (int i=0; i < s.length; i++) {
 System.out.println(s[i]);
 }
 }
}
```



# Directories

- Another two useful File utility methods are `mkdir( )` and `makedirs( )`.
- The `mkdir( )` method creates a directory, returning **true on success and false on failure**. Failure indicates that the path specified in the File object already exists, or that the directory cannot be created because the entire path does not exist yet.
- To create a directory for which no path exists, use the `makedirs( )` method. It creates both a directory and all the parents of the directory.

# Input Stream

- InputStream is an abstract class that defines Java's model of streaming byte input.
- All of the methods in this class will throw an IOException on error conditions.

# Input Stream

| Method                                                                               | Description                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int available( )</code>                                                        | Returns the number of bytes of input currently available for reading.                                                                                                                                        |
| <code>void close( )</code>                                                           | Closes the input source. Further read attempts will generate an <b>IOException</b> .                                                                                                                         |
| <code>void mark(int <i>numBytes</i>)</code>                                          | Places a mark at the current point in the input stream that will remain valid until <i>numBytes</i> bytes are read.                                                                                          |
| <code>boolean markSupported( )</code>                                                | Returns <b>true</b> if <b>mark( )</b> / <b>reset( )</b> are supported by the invoking stream.                                                                                                                |
| <code>int read( )</code>                                                             | Returns an integer representation of the next available byte of input. -1 is returned when the end of the file is encountered.                                                                               |
| <code>int read(byte <i>buffer</i>[ ])</code>                                         | Attempts to read up to <i>buffer.length</i> bytes into <i>buffer</i> and returns the actual number of bytes that were successfully read. -1 is returned when the end of the file is encountered.             |
| <code>int read(byte <i>buffer</i>[ ], int <i>offset</i>, int <i>numBytes</i>)</code> | Attempts to read up to <i>numBytes</i> bytes into <i>buffer</i> starting at <i>buffer[offset]</i> , returning the number of bytes successfully read. -1 is returned when the end of the file is encountered. |

**Table 17-1.** The Methods Defined by InputStream

# Input Stream

| Method                                       | Description                                                                                              |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>void reset( )</code>                   | Resets the input pointer to the previously set mark.                                                     |
| <code>long skip(long <i>numBytes</i>)</code> | Ignores (that is, skips) <i>numBytes</i> bytes of input, returning the number of bytes actually ignored. |

**Table 17-1.** *The Methods Defined by InputStream* (continued)

# OutputStream

- OutputStream is an abstract class that defines streaming byte output.
- All of the methods in this class return a void value and throw an IOException in the case of errors

# Output Stream

| Method                                                                                                       | Description                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void close( )</code>                                                                                   | Closes the output stream. Further write attempts will generate an <b>IOException</b> .                                                                                                           |
| <code>void flush( )</code>                                                                                   | Finalizes the output state so that any buffers are cleared. That is, it flushes the output buffers.                                                                                              |
| <code>void write(int <i>b</i>)</code>                                                                        | Writes a single byte to an output stream. Note that the parameter is an <b>int</b> , which allows you to call <b>write( )</b> with expressions without having to cast them back to <b>byte</b> . |
| <code>void write(byte <i>buffer</i>[ ])</code>                                                               | Writes a complete array of bytes to an output stream.                                                                                                                                            |
| <code>void write(byte <i>buffer</i>[ ], int <i>offset</i>,<br/>                  int <i>numBytes</i>)</code> | Writes a subrange of <i>numBytes</i> bytes from the array <i>buffer</i> , beginning at <i>buffer</i> [ <i>offset</i> ].                                                                          |

**Table 17-2.**    *The Methods Defined by OutputStream*

# SequenceInputStream

- The `SequenceInputStream` class allows you to concatenate multiple `InputStream`s.
- The construction of a `SequenceInputStream` is different from any other `InputStream`.
- A `SequenceInputStream` constructor uses either a pair of `InputStream`s or an `Enumeration` of `InputStream`s as its argument:

```
SequenceInputStream(InputStream first,
InputStream second)
```

```
SequenceInputStream(Enumeration streamEnum)
```

```
import java.io.*;
import java.util.*;

class InputStreamEnumerator implements Enumeration<FileInputStream>
{
 private Enumeration<String> files;

 public InputStreamEnumerator(Vector<String> files) {
 this.files = files.elements();
 }

 public boolean hasMoreElements() {
 return files.hasMoreElements();
 }

 public FileInputStream nextElement() {
 try {
 return new FileInputStream(files.nextElement().toString());
 } catch (IOException e) {
 return null;
 }
 }
}
```



```
class SequenceInputStreamDemo {
 public static void main(String args[]) {
 int c;
 Vector<String> files = new Vector<String>();
 files.addElement("file1.txt");
 files.addElement("file2.txt");
 files.addElement("file3.txt");
 InputStreamEnumerator ise = new InputStreamEnumerator(files);
 InputStream input = new SequenceInputStream(ise);
 try {
 while ((c = input.read()) != -1)
 System.out.print((char) c);
 } catch (NullPointerException e) {
 System.out.println("Error Opening File.");
 } catch (IOException e) {
 System.out.println("I/O Error: " + e);
 } finally {
 try {
 input.close();
 } catch (IOException e) {
 System.out.println("Error Closing SequenceInputStream");
 }
 }
 }
}
```

# Reader Class

- Reader is an abstract class that defines Java's model of streaming character input.
- All of the methods in this class will throw an `IOException` on error conditions.

| Method                                                                                                                                    | Description                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>abstract void close( )</code>                                                                                                       | Closes the input source. Further read attempts will generate an <b>IOException</b> .                                                                                                                                   |
| <code>void mark(int <i>numChars</i>)</code>                                                                                               | Places a mark at the current point in the input stream that will remain valid until <i>numChars</i> characters are read.                                                                                               |
| <code>boolean markSupported( )</code>                                                                                                     | Returns <b>true</b> if <b>mark( )</b> / <b>reset( )</b> are supported on this stream.                                                                                                                                  |
| <code>int read( )</code>                                                                                                                  | Returns an integer representation of the next available character from the invoking input stream. -1 is returned when the end of the file is encountered.                                                              |
| <code>int read(char <i>buffer</i>[ ])</code>                                                                                              | Attempts to read up to <i>buffer.length</i> characters into <i>buffer</i> and returns the actual number of characters that were successfully read. -1 is returned when the end of the file is encountered.             |
| <code>abstract int read(char <i>buffer</i>[ ],<br/>                  int <i>offset</i>,<br/>                  int <i>numChars</i>)</code> | Attempts to read up to <i>numChars</i> characters into <i>buffer</i> starting at <i>buffer[offset]</i> , returning the number of characters successfully read. -1 is returned when the end of the file is encountered. |
| <code>boolean ready( )</code>                                                                                                             | Returns <b>true</b> if the next input request will not wait. Otherwise, it returns <b>false</b> .                                                                                                                      |
| <code>void reset( )</code>                                                                                                                | Resets the input pointer to the previously set mark.                                                                                                                                                                   |
| <code>long skip(long <i>numChars</i>)</code>                                                                                              | Skips over <i>numChars</i> characters of input, returning the number of characters actually skipped.                                                                                                                   |

**Table 17-3.**    *The Methods Defined by Reader*

# Writer Class

- Writer is an abstract class that defines streaming character output.
- All of the methods in this class return a void value and throw an IOException in the case of errors.

| Method                                                                                      | Description                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| abstract void close( )                                                                      | Closes the output stream. Further write attempts will generate an <b>IOException</b> .                                                                                                                       |
| abstract void flush( )                                                                      | Finalizes the output state so that any buffers are cleared. That is, it flushes the output buffers.                                                                                                          |
| void write(int <i>ch</i> )                                                                  | Writes a single character to the invoking output stream. Note that the parameter is an <b>int</b> , which allows you to call <b>write</b> with expressions without having to cast them back to <b>char</b> . |
| void write(char <i>buffer</i> [ ])                                                          | Writes a complete array of characters to the invoking output stream.                                                                                                                                         |
| abstract void write(char <i>buffer</i> [ ],<br>int <i>offset</i> ,<br>int <i>numChars</i> ) | Writes a subrange of <i>numChars</i> characters from the array <i>buffer</i> , beginning at <i>buffer[offset]</i> to the invoking output stream.                                                             |
| void write(String <i>str</i> )                                                              | Writes <i>str</i> to the invoking output stream.                                                                                                                                                             |
| void write(String <i>str</i> , int <i>offset</i> ,<br>int <i>numChars</i> )                 | Writes a subrange of <i>numChars</i> characters from the array <i>str</i> , beginning at the specified <i>offset</i> .                                                                                       |

**Table 17-4.**    *The Methods Defined by Writer*

# Console Class

- It was added to java.io in JDK 6
- Used to read from and write to the console , if one exists.
- Console is primarily a convenience class because most of its functionality is available through System.in and System.out
- Console supplies NO constructors. Instead, a Console object is obtained as follows
  - static Console console()
  - If console is available, then a reference to it is returned. Otherwise Null is returned.

| Method                                                                                | Description                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void flush( )</code>                                                            | Causes buffered output to be written physically to the console.                                                                                                                                                                                                                                                                               |
| <code>Console format(String <i>fmtString</i>,<br/>Object...<i>args</i>)</code>        | Writes <i>args</i> to the console using the format specified by <i>fmtString</i> .                                                                                                                                                                                                                                                            |
| <code>Console printf(String <i>fmtString</i>,<br/>Object...<i>args</i>)</code>        | Writes <i>args</i> to the console using the format specified by <i>fmtString</i> .                                                                                                                                                                                                                                                            |
| <code>Reader reader( )</code>                                                         | Returns a reference to a <b>Reader</b> connected to the console.                                                                                                                                                                                                                                                                              |
| <code>String readLine( )</code>                                                       | Reads and returns a string entered at the keyboard. Input stops when the user presses ENTER. If the end of the console input stream has been reached, <b>null</b> is returned. An <b>IOException</b> is thrown on failure.                                                                                                                    |
| <code>String readLine(String <i>fmtString</i>,<br/>Object...<i>args</i>)</code>       | Displays a prompting string formatted as specified by <i>fmtString</i> and <i>args</i> , and then reads and returns a string entered at the keyboard. Input stops when the user presses ENTER. If the end of the console input stream has been reached, <b>null</b> is returned. An <b>IOException</b> is thrown on failure.                  |
| <code>char[ ] readPassword( )</code>                                                  | Reads a string entered at the keyboard. Input stops when the user presses ENTER. The string is not displayed. If the end of the console input stream has been reached, <b>null</b> is returned. An <b>IOException</b> is thrown on failure.                                                                                                   |
| <code>char[ ] readPassword(String <i>fmtString</i>,<br/>Object... <i>args</i>)</code> | Displays a prompting string formatted as specified by <i>fmtString</i> and <i>args</i> , and then reads a string entered at the keyboard. Input stops when the user presses ENTER. The string is not displayed. If the end of the console input stream has been reached, <b>null</b> is returned. An <b>IOException</b> is thrown on failure. |
| <code>PrintWriter writer( )</code>                                                    | Returns a reference to a <b>Writer</b> connected to the console.                                                                                                                                                                                                                                                                              |